```
================================================================================
Main Execution
================================================================================

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"run_dir",

type=str,

help="run directory containing study outputs",

)

parser.add_argument(

"--output",

type=str,

default=None,
```

help="output directory for analysis (default: <run_dir>/analysis)",

)

parser.add_argument(

"--n-bootstrap",

type=int,

default=1000,

help="number of bootstrap iterations (default: 1000)",

)

parser.add_argument(

"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

```
help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")

if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")

return True

def main() -> None:

text

args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)
```

```
# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:

print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)
```

```
# Require exactly one input CSV and NPZ

if len(csv_files) != 1:

print(f"Expected exactly one scalar CSV, found {len(csv_files)}: {[[p.name](http://p.name) for p in csv_files]}")

sys.exit(1)

if len(npz_files) != 1:

print(f"Expected exactly one branch NPZ, found {len(npz_files)}: {[[p.name](http://p.name) for p in npz_files]}")

sys.exit(1)

if len(manifest_files) != 1:

print(f"Expected exactly one manifest, found {len(manifest_files)}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory with stale-file protection
```

```
if args.output is None:
output_dir = run_dir / "analysis"
else:
output_dir = Path(args.output)


# Check for stale files in output directory
expected_outputs = [
"twindow_paired_regime_contrasts.csv",
"twindow_paired_regime_contrasts_fixed_physical.csv",
"twindow_paired_regime_contrasts_normalized_EMA.csv",
"twindow_branch_interactions.csv",
"twindow_branch_interactions_fixed_physical.csv",
"twindow_branch_interactions_normalized_EMA.csv",
"twindow_cross_condition_topology.csv",
"twindow_recovery_by_seed.csv",
"twindow_recovery_summary.csv",
"twindow_regime_summary.csv",
```

```
"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_regime_secondary_summary.csv",

"twindow_branch_DID_secondary_summary.csv",

"twindow_recovery_interactions.csv",

"twindow_recovery_interactions_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]



stale_files = []

if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)
```

```
if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)


# If --overwrite is passed, remove ALL expected files before starting

if stale_files and args.overwrite:

print(f" Removing {len(stale_files)} stale files from output directory...")

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

try:

file_path.unlink()

except Exception as e:

print(f" WARNING: Could not remove stale file {expected_file}: {e}")



output_dir.mkdir(parents=True, exist_ok=True)
```

```
# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")

print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)
```

```
if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
```

```
rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")
```

```
regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)



# Stage 3: Branch DID (Primary outcomes - fixed_physical)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)
```

```
print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary branch DID into long format

branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)



# Stage 4b: Recovery Scaling (REQUIRED for production)

print("\nStage 4b: Computing recovery scaling metrics...")

recovery = compute_recovery_scaling(df, npz_path, b_c)
```

```
# Validate recovery groups have exactly 40 seeds each

print("\n Validating recovery group sizes...")

recovery_group_sizes = recovery.groupby(

["rho", "branch", "horizon_system", "metric"]

).size()

bad_groups = recovery_group_sizes[recovery_group_sizes != EXPECTED_N_SEEDS]

if not bad_groups.empty:

raise RuntimeError(

f"Recovery groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)



# Validate restricted recovery fields have finite values

print(" Validating restricted recovery fields...")

for field in RESTRICTED_RECOVERY_FIELDS:

non_finite = ~np.isfinite([pd.to](http://pd.to)_numeric(recovery[field], errors="coerce"))

if non_finite.any():
```

```
raise RuntimeError(
f"Recovery field '{field}' has {int(non_finite.sum())} non-finite values"
)


print(" ✅ Recovery validation passed")


# Stage 4c: Recovery Interaction DID
print("\nStage 4c: Computing recovery interaction DID...")
recovery_did = compute_recovery_did(recovery)


# Stage 5: Seed-Level Summaries
print("\nStage 5: Computing seed-level summaries...")
summaries = compute_seed_level_summaries(
regime_contrasts, branch_did, topology, recovery,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed,
)
```

```
# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")

secondary_summaries = compute_seed_level_summaries(

regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Validate recovery DID groups have exactly 40 seeds each

print("\n Validating recovery DID group sizes...")

recovery_did_group_sizes = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

).size()

if not (recovery_did_group_sizes == EXPECTED_N_SEEDS).all():

bad_groups = recovery_did_group_sizes[recovery_did_group_sizes != EXPECTED_N_SEEDS]

raise RuntimeError(
```

```
f"Recovery DID groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "
f"Bad groups: {bad_groups.head().to_dict()}"
)
print(" ✅ Recovery DID validation passed")


# Stage 5c: Recovery DID summaries
print("\nStage 5c: Computing recovery DID summaries...")
recovery_did_summaries = {}
if not recovery_did.empty:
recovery_did_summary = recovery_did.groupby(
["rho", "branch", "horizon_system", "metric", "outcome"],
as_index=False,
).agg(
mean=("recovery_difference_in_differences", "mean"),
std=("recovery_difference_in_differences", "std"),
n=("recovery_difference_in_differences", "count"),
)
```

```
# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):

values = group["recovery_difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;amp;

(recovery_did_summary["branch"] == branch) &amp;amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_did_summary["metric"] == metric) &amp;amp;
```

```
(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary

print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# Save outputs - construct output_files incrementally
```

```
print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)

output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")



# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)

regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"
```

```
save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")



# Branch DID (combined primary + secondary)

branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_combined_path, branch_did_combined)

output_files["branch_did_combined"] = branch_did_combined_path

print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")



# Branch DID (primary only - fixed_physical)

branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Branch DID (secondary only - normalized_EMA)
```

```
branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"

save_csv_atomic(branch_did_secondary_path, branch_did_secondary)

output_files["branch_did_secondary"] = branch_did_secondary_path

print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)

output_files["recovery_by_seed"] = recovery_raw_path
```

```
print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"
```

```
save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:

recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")



# Recovery DID (raw)

if not recovery_did.empty:

recovery_did_path = output_dir / "twindow_recovery_interactions.csv"

save_csv_atomic(recovery_did_path, recovery_did)

output_files["recovery_did"] = recovery_did_path

print(f" Saved: {recovery_did_[path.name](http://path.name)}")
```

```
# Recovery DID summaries

if not recovery_did_summary.empty:

recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"

save_csv_atomic(recovery_did_summary_path, recovery_did_summary)

output_files["recovery_did_summary"] = recovery_did_summary_path

print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")



# Secondary summaries

if not regime_summaries_secondary.empty:

regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"

save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)

output_files["regime_summary_secondary"] = regime_summary_secondary_path

print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")



if not did_summaries_secondary.empty:

did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"
```

```
save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)

output_files["did_summary_secondary"] = did_summary_secondary_path

print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")



# Text report

report = generate_text_report(

validation, regime_contrasts, branch_did, topology,

regime_summaries, did_summaries, topology_summaries,

recovery_summaries, regime_summaries_secondary, did_summaries_secondary,

recovery_did_summary

)

report_path = output_dir / "twindow_sensitivity_report.txt"

save_text_atomic(report_path, report)

output_files["report"] = report_path

print(f" Saved: {report_[path.name](http://path.name)}")



# Analysis manifest
```

```
analysis_manifest = generate_analysis_manifest(

csv_path, npz_path, manifest_path, validation, output_files,

source_path=Path(__file__),

bootstrap_seed=args.bootstrap_seed,

n_bootstrap=args.n_bootstrap,

command_line=command_line,

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)
```

print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()

Ignoring paste corruption, this version fixes most of the previously identified analysis issues. However, as currently ordered, it contains one immediate import-time failure.
Verdict

As pasted: NO-GO — the script raises NameError during import
After the constant-order fix: analysis logic is close to production-ready
Regime contrasts, branch DID, recovery DID structure: now correct
Remaining main concerns: input integrity, recovery target validation, and complete summary-count validation

P0: immediate import failure
1. NON_CONTINUE_BRANCHES is referenced before it is defined

At module scope:

Python

EXPECTED_N_RECOVERY_DID = (
EXPECTED_N_SEEDS
* (len(EXPECTED_RHO_VALUES) - 1)
* len(NON_CONTINUE_BRANCHES)

* 2 * 2 * 13
)

and:

Python

```
EXPECTED_N_RECOVERY_DID_GROUPS = (
(len(EXPECTED_RHO_VALUES) - 1)
* len(NON_CONTINUE_BRANCHES)
* 2 * 2 * 13
)
```

appear before:

Python

```
NON_CONTINUE_BRANCHES = (
"reset_ema",
"reset_B",
"future_shuffle",
)
```

Python evaluates these expressions while importing the module, so running the script—even with --help—will fail:

text

```
NameError: name 'NON_CONTINUE_BRANCHES' is not defined
```

py_compile may not catch this because it does not execute module-level expressions.
Fix

Move this definition above all expected-count calculations:

Python

```
EXPECTED_BRANCHES = (
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
)

NON_CONTINUE_BRANCHES = (
"reset_ema",
"reset_B",
"future_shuffle",
)

EXPECTED_N_CONDITIONS = (
EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)
)

EXPECTED_N_ROWS = (
EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)
)

EXPECTED_N_RECOVERY_DID = (
EXPECTED_N_SEEDS
* (len(EXPECTED_RHO_VALUES) - 1)
* len(NON_CONTINUE_BRANCHES)
* 2
* 2
* 13
)
```

Remove the later duplicate definition.
P0: scientific input validation
2. T_window is not validated before recovery normalization

Recovery uses:

Python

```
t_window = float(
```

```
df[
(df["seed"] == seed)
& (df["rho"] == rho)
]["T_window"].iloc[0]
)
```

but Stage 1 does not verify:

T_window exists;
it is finite and positive;
it is consistent across the four branches;
it equals the condition value recorded in the simulation manifest.

A corrupted T_window column would directly corrupt every trec/Twindowtrec/Twindow result.
Fix

Validate the manifest’s paired rho and TwindowTwindow values:

Python

```
manifest_rhos = [
float(x)
for x in manifest["study"]["rho_values"]
]
manifest_twindows = [
float(x)
for x in manifest["study"]["T_window_values"]
]

expected_twindow_by_rho = dict(
zip(manifest_rhos, manifest_twindows)
)
```

Then:

Python

```
if "T_window" not in df.columns:
```

```
results["status"] = "failed"
results["errors"].append(
"T_window missing from scalar CSV"
)
else:
tw = pd.to_numeric(
df["T_window"],
errors="coerce",
)

if (~np.isfinite(tw) | (tw <= 0)).any():
results["status"] = "failed"
results["errors"].append(
"T_window contains non-finite or non-positive values"
)

for rho, expected_twindow in expected_twindow_by_rho.items():
values = df.loc[
df["rho"] == rho,
"T_window",
].unique()

if len(values) != 1:
results["status"] = "failed"
results["errors"].append(
f"rho={rho} has {len(values)} distinct "
f"T_window values"
)
elif float(values[0]) != expected_twindow:
results["status"] = "failed"
results["errors"].append(
f"rho={rho}: T_window={values[0]} "
f"!= expected {expected_twindow}"
)
```

3. D_B_final is not validated, although it defines density targets

Density recovery uses:

Python

```
density_target_continue = (
0.9 * float(cont_row["D_B_final"])
)
```

Only primary and secondary late outcomes are currently validated. If D_B_final is NaN, threshold comparisons all become false, and restricted times can still appear finite because they are replaced by the censoring horizon.

Require:

Python

```
if "D_B_final" not in df.columns:
...
else:
density = pd.to_numeric(
df["D_B_final"],
errors="coerce",
)

invalid = (
~np.isfinite(density)
| (density < 0.0)
| (density > 1.0)
)

if invalid.any():
results["status"] = "failed"
results["errors"].append(
f"D_B_final has {int(invalid.sum())} "
f"invalid values"
)
```

The similarity target fields are now covered by primary/secondary finite validation.
P0: input integrity
4. Simulation output hashes still are not verified

The analysis computes hashes for its manifest, but Stage 1 does not compare the actual inputs against the simulation manifest’s recorded hashes.

A modified CSV, NPZ, condition-status file, or baseline certificate could therefore pass structural checks.

Assuming the corrected simulation driver records:

text

scalar_csv
branch_npz
condition_status_csv
baseline_certificate
baseline_seed_results

add:

Python

recorded_hashes = manifest.get("output_hashes", {})

files_to_verify = {
"scalar_csv": csv_path,
"branch_npz": npz_path,
"condition_status_csv": condition_status_path,
"baseline_certificate": baseline_cert_path,
}

for label, path in files_to_verify.items():
expected_hash = recorded_hashes.get(label)
observed_hash = file_sha256(path)

if expected_hash is None:
results["status"] = "failed"
results["errors"].append(
f"Simulation manifest missing output hash "
f"for {label}"

)
elif observed_hash != expected_hash:
results["status"] = "failed"
results["errors"].append(
f"Hash mismatch for {label}: "
f"expected={expected_hash}, "
f"observed={observed_hash}"
)

Do this before scientific calculations.

Also compare the baseline certificate’s seed keys with the simulation manifest’s exact paired-seed list, not only its count.
P1: expected group validation
5. Expected recovery group constants are defined but not used

You correctly validate that every observed recovery group contains 40 rows, but do not require exactly 80 groups.

A missing whole group is absent from recovery_group_sizes, so the existing test alone would not detect it.

Add:

Python

if len(recovery_group_sizes) != EXPECTED_N_RECOVERY_GROUPS:
raise RuntimeError(
f"Expected {EXPECTED_N_RECOVERY_GROUPS} "
f"recovery groups, got "
f"{len(recovery_group_sizes)}"
)

The expected number is:
5×4×2×2=80.
5×4×2×2=80.

Likewise for recovery DID:

Python

```
if (
len(recovery_did_group_sizes)
!= EXPECTED_N_RECOVERY_DID_GROUPS
):
raise RuntimeError(
f"Expected {EXPECTED_N_RECOVERY_DID_GROUPS} "
f"recovery DID groups, got "
f"{len(recovery_did_group_sizes)}"
)
```

The expected number is:
4×3×2×2×13=624.
4×3×2×2×13=624.

The exact 24,960 raw recovery-DID row check is already correct.
6. Validate regime, DID, and topology summary group sizes

Stage 1 makes the regime and ordinary DID group counts highly likely, but explicit checks improve the certificate.
Regime summaries

For each primary or secondary analysis:

text

```
4 non-baseline rho × 3 outcomes = 12 groups
40 seeds per group
```

Branch DID summaries

text

```
4 rho × 3 branches × 3 outcomes = 36 groups
40 seeds per group
```

Topology summaries

text

4 rho groups
40 finite values per metric per group

Add a finite check immediately after topology construction:

Python

```
TOPOLOGY_COLUMNS = (
"cross_T_cosine",
"cross_T_Jaccard",
"cross_T_normalized_distance",
)

for column in TOPOLOGY_COLUMNS:
values = pd.to_numeric(
topology[column],
errors="coerce",
)

if not np.isfinite(values).all():
raise RuntimeError(
f"Topology metric {column} contains "
f"non-finite values"
)

topology_sizes = topology.groupby("rho").size()

if len(topology_sizes) != 4:
raise RuntimeError(...)

if not (topology_sizes == EXPECTED_N_SEEDS).all():
raise RuntimeError(...)
```

Without this, 160 topology rows containing NaN metrics still satisfy the row-count check.

7. Validate topology matrices before computing metrics

For every loaded B_final, require:

Python

```
B.shape == (12, 12)
np.all(np.isfinite(B))
np.allclose(B, B.T, rtol=0, atol=1e-12)
np.allclose(np.diag(B), 0, rtol=0, atol=1e-12)
np.all(B >= -1e-12)
```

The normalized distance denominator uses the ρ=1ρ=1 matrix, which is the intended definition. A zero denominator should be treated as a production failure here, not merely emitted as NaN.
Recovery trajectory validation
8. Validate all three trajectory lengths against sample_times

Current checks compare:

text

```
branch density length versus continue density length
branch similarity length versus continue similarity length
```

but do not explicitly require:

Python

```
len(sample_times)
== len(density_trajectory)
== len(similarity_trajectory)
```

Add:

Python

```
if not (
```

len(sample_times)
== len(density_trajectory)
== len(similarity_trajectory)
):
raise RuntimeError(...)

Also require:

Python

len(sample_times) == 640
np.all(np.isfinite(sample_times))
np.all(np.diff(sample_times) > 0)

Preferably validate the exact expected schedule:

Python

expected_times = (
np.arange(0, 32000, 50, dtype=float) + 1.0
) * 0.01

if not np.array_equal(sample_times, expected_times):
raise RuntimeError(...)

9. Validate density and similarity trajectory ranges

Density should always be finite and in [0,1][0,1]:

Python

if not np.all(np.isfinite(density_trajectory)):
raise RuntimeError(...)

if np.any(
(density_trajectory < 0)
| (density_trajectory > 1)

):
raise RuntimeError(...)

Similarity may be NaN when the current BB vector is zero. For finite values require approximately:

Python

```
finite_similarity = similarity_trajectory[
np.isfinite(similarity_trajectory)
]

if np.any(
(finite_similarity < -1.0 - 1e-12)
| (finite_similarity > 1.0 + 1e-12)
):
raise RuntimeError(...)
```

Record the number of similarity NaNs imputed to zero for recovery/AUC calculations.
10. Add the zero-imputation convention to recovery rows

The AUC docstring now correctly describes the recovery-only convention, but the raw recovery CSV does not record how often it was used.

Add to similarity rows:

Python

```
"target_nan_samples_imputed_zero": int(
np.sum(~np.isfinite(similarity_trajectory))
),
"reference_nan_samples_imputed_zero": int(
np.sum(~np.isfinite(
cont_similarity_trajectory
))
),
"similarity_nan_convention": (
"undefined zero-vector cosine imputed as "
"zero for recovery and AUC only"
```

),

Add the same convention to the analysis manifest and text report.
Validation control flow
11. Manifest cohort validation failures are downgraded to warnings

Both condition-status and CSV exact-cohort checks use:

Python

except Exception as e:
results["warnings"].append(...)

For production analysis, inability to validate the cohort against the manifest must be fatal.

Change these to:

Python

results["status"] = "failed"
results["errors"].append(...)
return results

The manifest structure is mandatory, not optional.
12. Stale output deletion occurs before input validation

With --overwrite, the script deletes existing analysis outputs before calling validate_run().

If the new input run is malformed, a prior valid analysis can be destroyed before the failure is discovered.

Safer order:

locate exactly one CSV/NPZ/manifest;
run Stage 1 validation;
only after validation passes, inspect/create/clean the output directory;
run analysis.

This is operational rather than mathematical, but important for evidence preservation.
Summary/report details
13. The summary docstring still says standard error

compute_seed_level_summaries() says:

text

Std error

but computes sample standard deviation:

Python

std=("...", "std")

Change the documentation to “sample standard deviation.” The report’s mean ± std presentation is then accurate.
14. Conditional sustained/held CIs remain absent

The report checks for:

text

ci_lower_sustained
ci_upper_sustained
ci_lower_held
ci_upper_held

but those columns are never calculated.

This remains optional. Either:

bootstrap raw sustained and held times conditional on observation; or
remove the unused report checks.

Restricted-time and probability intervals already provide the primary censoring-aware summaries.
15. CSV atomic writing still lacks file fsync

save_csv_atomic() writes through pandas to a temporary path and replaces it, but only fsyncs the directory.

Use a file handle:

Python

```
with open(
temp_path,
"w",
encoding="utf-8",
newline="",
) as handle:
df.to_csv(handle, index=False)
handle.flush()
os.fsync(handle.fileno())
```

Then replace and fsync the directory.

This is nonblocking operational hardening.
Minor cleanup

b_c is unused in compute_recovery_scaling().
df is unused in compute_cross_t_topology().
MATRIX_KEYS is unused.
csv and top-level hashlib imports appear unused.
similarity_target = similarity_target is a no-op.
load_npz_selective() should raise on corrupt NPZ instead of warning and returning an empty dictionary.
Record these expected counts in the analysis manifest:

text

```
recovery rows: 3200
```

recovery groups: 80
recovery DID rows: 24960
recovery DID groups: 624

State explicitly in the report that density rows are duplicated across horizon systems by design and must not be pooled.
The discrete held definition should be documented as:

text

ceil(T_window / saved-sample interval)
consecutive saved samples

rather than an exact continuous-time guarantee.

What is now correct

The following previous issues are fixed:

normalized secondary outcomes are validated;
similarity target metadata is horizon-dependent;
density target metadata remains horizon-independent;
recovery-interaction files are protected against stale output mixing;
recovery DID excludes continue;
exact recovery-DID count is 24,960;
missing DID cells are fatal;
bootstrap arguments were removed from raw DID computation;
np.trapezoid is preferred;
every observed recovery and recovery-DID group must contain 40 seed rows;
baseline certificate contents and all seed statuses are checked;
baseline certificate hash is exposed for analysis provenance;
condition-status duplicate and exact-coverage checks are present;
exact CSV cohort coverage is checked;
primary and secondary analyses remain separated;
seed-level pairing and bootstrap resampling remain correct;
density rows remain separated by horizon system;
output stale-file protection now covers all generated filenames.

Bottom line

The first required correction is simply ordering:

Define NON_CONTINUE_BRANCHES before computing constants that reference it.

Without that, the script cannot start. After fixing it, validate TwindowTwindow, D_B_final, actual input hashes, and expected group counts before treating the analysis as freezeable. The core estimands themselves are now correctly implemented.

What is the work order and commands to test all three of these files?

Use a strict gated workflow. Stop immediately at the first failed stage. Do not use --overwrite on the first pass, and do not launch the grid until the baseline-only certificate is exact.
0. Apply the remaining source corrections first

At minimum, before testing:
Engine

require protocol.branches == manifest branches in mandatory baseline;
validate returned branch horizon metadata;
validate exact CSV (seed, branch) key coverage.

Driver

preferably move dry-run before mandatory frozen-path checks;
align the manifest matrix schema with engine.REQUIRED_BRANCH_MATRIX_KEYS;
validate rho, T_window, and recovery_end_ema.

Analysis

Move:

Python

NON_CONTINUE_BRANCHES = (
"reset_ema",
"reset_B",

"future_shuffle",
)

above every expected-count expression that uses it. Otherwise importing or running the analysis script raises NameError.
1. Set paths and preserve source hashes

Run from the directory containing the three scripts:

Bash

set -euo pipefail

export ENGINE="aief_experiment_engine_v1.py"
export DRIVER="AIEF_1_4_Twindow_sensitivity.py"
export ANALYSIS="AIEF_1_4_Twindow_sensitivity_analysis.py"

export FROZEN_DIR="runs/5a9f093a2fe0"
export FROZEN_MANIFEST="$FROZEN_DIR/manifest.json"
export FROZEN_NPZ="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz"
export FROZEN_SCALAR="$FROZEN_DIR/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv"

test -f "$ENGINE"
test -f "$DRIVER"
test -f "$ANALYSIS"
test -f "$FROZEN_MANIFEST"
test -f "$FROZEN_NPZ"
test -f "$FROZEN_SCALAR"

sha256sum \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS" \
> twindow_pretest_source_sha256.txt

cat twindow_pretest_source_sha256.txt

Do not edit the files after this point without restarting the test sequence.

2. Record and compare the environment

Bash

```
python3 --version

python3 - <<'PY'
import json
import platform
import numpy as np
import pandas as pd

with open("runs/5a9f093a2fe0/manifest.json", encoding="utf-8") as f:
frozen = json.load(f)

print("Current Python:", platform.python_version())
print("Current NumPy: ", np.__version__)
print("Current pandas:", pd.__version__)
print("Current OS: ", platform.platform())
print()
print("Frozen environment:")
print(json.dumps(frozen.get("environment", {}), indent=2))
PY
```

A NumPy or Python mismatch does not automatically invalidate the test, but exact replay failure under a changed numerical environment must not immediately be interpreted as a model-code failure.
3. Compile all three files

Bash

```
python3 -m py_compile \
"$ENGINE" \
"$DRIVER" \
"$ANALYSIS"
```

Note that py_compile does not execute module-level expressions, so it would not detect the previous NON_CONTINUE_BRANCHES ordering bug.
4. Import all three modules

Bash

```
python3 - <<'PY'
import aief_experiment_engine_v1
import AIEF_1_4_Twindow_sensitivity
import AIEF_1_4_Twindow_sensitivity_analysis

print("All three modules imported successfully.")
PY
```

This stage must pass before proceeding.
5. Run CLI/help/import validation

Bash

```
python3 "$ENGINE"

python3 "$DRIVER" --help >/dev/null
python3 "$ANALYSIS" --help >/dev/null

echo "Engine main and both CLI help commands passed."
```

6. Run cheap schema assertions

Bash

```
python3 - <<'PY'
import aief_experiment_engine_v1 as e
import AIEF_1_4_Twindow_sensitivity as d
import AIEF_1_4_Twindow_sensitivity_analysis as a

# Engine schema integrity.
assert len(e.EXTENDED_BRANCH_SCALAR_KEYS) == len(
set(e.EXTENDED_BRANCH_SCALAR_KEYS)
)
assert set(e.EXTENDED_BRANCH_SCALAR_KEYS) == set(
```

```
e.EXTENDED_SCALAR_EQUIVALENCE_MAP
)
assert set(e.EXTENDED_BRANCH_MATRIX_KEYS) == set(
e.EXTENDED_MATRIX_EQUIVALENCE_MAP
)

# Default configuration.
e.DEFAULT_MODEL_CONFIG.validate()
e.DEFAULT_PROTOCOL_CONFIG.validate(e.DEFAULT_MODEL_CONFIG)

# Driver/engine contract.
d.validate_engine_contract()
assert set(d.STUDY_BRANCHES) <= set(d.FROZEN_BRANCH_ORDER)
assert len(d.STUDY_BRANCHES) == len(set(d.STUDY_BRANCHES))
assert len(d.RHO_VALUES) == len(set(d.RHO_VALUES))
assert 1.0 in d.RHO_VALUES

# Analysis prespecified counts.
assert a.EXPECTED_N_CONDITIONS == 200
assert a.EXPECTED_N_ROWS == 800
assert a.EXPECTED_N_REGIME_CONTRASTS == 480
assert a.EXPECTED_N_DID == 1440
assert a.EXPECTED_N_TOPOLOGY == 160
assert a.EXPECTED_N_RECOVERY_DID == 24960
assert a.EXPECTED_N_RECOVERY_GROUPS == 80
assert a.EXPECTED_N_RECOVERY_DID_GROUPS == 624

print("Static schema and count assertions passed.")
PY
```

7. Exercise the analysis helper functions

Bash

```
python3 - <<'PY'
import numpy as np
import AIEF_1_4_Twindow_sensitivity_analysis as a

times = np.array([0.5, 1.0, 1.5, 2.0])
```

```
values = np.array([0.0, 0.8, 0.9, 1.0])

t, observed = a.first_crossing_time(times, values, 0.9)
assert observed and t == 1.5

t, observed = a.first_time_sustained_for_rest(
times, values, 0.8
)
assert observed and t == 1.0

auc = a.trapezoidal_auc_deficit(
times,
values,
values,
)
assert auc == 0.0

print("Analysis helper smoke tests passed.")
PY
```

8. Run the driver dry-run

The current pasted driver requires frozen paths before reaching dry-run, so supply them even if you later move the dry-run block:

Bash

```
python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--dry-run
```

Confirm that it reports:

text

```
5 rho values
```

4 study branches
40 seeds
configuration validated successfully

9. Optional but recommended: one-seed continue smoke replay

This runs only one prefix and one continue future before the expensive 40-seed baseline.

Bash

```
python3 - <<'PY'
import csv
import json
import math
import os

import numpy as np
import aief_experiment_engine_v1 as e

manifest_path = os.environ["FROZEN_MANIFEST"]
npz_path = os.environ["FROZEN_NPZ"]
csv_path = os.environ["FROZEN_SCALAR"]

with open(manifest_path, encoding="utf-8") as f:
manifest = json.load(f)

seed = int(manifest["randomization"]["paired_seeds"][0])
branch_order = tuple(manifest["protocol"]["branches"])

config = e.DEFAULT_MODEL_CONFIG
protocol = e.DEFAULT_PROTOCOL_CONFIG
config.validate()
protocol.validate(config)

split_state, prefix = e.simulate_prefix(
seed,
config,
protocol,
```

```
)

branch_rngs = e.make_branch_rngs(seed, branch_order)

result = e.simulate_future(
split_state,
prefix,
"continue",
config,
protocol,
branch_rngs["continue"],
)

e.finalize_historical_branch_metrics(
{"continue": result}
)

with np.load(npz_path, allow_pickle=False) as frozen:
for key in e.REQUIRED_PREFIX_KEYS:
frozen_key = f"prefix_seed_{seed}_{key}"
assert e.exact_array_equal_with_nan(
prefix[key],
frozen[frozen_key],
), frozen_key

for key in e.FROZEN_BRANCH_MATRIX_KEYS:
frozen_key = f"continue_seed_{seed}_{key}"
assert e.exact_array_equal_with_nan(
result[key],
frozen[frozen_key],
), frozen_key

for key, legacy in e.EXTENDED_MATRIX_EQUIVALENCE_MAP.items():
assert e.exact_array_equal_with_nan(
result[key],
result[legacy],
), f"{key} != {legacy}"

with open(csv_path, newline="", encoding="utf-8") as f:
```

```
rows = [
row for row in csv.DictReader(f)
if int(row["seed"]) == seed
and row["branch"] == "continue"
]


assert len(rows) == 1
frozen_row = rows[0]


for key in e.FROZEN_BRANCH_SCALAR_KEYS:
observed = float(result[key])
expected = float(frozen_row[key])


equal = (
math.isnan(observed) and math.isnan(expected)
) or observed == expected


assert equal, (
key,
observed,
expected,
)


for key, legacy in e.EXTENDED_SCALAR_EQUIVALENCE_MAP.items():
observed = float(result[key])
expected = float(result[legacy])


equal = (
math.isnan(observed) and math.isnan(expected)
) or observed == expected


assert equal, (
key,
legacy,
observed,
expected,
)


for key, expected in e.BASELINE_HORIZON_METADATA.items():
```

```
assert int(result[key]) == int(expected), (
key,
result[key],
expected,
)

print(f"One-seed continue exact smoke replay passed: {seed}")
PY
```

If this fails, do not launch the all-seed baseline.
10. Run the mandatory baseline-only test

Use a fresh root. Do not use --overwrite.

Bash

```
export BASELINE_ROOT="twindow_baseline_tests_20260809"

test ! -e "$BASELINE_ROOT"

set -o pipefail
time python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--baseline-only \
--run-root "$BASELINE_ROOT" \
2>&1 | tee twindow_baseline_only.log
```

Locate the generated run directory:

Bash

```
mapfile -t BASELINE_DIRS < <(
find "$BASELINE_ROOT" \
-mindepth 1 \
-maxdepth 1 \
-type d \
```

```
-print
)

test "${#BASELINE_DIRS[@]}" -eq 1
export BASELINE_DIR="${BASELINE_DIRS[0]}"

echo "Baseline directory: $BASELINE_DIR"
```

11. Audit the baseline certificate

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

root = Path(os.environ["BASELINE_DIR"])

with open(
root / "baseline_equivalence_certificate.json",
encoding="utf-8",
) as f:
cert = json.load(f)

with open(root / "manifest.json", encoding="utf-8") as f:
manifest = json.load(f)

assert cert["status"] == "passed"
assert cert["n_seeds"] == 40
assert len(cert["seed_results"]) == 40

expected_branches = [
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
```

```
"full_reset",
"future_shuffle",
]

assert cert["protocol"]["branches"] == expected_branches

for seed, result in cert["seed_results"].items():
assert result["status"] == "passed", seed

failed_checks = [
name
for name, passed in result.get("checks", {}).items()
if not passed
]
assert not failed_checks, (seed, failed_checks[:10])

assert cert.get("frozen_manifest_file_sha256")
assert cert.get("frozen_npz_file_sha256")
assert cert.get("frozen_scalar_csv_file_sha256")
assert cert.get("engine_source_sha256")

assert manifest["execution_mode"] == "baseline_only"
assert manifest["baseline_certified"] is True
assert manifest["freezeable"] is False
assert (
manifest["completion"]["status"]
== "baseline_only_completed"
)
assert manifest["completion"]["study_grid_started"] is False

assert not list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*_scalar_summary.csv"
)
)
assert not list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*_branch_diagnostics.npz"
)
)
```

print("Baseline-only certificate audit passed.")
PY

Also manually inspect:

Bash

cat "$BASELINE_DIR/manifest.json"

If any baseline check fails, stop. Do not reinterpret a partial or near-exact result as passage.
12. Confirm the source files did not change

Bash

sha256sum -c twindow_pretest_source_sha256.txt

All three must report OK.
13. Run the full five-level grid

Use a new production root, not the baseline-test root. The production driver should rerun its mandatory baseline and then launch the grid.

Bash

export PRODUCTION_ROOT="twindow_production_runs_20260809"

test ! -e "$PRODUCTION_ROOT"

set -o pipefail
time python3 "$DRIVER" \
--frozen-manifest "$FROZEN_MANIFEST" \
--frozen-npz "$FROZEN_NPZ" \
--frozen-scalar "$FROZEN_SCALAR" \
--run-root "$PRODUCTION_ROOT" \
2>&1 | tee twindow_full_grid.log

Locate its run directory:

Bash

```
mapfile -t PRODUCTION_DIRS < <(
find "$PRODUCTION_ROOT" \
-mindepth 1 \
-maxdepth 1 \
-type d \
-print
)

test "${#PRODUCTION_DIRS[@]}" -eq 1
export RUN_DIR="${PRODUCTION_DIRS[0]}"

echo "Production run directory: $RUN_DIR"
```

14. Audit the completed grid

Bash

```
python3 - <<'PY'
import hashlib
import json
import os
from pathlib import Path

import numpy as np
import pandas as pd

root = Path(os.environ["RUN_DIR"])

with open(root / "manifest.json", encoding="utf-8") as f:
manifest = json.load(f)

assert manifest["freezeable"] is True
assert manifest["baseline_equivalence"]["status"] == "passed"
```

```
completion = manifest["completion"]
assert completion["status"] == "completed"
assert completion["n_conditions_requested"] == 200
assert completion["n_conditions_successful"] == 200
assert completion["n_conditions_failed"] == 0
assert completion["n_scalar_rows"] == 800
assert completion["n_missing_primary_outcomes"] == 0
assert completion["n_missing_secondary_outcomes"] == 0


status = pd.read_csv(
root / "twindow_condition_status.csv",
dtype={"seed": str},
)
assert len(status) == 200
assert not status.duplicated(["seed", "rho"]).any()
assert (status["status"] == "success").all()


csv_files = list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*_scalar_summary.csv"
)
)
npz_files = list(
root.glob(
"AIEF_1_4_Twindow_sensitivity_*_branch_diagnostics.npz"
)
)


assert len(csv_files) == 1
assert len(npz_files) == 1


df = pd.read_csv(csv_files[0], dtype={"seed": str})


assert len(df) == 800
assert df["seed"].nunique() == 40
assert set(df["rho"].astype(float)) == {
0.5, 0.75, 1.0, 1.5, 2.0
}
assert set(df["branch"]) == {
"continue",
```

```
"reset_ema",
"reset_B",
"future_shuffle",
}
assert not df.duplicated(["seed", "rho", "branch"]).any()


primary = [
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"P_mean_late_fixed_physical",
]
secondary = [
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"P_mean_late_normalized_EMA",
]


for column in primary + secondary:
values = pd.to_numeric(df[column], errors="coerce")
assert np.isfinite(values).all(), column


def sha256(path):
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(
lambda: handle.read(1024 * 1024),
b"",
):
digest.update(block)
return digest.hexdigest()


hash_paths = {
"scalar_csv": csv_files[0],
"branch_npz": npz_files[0],
"condition_status_csv":
root / "twindow_condition_status.csv",
"baseline_certificate":
root / "baseline_equivalence_certificate.json",
"baseline_seed_results":
root / "baseline_equivalence_by_seed.json",
}
```

```
for label, path in hash_paths.items():
expected = manifest["output_hashes"][label]
observed = sha256(path)
assert expected == observed, label


with np.load(npz_files[0], allow_pickle=False) as data:
keys = set(data.files)


for seed in manifest["randomization"]["paired_seeds"]:
for rho in manifest["study"]["rho_values"]:
rho = float(rho)


for key in (
"B_at_split",
"history_at_split",
):
required = (
f"prefix_seed_{seed}_rho_{rho}_{key}"
)
assert required in keys, required


for branch in manifest["protocol"]["branches"]:
for key in (
"B_final",
"sample_times",
"density_trajectory",
"similarity_trajectory",
):
required = (
f"{branch}_seed_{seed}_rho_{rho}_{key}"
)
assert required in keys, required


print("Full-grid audit passed.")
PY
```

15. Run the analysis

Use a fresh analysis directory; omit --overwrite on the first analysis.

Bash

```
set -o pipefail
time python3 "$ANALYSIS" \
"$RUN_DIR" \
--bootstrap-seed 20260808 \
--n-bootstrap 5000 \
2>&1 | tee twindow_analysis.log
```

16. Audit the analysis outputs

Bash

```
python3 - <<'PY'
import json
import os
from pathlib import Path

import pandas as pd

analysis_dir = Path(os.environ["RUN_DIR"]) / "analysis"

expected_files = {
"twindow_paired_regime_contrasts.csv",
"twindow_paired_regime_contrasts_fixed_physical.csv",
"twindow_paired_regime_contrasts_normalized_EMA.csv",
"twindow_branch_interactions.csv",
"twindow_branch_interactions_fixed_physical.csv",
"twindow_branch_interactions_normalized_EMA.csv",
"twindow_cross_condition_topology.csv",
"twindow_recovery_by_seed.csv",
"twindow_recovery_summary.csv",
"twindow_recovery_interactions.csv",
"twindow_recovery_interactions_summary.csv",
"twindow_regime_summary.csv",
"twindow_branch_DID_summary.csv",
"twindow_topology_summary.csv",
```

```
"twindow_regime_secondary_summary.csv",
"twindow_branch_DID_secondary_summary.csv",
"twindow_sensitivity_report.txt",
"analysis_manifest.json",
}

missing = [
name
for name in expected_files
if not (analysis_dir / name).is_file()
]
assert not missing, missing

expected_rows = {
"twindow_paired_regime_contrasts.csv": 960,
"twindow_paired_regime_contrasts_fixed_physical.csv": 480,
"twindow_paired_regime_contrasts_normalized_EMA.csv": 480,
"twindow_branch_interactions.csv": 2880,
"twindow_branch_interactions_fixed_physical.csv": 1440,
"twindow_branch_interactions_normalized_EMA.csv": 1440,
"twindow_cross_condition_topology.csv": 160,
"twindow_recovery_by_seed.csv": 3200,
"twindow_recovery_summary.csv": 80,
"twindow_recovery_interactions.csv": 24960,
"twindow_recovery_interactions_summary.csv": 624,
"twindow_regime_summary.csv": 12,
"twindow_branch_DID_summary.csv": 36,
"twindow_topology_summary.csv": 4,
"twindow_regime_secondary_summary.csv": 12,
"twindow_branch_DID_secondary_summary.csv": 36,
}

for name, expected in expected_rows.items():
observed = len(pd.read_csv(analysis_dir / name))
assert observed == expected, (
name,
observed,
expected,
)

recovery = pd.read_csv(
```

```
analysis_dir / "twindow_recovery_by_seed.csv",
dtype={"seed": str},
)

sizes = recovery.groupby(
["rho", "branch", "horizon_system", "metric"]
).size()

assert len(sizes) == 80
assert (sizes == 40).all()

recovery_did = pd.read_csv(
analysis_dir / "twindow_recovery_interactions.csv",
dtype={"seed": str},
)

did_sizes = recovery_did.groupby(
["rho", "branch", "horizon_system", "metric", "outcome"]
).size()

assert len(did_sizes) == 624
assert (did_sizes == 40).all()
assert set(recovery_did["branch"]) == {
"reset_ema",
"reset_B",
"future_shuffle",
}

with open(
analysis_dir / "analysis_manifest.json",
encoding="utf-8",
) as f:
manifest = json.load(f)

assert manifest["validation"]["status"] == "passed"
assert manifest["bootstrap"]["seed"] == 20260808
assert manifest["bootstrap"]["n_iterations"] == 5000

print("Analysis-output audit passed.")
```

PY

17. Final source-integrity check

Bash

sha256sum -c twindow_pretest_source_sha256.txt

All three files must still report OK.
Final gate

Only after all stages pass should you:

freeze the production run directory;
freeze the analysis directory;
record the three source hashes;
update the evidence-lineage manifest;
proceed to interpretation.

The baseline-only run, full grid, and analysis are stages of one workflow. Repeated executions with the same seed cohort are reproducibility checks, not independent experiments.

We will do one final pass through evaluating each file before our seminal testing and running phase. Here is the first file (ignoring the copy paste errors) :

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

```
import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04
```

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = float(np.exp(-FROZEN_DT / FROZEN_T_WINDOW))

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

```
FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
# 8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END /
FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END /
FROZEN_DT)) # 8750
Baseline horizon metadata for validation

BASELINE_HORIZON_METADATA = {
```

text

"window_steps": FROZEN_WINDOW_STEPS,

"recovery_end_ema": FROZEN_RECOVERY_END,

"fixed_immediate_end": FIXED_PHYSICAL_IMMEDIATE_STEPS,

"fixed_recovery_end": FIXED_PHYSICAL_RECOVERY_STEPS,

}
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

def T_star(self) -> float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega

@property

def alpha(self) -> float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return float(np.exp(-self.dt / self.T_window))

@property

def capacity(self) -> float:

"""Projection capacity: (a - delta) / q"""

return (self.a - self.delta) / self.q

@property

def delay_steps(self) -> int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))

def validate(self) -> None:

"""Validate model configuration with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

All scalar parameters must be finite.

"""

```
# P0 #5: Validate N is a positive integer (not a float)
if (
isinstance(self.N, (bool, np.bool_))
or not isinstance(self.N, (int, np.integer))
or self.N &lt;= 0
):
raise ValueError("N must be a positive integer")

# Validate all other scalar parameters are finite
if not np.isfinite(self.dt) or self.dt &lt;= 0:
raise ValueError("dt must be finite and positive")
if not np.isfinite(self.tau) or self.tau &lt;= 0:
raise ValueError("tau must be finite and positive")
if not np.isfinite(self.a) or self.a &lt;= 0:
raise ValueError("a must be finite and positive")
if not np.isfinite(self.q) or self.q &lt;= 0:
raise ValueError("q must be finite and positive")
```

```
if not np.isfinite([self.delta](http://self.delta)) or [self.delta](http://self.delta) &lt; 0:

raise ValueError("delta must be finite and nonnegative")

if not np.isfinite(self.eta) or self.eta &lt; 0:

raise ValueError("eta must be finite and nonnegative")

if not np.isfinite([self.mu](http://self.mu)) or [self.mu](http://self.mu) &lt; 0:

raise ValueError("mu must be finite and nonnegative")

if not np.isfinite(self.gamma) or self.gamma &lt; 0:

raise ValueError("gamma must be finite and nonnegative")

if not np.isfinite(self.T_window) or self.T_window &lt;= 0:

raise ValueError("T_window must be finite and positive")

if not np.isfinite(self.B_c) or self.B_c &lt; 0:

raise ValueError("B_c must be finite and nonnegative")

if not np.isfinite(self.B_init_scale) or self.B_init_scale &lt; 0:

raise ValueError("B_init_scale must be finite and nonnegative")

if not np.isfinite(self.scale_epsilon) or self.scale_epsilon &lt;= 0:

raise ValueError("scale_epsilon must be finite and positive")


# Validate N is positive integer
```

if self.N <= 0:

raise ValueError("N must be positive")

Validate omega

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if not np.all(np.isfinite(self.omega)):

raise ValueError("omega must contain only finite values")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

Strict consistency: delay_steps * dt must equal tau (exact)

if not np.isclose(self.delay_steps * self.dt, self.tau, rtol=0.0, atol=0.0):

raise ValueError(

f"delay_steps *dt ({self.delay_steps}* {self.dt} = {self.delay_steps * self.dt}) "

f"must equal tau ({self.tau}) exactly"

)

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

```
def validate(self, model: ModelConfig) -&gt; None:

"""Validate protocol against model with strict consistency checks.


Uses rtol=0, atol=0 for exact compatibility with frozen production.

"""

# P0 #6: Validate protocol values are finite

if not np.all(np.isfinite([self.T_sim, self.burn_end_time, self.split_time, self.future_duration])):

raise ValueError("All protocol time parameters must be finite")


# P0 #6: Validate branches are non-empty and have no duplicates

if not self.branches:

raise ValueError("branches must be nonempty")

if len(self.branches) != len(set(self.branches)):

raise ValueError("protocol branches contain duplicates")


# P1 #10-11: Validation hardening - integer validation

# Validate diagnostic_stride is a positive integer (not bool)

if (
```

```
isinstance(self.diagnostic_stride, (bool, np.bool_))

or not isinstance(self.diagnostic_stride, (int, np.integer))

or self.diagnostic_stride &lt;= 0

):

raise ValueError("diagnostic_stride must be a positive integer")


if self.T_sim &lt;= 0:

raise ValueError("T_sim must be positive")

if not (0 &lt; self.burn_end_time &lt; self.split_time &lt; self.T_sim):

raise ValueError("Invalid time parameters: must have 0 &lt; burn_end &lt; split &lt; T_sim")

if self.future_duration &lt;= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride &lt;= 0:

raise ValueError("diagnostic_stride must be positive")


# Strict consistency: split_time + future_duration must equal T_sim

if not np.isclose(self.split_time + self.future_duration, self.T_sim, rtol=0.0, atol=0.0):

raise ValueError(
```

```
f"split_time ({self.split_time}) + future_duration ({self.future_duration}) "

f"= {self.split_time + self.future_duration} must equal T_sim ({self.T_sim}) exactly"

)



# Validate that all time parameters are compatible with dt (exact divisibility)

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=0.0,
atol=0.0):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})
exactly")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=0.0,
atol=0.0):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt})
exactly")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(
```

```
text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
```

Default protocol matching frozen run

```
DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(
```

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==
Cached Utility Values
==
Cache for compute_edges to avoid recomputing on every step

EDGESCACHE: dict[int, tuple[np.ndarray, np.ndarray]] = {}
Cache for omega array conversion

OMEGAARRAY_CACHE: dict[tuple[float, ...], np.ndarray] = {}
==
Utility Functions
==

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges.

Cached to avoid recomputing on every step.

"""

if N not in *EDGES*CACHE:

*EDGES*CACHE[N] = np.triu_indices(N, k=1)

return *EDGES*CACHE[N]

def get_omega_array(omega_tuple: tuple[float, ...]) -> np.ndarray:

text

"""Get omega as numpy array with caching.

Cached to avoid recomputing on every step.

"""

if omega_tuple not in *OMEGA*ARRAY_CACHE:

*OMEGA*ARRAY_CACHE[omega_tuple] = np.asarray(omega_tuple, dtype=float)

return *OMEGA*ARRAY_CACHE[omega_tuple]

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B < -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: handle.read(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

```
def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp")

try:
```

```
with os.fdopen(fd, "w", encoding="utf-8") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(str(tmp_path), str(path))
finally:
if os.path.exists(tmp_path):
os.remove(tmp_path)
def atomic_write_json(path: Path, data: dict[str, Any]) -> None:
text
"""Write JSON atomically with fsync."""
text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"
atomic_write_text(path, text)
def atomic_write_npz(path: Path, **kwargs: Any) -> None:
text
"""Write NPZ atomically with fsync."""
path.parent.mkdir(parents=True, exist_ok=True)
```

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{path.name}.", suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

==========
RNG Utilities (matching frozen production exactly)
==========

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].

This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.

Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

==========
Projection Accumulator (matching production)
==========

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -> None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

```
rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -&gt; dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations &gt; 0

else float('nan'),
```

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()

Handle history input - ensure it's a list of arrays

```
if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)



@property

def N(self) -&gt; int:
```

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency.

Checks:

- State dimension matches config.N

- History length is exactly delay_steps + 1

- All arrays have correct shapes

- B matrix is symmetric, zero-diagonal, non-negative

- All history vectors have shape (N,)

- All arrays are finite

"""

Check state dimension matches config

if self.N != config.N:

raise RuntimeError(

f"State N ({self.N}) does not match config N ({config.N})"

```
)

# Check history length
expected_history_len = config.delay_steps + 1
if len(self.history) != expected_history_len:
raise RuntimeError(
f"History length {len(self.history)} != expected {expected_history_len} "
f"(delay_steps={config.delay_steps}) at step {step}"
)

# Check all history vectors have correct shape
for i, h in enumerate(self.history):
if h.shape != (self.N,):
raise RuntimeError(
f"History[{i}] shape {h.shape} != ({self.N},)"
)

# Check theta shape
if self.theta.shape != (self.N,):
```

```
raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")


# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")


# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")


# Check all arrays are finite

if not np.all(np.isfinite(self.theta)):

raise RuntimeError("theta contains non-finite values")

if not np.all(np.isfinite(self.B)):

raise RuntimeError("B contains non-finite values")

if not np.all(np.isfinite(self.coherence_acc)):

raise RuntimeError("coherence_acc contains non-finite values")

for i, h in enumerate(self.history):
```

```
if not np.all(np.isfinite(h)):

raise RuntimeError(f"History[{i}] contains non-finite values")



# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")



# Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")



# Check B non-negativity

if np.any(self.B &lt; -1e-12):

raise RuntimeError("B matrix has negative entries")

============================================================================
====
Simulation Core (exact reproduction of frozen production)
============================================================================
====

def initialize_state(

text
```

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.

Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)

Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi

Initialize B: small random symmetric matrix

```
B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)



# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(
```

```
text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]



# Convert omega to numpy array for indexing (cached)

omega_arr = get_omega_array([config.omega](http://config.omega))



dphi = (

state.theta[:, None]
```

```
- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(
```

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values

```
# Update B
B_unprojected = state.B + config.dt * (
(config.eta + [config.mu](http://config.mu)) * R_driver
- [config.mu](http://config.mu)
- config.gamma * state.B
)
B_unprojected = np.maximum(B_unprojected, 0.0)
B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)
np.fill_diagonal(B_unprojected, 0.0)


# Projection diagnostics (matching production exactly)
proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))


# Check gap constraint
row_sum_pre = np.sum(B_unprojected, axis=1)
g_min_pre = float(np.min(config.a - config.q * row_sum_pre))
```

pre_violation = bool(g_min_pre < config.delta)

Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))

projection_active = bool(correction_norm > FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)

```
# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = get_omega_array([config.omega](http://config.omega))

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)
```

Update history

state.history.append(state.theta.copy())

if len(state.history) > config.delay_steps + 1:

state.history.pop(0)

info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

```
unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)



# Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)



# Get RNG for prefix simulation loop

# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0
```

```
burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)


# P1 #10-11: State validation at transition boundaries

if step == burn_end or step == hist_end - 1:

state.validate(config, step)


if burn_end &lt;= step &lt; hist_end:

R_past_sum += R_physical

R_past_count += 1



prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),
```

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.

Exact reproduction of production apply_intervention().

"""

Validate branch is known

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}")

N = state.N

if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

Zero coherence accumulator

state.coherence_acc.fill(0.0)

if branch in {"reset_B", "reset_B_ema", "full_reset"}:

Zero B

state.B.fill(0.0)

if branch in {"B_carrier_only", "full_reset"}:

```
# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:
```

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)

Fixed physical horizons (NOT T_window dependent)

These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

```
# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),
```

```
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
}

# B matrices (mean B values)
B_sums_ema = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
}
B_sums_fixed = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
```

```
}


# Counts for each horizon in both systems
counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}
counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}


# R matrices
R_physical_sum = np.zeros((N, N), dtype=float)
R_driver_sum = np.zeros((N, N), dtype=float)


# Projection accumulator (matching production)
projection = ProjectionTotals()


# Trajectory accumulators
sample_times = []
density_trajectory = []
```

```
similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):

R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)
```

Update projection accumulator

projection.update(info)

P1 #10-11: State validation at transition boundaries

if step == 0 or step == window_steps or step == recovery_end_ema or step == fixed_immediate_end or step == fixed_recovery_end:

state.validate(config, step)

Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver

Determine which horizons this step belongs to for BOTH systems

selected = (state.B > config.B_c).astype(float)

=== EMA-Normalized Horizons ===

Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1

recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end <= step < fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1

```
# late: fixed_recovery_end to end

if fixed_recovery_end &lt;= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1


# Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]
```

```
norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:

r_late_ema_sum += r_value

r_late_ema_count += 1


if fixed_recovery_end &lt;= step:
```

r_late_fixed_sum += r_value

r_late_fixed_count += 1

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

```
R_physical_mean = R_physical_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))



# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')
```

```
)
emergence_ema = (
float(np.mean(p_late_edges_ema[~split_selected]))
if np.any(~split_selected)
else float('nan')
)

# Fixed physical
p_late_edges_fixed = P_fixed["late"][edges]
retention_fixed = (
float(np.mean(p_late_edges_fixed[split_selected]))
if np.any(split_selected)
else float('nan')
)
emergence_fixed = (
float(np.mean(p_late_edges_fixed[~split_selected]))
if np.any(~split_selected)
```

```
else float('nan')

)



# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)



# Build result with BOTH horizon systems

result = {

# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],
```

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

=== Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),

Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

**projection.metrics(),

Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

}

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) & np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) < 3 or np.std(x) < 1e-12 or np.std(y) < 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total <= 1e-12 or sum_sq <= 1e-12:

return 0.0

```
return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total &lt;= 1e-12:

return 0.0
```

```
probabilities = weights / total

positive = probabilities &gt; 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

def time_to_at_least(

text

sample_times: np.ndarray,

values: np.ndarray,

target: float,

) -> float:

text

"""Find the first time at which values reach at least the target.



Matches frozen production exactly.

Returns NaN if target is never reached.

"""

sample_times = np.asarray(sample_times, dtype=float)
```

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values >= target))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def time_to_below(

text

sample_times: np.ndarray,

values: np.ndarray,

threshold: float,

) -> float:

text

"""Find the first time at which values drop below the threshold.

Matches frozen production exactly.

Returns NaN if threshold is never dropped below.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values < threshold))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def finalize_historical_branch_metrics(

text

branch_results: dict[str, dict[str, Any]],

) -> None:

text

"""Compute and add the 8 historical scalar fields that compare branches to continue.

These fields are computed in frozen production by run_seed() after all branches

are simulated. They must be computed here to match the frozen schema.

The 8 fields are:

- Time_to_50pct_continue_final_density

- Time_to_90pct_continue_final_density

- Time_to_50pct_continue_late_similarity

- Time_to_90pct_continue_late_similarity

- Time_to_similarity_below_0_9

- Time_to_similarity_below_0_5

- Late_persistence_ratio_to_continue

- Late_similarity_ratio_to_continue

These are computed for each branch relative to the continue branch reference values.

"""

Get continue branch results for reference

if "continue" not in branch_results:

raise RuntimeError("continue branch is required for historical metric computation")

continue_result = branch_results["continue"]

Get reference values from continue branch

reference_density = float(continue_result["D_B_final"])

```
reference_late_similarity = float(continue_result["Weighted_similarity_late"])

reference_late_persistence = float(continue_result["P_mean_late"])



for branch, result in branch_results.items():

# Use direct indexing to fail if keys are missing

times = result["sample_times"]

density = result["density_trajectory"]

similarity = result["similarity_trajectory"]


# Compute time-based metrics relative to continue branch reference


# Compute time-based metrics relative to continue branch reference

result["Time_to_50pct_continue_final_density"] = time_to_at_least(

times, density, 0.5 * reference_density

)

result["Time_to_90pct_continue_final_density"] = time_to_at_least(

times, density, 0.9 * reference_density

)
```

```
result["Time_to_50pct_continue_late_similarity"] = time_to_at_least(
times, similarity, 0.5 * reference_late_similarity
)
result["Time_to_90pct_continue_late_similarity"] = time_to_at_least(
times, similarity, 0.9 * reference_late_similarity
)
result["Time_to_similarity_below_0_9"] = time_to_below(
times, similarity, 0.9
)
result["Time_to_similarity_below_0_5"] = time_to_below(
times, similarity, 0.5
)

# Compute ratio metrics - use direct indexing to fail if missing
branch_late_persistence = float(result["P_mean_late"])
branch_late_similarity = float(result["Weighted_similarity_late"])

# P0 #8: Restore literal frozen persistence-ratio logic
```

```
# Frozen production uses: if abs(reference_late_persistence) &gt; ZERO_TOL

# NOT: if np.isfinite(reference_late_persistence) and abs(...) &gt; ZERO_TOL

# For persistence ratio, use the exact frozen condition

if abs(reference_late_persistence) &gt; FROZEN_ZERO_TOL:

result["Late_persistence_ratio_to_continue"] = (

branch_late_persistence / reference_late_persistence

)

else:

result["Late_persistence_ratio_to_continue"] = float('nan')


# For similarity ratio, keep the np.isfinite() check as frozen source uses it there

if np.isfinite(reference_late_similarity) and abs(reference_late_similarity) &gt; FROZEN_ZERO_TOL:

result["Late_similarity_ratio_to_continue"] = (

branch_late_similarity / reference_late_similarity

)

else:

result["Late_similarity_ratio_to_continue"] = float('nan')
```

==========
Required Output Schema for Baseline Equivalence
==========
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
Extended matrix equivalence map for internal consistency validation at rho=1
Maps extended matrix keys to their frozen equivalents

EXTENDED_MATRIX_EQUIVALENCE_MAP = {

text

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS +
EXTENDED_BRANCH_MATRIX_KEYS
P0 #7: Matrix schema-map integrity should mirror scalar integrity
Validate no duplicates in extended matrix keys

if len(EXTENDED_BRANCH_MATRIX_KEYS) !=
len(set(EXTENDED_BRANCH_MATRIX_KEYS)):

text

duplicates = [k for k in EXTENDED_BRANCH_MATRIX_KEYS if
list(EXTENDED_BRANCH_MATRIX_KEYS).count(k) > 1]

raise RuntimeError(f"Duplicate extended matrix schema keys: {sorted(set(duplicates))}")

Validate matrix map consistency

if set(EXTENDED_BRANCH_MATRIX_KEYS) !=
set(EXTENDED_MATRIX_EQUIVALENCE_MAP.keys()):

text

missing_map = set(EXTENDED_BRANCH_MATRIX_KEYS) -
set(EXTENDED_MATRIX_EQUIVALENCE_MAP.keys())

extra_map = set(EXTENDED_MATRIX_EQUIVALENCE_MAP.keys()) -
set(EXTENDED_BRANCH_MATRIX_KEYS)

```
raise RuntimeError(

f"Extended matrix map mismatch: "

f"missing={sorted(missing_map)}, extra={sorted(extra_map)}"

)

FROZEN BRANCH SCALAR KEYS: All 42 historical fields from frozen production
These MUST exist in the frozen CSV and are validated against it

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",
```

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Historical cross-branch time-based metrics (from frozen production)

"Time_to_50pct_continue_final_density",

"Time_to_90pct_continue_final_density",

"Time_to_50pct_continue_late_similarity",

"Time_to_90pct_continue_late_similarity",

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
EXTENDED BRANCH SCALAR KEYS: New fields in v1 for dual-horizon support
These do NOT exist in frozen CSV/NPZ, so are validated internally at rho=1

```
EXTENDED_BRANCH_SCALAR_KEYS = (

text

# Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

# Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

# Additional extended scalars for complete coverage

"P_mean_full_fixed_physical",

"P_mean_immediate_fixed_physical",

"P_mean_recovery_fixed_physical",
```

"P_mean_full_normalized_EMA",

"P_mean_immediate_normalized_EMA",

"P_mean_recovery_normalized_EMA",

"Retention_selected_late_fixed_physical",

"Emergence_unselected_late_fixed_physical",

"Retention_selected_late_normalized_EMA",

"Emergence_unselected_late_normalized_EMA",

"Corr_Rpast_Pfull_fixed_physical",

"Corr_Rpast_Pfull_normalized_EMA",

"Jaccard_final_fixed_physical",

)
Extended scalar equivalence map for internal consistency validation at rho=1
Maps extended keys to their frozen equivalents

EXTENDED_SCALAR_EQUIVALENCE_MAP = {

text

"P_mean_full_fixed_physical": "P_mean_full",

"P_mean_immediate_fixed_physical": "P_mean_immediate",

"P_mean_recovery_fixed_physical": "P_mean_recovery",

"P_mean_late_fixed_physical": "P_mean_late",

"P_mean_full_normalized_EMA": "P_mean_full",

"P_mean_immediate_normalized_EMA": "P_mean_immediate",

"P_mean_recovery_normalized_EMA": "P_mean_recovery",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"Corr_Rpast_Pfull_fixed_physical": "Corr_Rpast_Pfull",

"Corr_Rpast_Pfull_normalized_EMA": "Corr_Rpast_Pfull",

"r_mean_late_fixed_physical": "r_mean_late",

"r_mean_late_normalized_EMA": "r_mean_late",

"Retention_selected_late_fixed_physical": "Retention_selected_late",

"Retention_selected_late_normalized_EMA": "Retention_selected_late",

"Emergence_unselected_late_fixed_physical": "Emergence_unselected_late",

"Emergence_unselected_late_normalized_EMA": "Emergence_unselected_late",

"Jaccard_final_fixed_physical": "Jaccard_final",

}
Validate no duplicates in extended scalar keys

if len(EXTENDED_BRANCH_SCALAR_KEYS) !=
len(set(EXTENDED_BRANCH_SCALAR_KEYS)):

text

duplicates = [k for k in EXTENDED_BRANCH_SCALAR_KEYS if
list(EXTENDED_BRANCH_SCALAR_KEYS).count(k) > 1]

raise RuntimeError(f"Duplicate extended scalar schema keys: {sorted(set(duplicates))}")

Validate schema-map consistency

if set(EXTENDED_BRANCH_SCALAR_KEYS) !=
set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys()):

text

missing_map = set(EXTENDED_BRANCH_SCALAR_KEYS) -
set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys())

extra_map = set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys()) -
set(EXTENDED_BRANCH_SCALAR_KEYS)

raise RuntimeError(

f"Extended scalar map mismatch: "

f"missing mappings={sorted(missing_map)}, extra mappings={sorted(extra_map)}"

)

All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

```
from pathlib import Path

import hashlib



# Preflight validation: validate config and protocol before any simulation

try:

config.validate()

except Exception as e:

return {

"status": "failed",

"error": f"Config validation failed: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}



try:

protocol.validate(config)
```

except Exception as e:

return {

"status": "failed",

"error": f"Protocol validation failed: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}

Helper function to compute source hash

def *source*sha256() -> str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

P1 #6: Preflight all authority files - check existence before opening

```
# Check frozen manifest exists

if not os.path.isfile(frozen_manifest_path):

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}



# Check frozen NPZ exists

if not os.path.isfile(frozen_npz_path):

return {

"status": "failed",

"error": f"Frozen NPZ not found: {frozen_npz_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,
```

```
"frozen_csv_path": frozen_csv_path,

}



# Early failure: frozen scalar CSV is required for scalar equivalence

if frozen_csv_path is None:

return {

"status": "failed",

"error": "frozen scalar CSV is required for scalar equivalence",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": None,

}



# Check frozen CSV exists (if provided)

if frozen_csv_path is not None and not os.path.isfile(frozen_csv_path):

return {

"status": "failed",
```

```
"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Record engine source hash

engine_source_hash = *source*sha256()



results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,
```

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

```
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
"diagnostic_stride": protocol.diagnostic_stride,
"branches": list(protocol.branches),
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"engine_source_sha256": engine_source_hash,
"seed_results": {},
}



# P0 #9: Catch malformed manifest - wrap in try/except
try:
# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
frozen_manifest = json.load(f)
```

```
# Validate required manifest structure before indexing
if "randomization" not in frozen_manifest:
return {
"status": "failed",
"error": "Frozen manifest missing 'randomization' section",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
if "protocol" not in frozen_manifest:
return {
"status": "failed",
"error": "Frozen manifest missing 'protocol' section",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
```

```
if "model" not in frozen_manifest:
return {
"status": "failed",
"error": "Frozen manifest missing 'model' section",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
if "schema" not in frozen_manifest:
return {
"status": "failed",
"error": "Frozen manifest missing 'schema' section",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}

if "paired_seeds" not in frozen_manifest["randomization"]:
```

```
return {
"status": "failed",
"error": "Frozen manifest missing 'randomization.paired_seeds'",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
if "branches" not in frozen_manifest["protocol"]:
return {
"status": "failed",
"error": "Frozen manifest missing 'protocol.branches'",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}

# P0 #10: Compare manifest schema to expected frozen schema
# Validate schema structure
```

```
manifest_schema = frozen_manifest.get("schema", {})


manifest_prefix = tuple(manifest_schema.get("prefix_keys", []))

manifest_matrices = tuple(manifest_schema.get("branch_matrix_keys", []))

manifest_scalars = tuple(manifest_schema.get("branch_scalar_keys", []))


# Validate prefix keys

if set(manifest_prefix) != set(REQUIRED_PREFIX_KEYS):

missing_prefix = set(REQUIRED_PREFIX_KEYS) - set(manifest_prefix)

extra_prefix = set(manifest_prefix) - set(REQUIRED_PREFIX_KEYS)

return {

"status": "failed",

"error": (

f"Manifest prefix_keys mismatch: "

f"missing={sorted(missing_prefix)}, extra={sorted(extra_prefix)}"

),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),
```

```
"frozen_csv_path": frozen_csv_path,

}


# Validate matrix keys - compare unique sets

if set(manifest_matrices) != set(FROZEN_BRANCH_MATRIX_KEYS):

missing_matrices = set(FROZEN_BRANCH_MATRIX_KEYS) - set(manifest_matrices)

extra_matrices = set(manifest_matrices) - set(FROZEN_BRANCH_MATRIX_KEYS)

return {

"status": "failed",

"error": (

f"Manifest branch_matrix_keys mismatch: "

f"missing={sorted(missing_matrices)}, extra={sorted(extra_matrices)}"

),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}
```

```
# Validate scalar keys - compare unique sets
if set(manifest_scalars) != set(FROZEN_BRANCH_SCALAR_KEYS):
missing_scalars = set(FROZEN_BRANCH_SCALAR_KEYS) - set(manifest_scalars)
extra_scalars = set(manifest_scalars) - set(FROZEN_BRANCH_SCALAR_KEYS)
return {
"status": "failed",
"error": (
f"Manifest branch_scalar_keys mismatch: "
f"missing={sorted(missing_scalars)}, extra={sorted(extra_scalars)}"
),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
except (json.JSONDecodeError, OSError) as e:
return {
"status": "failed",
"error": f"Malformed frozen manifest {frozen_manifest_path}: {e}",
```

```
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}


# P0 #4: Validate seed/branch authority - seeds and branches must match manifest
manifest_seeds = [int(x) for x in frozen_manifest["randomization"]["paired_seeds"]]
manifest_branches = tuple(frozen_manifest["protocol"]["branches"])
if list(seeds) != manifest_seeds:
return {
"status": "failed",
"error": f"Seed authority mismatch: seeds={seeds}, manifest={manifest_seeds}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
if tuple(branch_order) != manifest_branches:
```

```
return {

"status": "failed",

"error": f"Branch authority mismatch: branch_order={branch_order}, manifest={manifest_branches}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #1: Validate protocol.branches matches manifest branches

# This is critical: branch_order defines RNG namespace, but protocol.branches defines executed branches

# Both must match the frozen manifest to prevent false-pass

if tuple(protocol.branches) != manifest_branches:

return {

"status": "failed",

"error": (

"Executed baseline branch mismatch: "
```

```
f"protocol.branches={protocol.branches}, "

f"manifest={manifest_branches}"

),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #3: Validate baseline horizon metadata

# At rho=1 (T_window = T_star), horizon metadata must match frozen values

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

expected_metadata = BASELINE_HORIZON_METADATA

actual_window_steps = int(round(config.T_window / config.dt))

actual_recovery_end_ema = min(5 * actual_window_steps, protocol.future_steps(config))

actual_fixed_immediate_end = int(round(17.5 / config.dt))

actual_fixed_recovery_end = int(round(87.5 / config.dt))



if actual_window_steps != expected_metadata["window_steps"]:
```

```
return {

"status": "failed",

"error": f"window_steps mismatch: actual={actual_window_steps},
expected={expected_metadata['window_steps']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_recovery_end_ema != expected_metadata["recovery_end_ema"]:

return {

"status": "failed",

"error": f"recovery_end_ema mismatch: actual={actual_recovery_end_ema},
expected={expected_metadata['recovery_end_ema']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_fixed_immediate_end != expected_metadata["fixed_immediate_end"]:
```

```
return {

"status": "failed",

"error": f"fixed_immediate_end mismatch: actual={actual_fixed_immediate_end},
expected={expected_metadata['fixed_immediate_end']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_fixed_recovery_end != expected_metadata["fixed_recovery_end"]:

return {

"status": "failed",

"error": f"fixed_recovery_end mismatch: actual={actual_fixed_recovery_end},
expected={expected_metadata['fixed_recovery_end']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}




# P0 #9: Catch malformed NPZ - wrap in try/except
```

```
try:
# Load frozen data
with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:
frozen_files = dict(frozen_data)
except Exception as e:
return {
"status": "failed",
"error": f"Malformed frozen NPZ {frozen_npz_path}: {e}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}


# P1 #7: Preflight NPZ schema - validate expected keys exist
# Check prefix keys for all seeds
for seed in seeds:
seed_str = str(seed)
```

```
for key in REQUIRED_PREFIX_KEYS:

frozen_key = f"prefix_seed_{seed_str}_{key}"

if frozen_key not in frozen_files:

return {

"status": "failed",

"error": f"Missing NPZ key: {frozen_key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Check branch matrix keys for all seeds and branches

for seed in seeds:

seed_str = str(seed)

for branch in branch_order:

for key in FROZEN_BRANCH_MATRIX_KEYS:

frozen_key = f"{branch}_seed_{seed_str}_{key}"
```

```
if frozen_key not in frozen_files:
return {
"status": "failed",
"error": f"Missing NPZ key: {frozen_key}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}


# Load frozen scalar CSV - REQUIRED for scalar equivalence
# Fail immediately if CSV is missing or invalid
if frozen_csv_path is None or not os.path.isfile(frozen_csv_path):
return {
"status": "failed",
"error": f"Frozen scalar CSV not found: {frozen_csv_path}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
```

"frozen_csv_path": frozen_csv_path,

}

frozen_scalar_csv: dict[str, dict[str, Any]] = {}

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

P1 #5: CSV duplicate detection - fail if duplicate (seed, branch) rows

if key in frozen_scalar_csv:

return {

"status": "failed",

```
"error": f"Duplicate CSV row detected for {key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

return {

"status": "failed",

"error": f"Could not load frozen CSV {frozen_csv_path}: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
```

```
"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #3: Validate exact CSV key coverage

# Expected: all seeds × all branch_order combinations

expected_csv_keys = {

f"{branch}_seed_{seed}"

for seed in seeds

for branch in branch_order

}

actual_csv_keys = set(frozen_scalar_csv.keys())



if actual_csv_keys != expected_csv_keys:

missing = sorted(expected_csv_keys - actual_csv_keys)

extra = sorted(actual_csv_keys - expected_csv_keys)

return {
```

```
"status": "failed",

"error": (

"Frozen CSV condition-key mismatch: "

f"{len(missing)} missing, {len(extra)} extra"

),

"missing_csv_keys": missing,

"extra_csv_keys": extra,

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Validate all expected scalar columns exist in every CSV row

for csv_key, row_values in frozen_scalar_csv.items():

missing = set(FROZEN_BRANCH_SCALAR_KEYS) - set(row_values)

if missing:

return {
```

"status": "failed",

"error": f"Frozen CSV row {csv_key} missing {len(missing)} required fields",

"missing_columns": sorted(missing),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

Validate frozen CSV has all expected seeds and branches

Expected: 40 seeds × 7 branches = 280 rows

expected_csv_rows = len(seeds) * len(branch_order)

actual_csv_rows = len(frozen_scalar_csv)

if actual_csv_rows != expected_csv_rows:

return {

"status": "failed",

"error": f"Frozen CSV has {actual_csv_rows} rows, expected {expected_csv_rows} (40 seeds × 7 branches)",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

```
"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Record frozen hashes - use actual file hashes, not canonical JSON hashes

# Record frozen hashes - use actual file hashes, not canonical JSON hashes

results["frozen_manifest_file_sha256"] = file_sha256(frozen_manifest_path)

results["frozen_npz_file_sha256"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_scalar_csv_file_sha256"] = file_sha256(frozen_csv_path)



# Also record engine source hash

results["engine_source_sha256"] = engine_source_hash



# Also keep canonical JSON hash for reference

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()
```

).hexdigest()

Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]

Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results

Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list(config.omega), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list(config.omega)}"

return results

Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

```
for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results



# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# P0 #4: Explicitly validate the frozen protocol and safeguard

# Validate protocol quantities against frozen constants and manifest

frozen_protocol = frozen_manifest.get("protocol", {})



protocol_checks = [
```

("T_sim", protocol.T_sim, FROZEN_T_SIM, frozen_protocol.get("T_sim")),

("n_steps", protocol.n_steps(config), FROZEN_N_STEPS, frozen_protocol.get("n_steps")),

("burn_end", protocol.burn_end(config), FROZEN_BURN_END, frozen_protocol.get("burn_end")),

("hist_end", protocol.hist_end(config), FROZEN_HIST_END, frozen_protocol.get("hist_end")),

("future_steps", protocol.future_steps(config), FROZEN_FUTURE_STEPS,
frozen_protocol.get("future_steps")),

("diagnostic_stride", protocol.diagnostic_stride, 50, frozen_protocol.get("diagnostic_stride")),

("scale_epsilon", config.scale_epsilon, FROZEN_ZERO_TOL, frozen_model.get("scale_epsilon",
FROZEN_ZERO_TOL)),

]

for name, observed, expected_frozen, expected_manifest in protocol_checks:

Check against frozen constant first

if observed != expected_frozen:

results["status"] = "failed"

results["error"] = (

f"{name} mismatch: observed={observed}, "

f"expected (frozen)={expected_frozen}"

```
)

return results


# Also check against manifest if available

if expected_manifest is not None and observed != expected_manifest:

results["status"] = "failed"

results["error"] = (

f"{name} mismatch: observed={observed}, "

f"expected (manifest)={expected_manifest}"

)

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)
```

```
seed_results = {"status": "passed", "checks": {}, "max_diffs": {}, "errors": {}}


try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,
```

```
)

# CRITICAL: Compute historical cross-branch metrics (the 8 time-based fields)
# These must be computed AFTER all branches are simulated
if "continue" in engine_results:
finalize_historical_branch_metrics(engine_results)

# Check prefix state - iterate over REQUIRED_PREFIX_KEYS
for key in REQUIRED_PREFIX_KEYS:
engine_val = prefix.get(key)
frozen_key = f"prefix_seed_{seed_str}_{key}"
frozen_val = frozen_files.get(frozen_key)

if frozen_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"prefix_{key}_present_in_frozen"] = False
all_passed = False
continue
```

else:

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = True

if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"prefix_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"prefix_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))

else:

```
seed_results["max_diffs"][f"prefix_{key}"] = None


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# P0 #2: Validate horizon metadata in branch outputs

# The preflight verifies config would produce correct metadata,

# but we must verify each returned engine_branch actually contains those values

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key, expected in BASELINE_HORIZON_METADATA.items():

observed = engine_branch.get(key)

passed = (

isinstance(observed, (int, np.integer))

and int(observed) == int(expected)

)

seed_results["checks"][f"{branch}_{key}_baseline_exact"] = passed
```

```
if not passed:

seed_results["status"] = "failed"

seed_results["errors"][f"{branch}_{key}_baseline_mismatch"] = (

f"observed={observed!r}, expected={expected!r}"

)

all_passed = False


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===

for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue
```

```
else:
seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True

if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False
all_passed = False
continue
else:
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True

if exact_array_equal_with_nan(engine_val, frozen_val):
seed_results["checks"][f"{branch}_{key}_exact"] = True
else:
seed_results["checks"][f"{branch}_{key}_exact"] = False
seed_results["status"] = "failed"
all_passed = False
if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):
```

```
if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"{branch}_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}"] = None


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===

# These do NOT exist in frozen NPZ. At rho=1, both horizon systems

# should produce identical results (1750 and 8750 steps match).

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# Use module-level map for consistency - direct indexing

frozen_equiv_key = EXTENDED_MATRIX_EQUIVALENCE_MAP[key]

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:

if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False
```

```
if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

if engine_val.shape != engine_frozen_equiv.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != equiv shape {engine_frozen_equiv.shape}"

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None


# === SCALAR KEYS ===

# Use module-level constants for consistency

# FROZEN_BRANCH_SCALAR_KEYS: All 42 historical fields from frozen production

# EXTENDED_BRANCH_SCALAR_KEYS: New fields in v1 for dual-horizon support


for key in FROZEN_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)
```

```
# P1 #8: Use CSV as authority, not NPZ fallback

# Get from CSV only (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]


# FROZEN scalars MUST exist in historical data

if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:

seed_results["status"] = "failed"
```

```
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# For scalars, compare directly - DO NOT silently skip

# Normalize to scalar value

try:

engine_scalar = float(engine_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_type_error"] = f"Cannot convert to float:
{type(engine_val).__name__}: {e}"

all_passed = False

continue
```

try:

frozen_scalar = float(frozen_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_frozen_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_frozen_type_error"] = f"Cannot convert frozen to float: {type(frozen_val).__name__}: {e}"

all_passed = False

continue

Compare with exact equality (rtol=0, atol=0)

engine_is_nan = bool(np.isnan(engine_scalar))

frozen_is_nan = bool(np.isnan(frozen_scalar))

if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_scalar, frozen_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

```
seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_scalar - frozen_scalar)) if not (engine_is_nan or frozen_is_nan) else None



# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True



# Use module-level map for consistency - direct indexing
```

```
frozen_equiv = EXTENDED_SCALAR_EQUIVALENCE_MAP[key]

if frozen_equiv:

# Get the engine's own frozen-equivalent value

engine_frozen_equiv = engine_branch.get(frozen_equiv)


if engine_frozen_equiv is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{frozen_equiv}_present_in_engine"] = False

all_passed = False

continue


# Validate internal consistency: extended should equal frozen equivalent

try:

engine_scalar = float(engine_val)

equiv_scalar = float(engine_frozen_equiv)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_internal_consistency"] = False
```

```
seed_results["errors"][f"{branch}_{key}_internal_type_error"] = str(e)

all_passed = False

continue


engine_is_nan = bool(np.isnan(engine_scalar))

equiv_is_nan = bool(np.isnan(equiv_scalar))


if engine_is_nan and equiv_is_nan:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

elif np.isclose(engine_scalar, equiv_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_internal"] = float(abs(engine_scalar - equiv_scalar)) if
not (engine_is_nan or equiv_is_nan) else None


except Exception as e:
```

```
seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False


results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":
```

```
print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")



return results

================================================================================
Main API
================================================================================
Known branches - allowlist for validation

KNOWN_BRANCHES = frozenset({

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",
```

"B_carrier_only",

"full_reset",

"future_shuffle",

})

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.

CRITICAL: branch_order defines the FULL frozen RNG namespace (7 branches).

protocol.branches defines which branches are actually executed.

```
This ensures future_shuffle gets the 7th RNG sequence (not the 4th)

as in production, even when only 4 study branches are executed.



Returns complete seed results with all branches.

"""

# Preflight validation: validate config and protocol

config.validate()

protocol.validate(config)



# Validate all protocol branches are known

for branch in protocol.branches:

if branch not in KNOWN_BRANCHES:

raise ValueError(

f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}"

)



# Validate all protocol branches are in branch_order (for RNG namespace)
```

```
for branch in protocol.branches:

if branch not in branch_order:

raise ValueError(

f"Branch '{branch}' in protocol.branches but not in branch_order. "

f"branch_order must include all protocol branches for correct RNG namespace."

)



# Validate branch_order has no duplicates and all are known

if len(branch_order) != len(set(branch_order)):

raise ValueError(f"branch_order has duplicate branches: {branch_order}")

for branch in branch_order:

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}' in branch_order. Known branches:
{sorted(KNOWN_BRANCHES)}")



# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)
```

```
# Create branch RNGs using frozen algorithm with FULL branch_order

branch_rngs = make_branch_rngs(seed, branch_order)



# Execute only the branches in protocol.branches, but use RNGs from branch_order

branch_results = {}

for branch in protocol.branches:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"This indicates branch_order={branch_order} does not match the RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),

prefix,
```

```
branch,

config,

protocol,

rng,

)



# Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in branch_results:

finalize_historical_branch_metrics(branch_results)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,
```

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

Quick validation

```
config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")
```

Ignoring copy/paste corruption, this engine is now very close to test-ready. The numerical kernel remains aligned with frozen production, and the previously important certificate gaps have been closed.
Verdict

Numerical kernel: approximately 9.7/10
RNG fidelity: approximately 9.7/10
Dual-horizon outputs: approximately 9.6/10
Mandatory baseline gate: approximately 9.5/10
Testing decision: fix the two small control/manifest issues below, then GO for compile → import → one-seed smoke → baseline-only

I would not make additional performance refactors before the baseline. At this point, minimizing source changes is preferable; let exact replay determine whether the implementation is correct.
P0: fix before running
1. protocol.branches manifest validation is unreachable due to indentation

This section contains:

Python

```
if "paired_seeds" not in frozen_manifest["randomization"]:
return {
...
}
if "branches" not in frozen_manifest["protocol"]:
return {
...
}
```

The branches check is indented inside the preceding if, after an unconditional return. It can never execute.

With the correct frozen manifest this does not affect the planned run, but a malformed manifest missing protocol.branches would later cause an uncaught KeyError here:

Python

```
manifest_branches = tuple(
frozen_manifest["protocol"]["branches"]
)
```

Fix

Dedent the branch check:

Python

```
if "paired_seeds" not in frozen_manifest["randomization"]:
return {
```

"status": "failed",
"error": (
"Frozen manifest missing "
"'randomization.paired_seeds'"
),
"frozen_manifest": str(
Path(frozen_manifest_path).resolve()
),
"frozen_npz": str(
Path(frozen_npz_path).resolve()
),
"frozen_csv_path": frozen_csv_path,
}

if "branches" not in frozen_manifest["protocol"]:
return {
"status": "failed",
"error": (
"Frozen manifest missing "
"'protocol.branches'"
),
"frozen_manifest": str(
Path(frozen_manifest_path).resolve()
),
"frozen_npz": str(
Path(frozen_npz_path).resolve()
),
"frozen_csv_path": frozen_csv_path,
}

2. T_sim is read from the wrong manifest section

Frozen production stores:

Python

frozen_manifest["model"]["T_sim"]

but the protocol check currently asks for:

Python

frozen_protocol.get("T_sim")

That returns None, so the engine checks protocol.T_sim against the hardcoded frozen constant but does not verify that the supplied manifest records the same value.

Change:

Python

```
(
"T_sim",
protocol.T_sim,
FROZEN_T_SIM,
frozen_protocol.get("T_sim"),
),
```

to:

Python

```
(
"T_sim",
protocol.T_sim,
FROZEN_T_SIM,
frozen_model.get("T_sim"),
),
```

Also add manifest checks for the horizon fields that the frozen protocol does contain:

Python

```
actual_window_steps = int(
round(config.T_window / config.dt)
)
actual_recovery_end = min(
5 * actual_window_steps,
```

protocol.future_steps(config),
)

protocol_checks.extend([
(
"window_steps",
actual_window_steps,
FROZEN_WINDOW_STEPS,
frozen_protocol.get("window_steps"),
),
(
"recovery_end",
actual_recovery_end,
FROZEN_RECOVERY_END,
frozen_protocol.get("recovery_end"),
),
])

Your returned branch metadata is now separately checked, which is correct.
Recommended exactness cleanup

These are small enough to settle before the source hash is frozen.
3. Make the alpha manifest comparison genuinely exact

Current code allows:

Python

atol=1e-15

despite describing the check as exact.

Now that the engine uses the same frozen construction:

Python

float(np.exp(-dt / T_window))

use:

Python

```
if not np.isclose(
frozen_model.get("alpha", 0.0),
config.alpha,
rtol=0.0,
atol=0.0,
):
...
```

The exact trajectory comparisons would probably catch a meaningful alpha discrepancy anyway, but the configuration certificate should stand on its own.
4. Use the named frozen gap tolerance

Replace:

Python

```
if g_min < config.delta - 1e-9:
```

with:

Python

```
if g_min < config.delta - FROZEN_GAP_TOL:
```

This is numerically identical and prevents drift between the declared constant and implementation.
5. Use strict relative tolerance in state checks

For validation—not dynamics—prefer:

Python

```
if not np.allclose(
self.B,
```

self.B.T,
rtol=0.0,
atol=FROZEN_ZERO_TOL,
):
...

and:

Python

if not np.allclose(
np.diag(self.B),
0.0,
rtol=0.0,
atol=FROZEN_ZERO_TOL,
):
...

The current default relative tolerance is unnecessarily permissive.

Apply the same change to validate_B_matrix().
Nonblocking robustness improvements

These do not need to delay the first baseline unless you want a completely polished reference API.
6. Check manifest schema uniqueness

Set equality does not detect duplicate schema entries. Add:

Python

for schema_name, schema_values in (
("prefix_keys", manifest_prefix),
("branch_matrix_keys", manifest_matrices),
("branch_scalar_keys", manifest_scalars),
):
if len(schema_values) != len(set(schema_values)):
return {
"status": "failed",

```
"error": (
f"Frozen manifest schema "
f"{schema_name} contains duplicates"
),
...
}
```

The actual frozen manifest should already be valid.
7. Reject malformed physical CSV rows explicitly

Rows with an empty seed or branch are currently ignored:

Python

```
if seed_id and branch:
...
```

An extra malformed row could therefore be invisible to the dictionary key-count check.

Track the physical row count and reject any row lacking identifiers:

Python

```
for line_number, row in enumerate(reader, start=2):
seed_id = row.get("seed", "")
branch = row.get("branch", "")

if not seed_id or not branch:
return {
"status": "failed",
"error": (
f"Frozen CSV line {line_number} "
f"missing seed or branch"
),
...
}
```

Again, this is authority-file hardening, not an expected issue with the actual frozen CSV.

8. Require continue when run_seed() promises historical metrics

If protocol.branches omits continue, run_seed() silently skips:

Python

finalize_historical_branch_metrics(...)

and returns branch results without the eight historical cross-branch scalar fields, despite the module-level required schema including them.

For this research stack, all intended protocols include continue. Make that explicit:

Python

if "continue" not in protocol.branches:
raise ValueError(
"run_seed requires the continue branch "
"to compute historical cross-branch metrics"
)

Alternatively, document that the eight fields are conditional on continue being executed.

The current sensitivity protocol contains continue, so the planned study is unaffected.
Performance decision

The remaining performance opportunities are:

cache config.alpha;
cache config.delay_steps;
use the frozen bounded deque;
mark cached edge/omega arrays read-only.

I would not change those immediately before the first baseline unless runtime is known to be prohibitive. They do not alter the scientific logic, and exact replay is now the more valuable test. If performance is unacceptable, optimize in a new source revision and rerun the complete baseline certificate.

What is now correctly closed

The latest version successfully addresses the substantive prior concerns:

positive-integer N validation;
finite protocol parameters;
nonempty and duplicate-free protocol branches;
full seven-branch executed baseline requirement;
exact seed cohort authority;
exact RNG branch-order authority;
all three frozen authority files preflighted;
malformed manifest and NPZ handling;
manifest schema comparison;
exact frozen NPZ key preflight;
CSV duplicate rejection;
exact CSV (seed, branch) coverage;
required scalar columns checked in every CSV row;
CSV-only scalar authority;
all 42 historical scalar fields;
historical cross-branch finalization;
literal frozen persistence-ratio condition;
complete extended scalar and matrix mappings;
mapping and duplicate integrity checks;
returned branch horizon metadata checks;
fixed safeguard validation;
NaN-aware exact array equality;
JSON-safe ordinary mismatch reporting;
fixed versus normalized horizon identities at $\rho=1$;
branch subset separated from full RNG namespace.

Final decision

After correcting:

the unreachable protocol.branches manifest check; and
the manifest location used for T_sim;

I consider this engine ready for the planned testing sequence.

The first saved baseline result must still be treated as finite replay evidence, not assumed from this code review. Only:

text

40/40 seeds passed
7/7 branches passed
all frozen arrays exact
all 42 historical scalars exact
all extended rho=1 identities exact
all horizon metadata exact

should authorize the full TwindowTwindow grid.

Next file we will evaluate is below:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import shutil

import sys

import time
```

```
import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

print("ERROR: pandas is required for production T_window study")

print("Install with: pip install pandas")

sys.exit(1)

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
```

Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5 # Also PHYSICAL_LATE_START

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
====
REQUIRED OUTPUT SCHEMA
==========
====
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
P1 #7: Separate study metadata from frozen scalar output
T_window and rho are study metadata, not part of historical frozen branch scalar schema

STUDY_METADATA_KEYS = (

text

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
P1 #6: Removed DESIRED_BRANCH_SCALAR_KEYS as they are always NaN
These fields require finalize_historical_branch_metrics() to be called, which is not done for individual (seed, rho) runs. The new analysis script computes richer condition-specific recovery estimands from trajectories, so these legacy fields are not needed.
All scalar keys for CSV output (required + study metadata)

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
STUDY_METADATA_KEYS
P0 #2: Separate frozen and study schemas
FROZEN keys must exist in historical frozen data (11 keys)

FROZEN_BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS
REQUIRED keys for study output - matches engine schema (all frozen + extended keys)

REQUIRED_BRANCH_MATRIX_KEYS = tuple(

text

engine.REQUIRED_BRANCH_MATRIX_KEYS

)
P0 #2: Validate at import that frozen keys are subset of engine keys

if not
set(FROZEN_BRANCH_MATRIX_KEYS).issubset(set(REQUIRED_BRANCH_MATRIX_KEYS)):

text

missing = set(FROZEN_BRANCH_MATRIX_KEYS) -
set(REQUIRED_BRANCH_MATRIX_KEYS)

raise RuntimeError(

f"Frozen matrix keys not subset of engine keys: {sorted(missing)}"

)

All matrix keys (use engine's full schema)

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

```
def is_finite_number(value: Any) -> bool:
text
"""Check if a value is a finite number.


Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.
"""
return (
isinstance(
value,
(
int,
float,
np.integer,
np.floating,
),
)
and np.isfinite(value)
```

```
)

def validate_prefix_outputs(prefix: dict[str, Any]) -> None:

text

"""P0 #5: Validate prefix outputs.



Validates that all required prefix keys are present with correct shapes.



Raises RuntimeError if required keys are missing or have wrong shapes.

"""

missing = set(REQUIRED_PREFIX_KEYS) - set(prefix)

if missing:

raise RuntimeError(f"schema: prefix missing {sorted(missing)}")



expected_shapes = {

"B_at_split": (FROZEN_N, FROZEN_N),

"R_physical_past": (FROZEN_N, FROZEN_N),

"theta_at_split": (FROZEN_N,),
```

"coherence_acc_at_split": (FROZEN_N, FROZEN_N),

"history_at_split": (FROZEN_DELAY_STEPS + 1, FROZEN_N),

"selected_at_split": (FROZEN_N, FROZEN_N),

}

for key, expected in expected_shapes.items():

if key in prefix:

observed = np.asarray(prefix[key])

if observed.shape != expected:

raise RuntimeError(f"schema: prefix {key} shape {observed.shape} != {expected}")

def validate_branch_outputs(

text

branch: str,

branch_result: dict[str, Any],

expected_rho: float,

expected_T_window: float,

) -> None:

```
text

"""Validate that a branch result contains all required outputs.



P0 #3: Now validates rho and T_window metadata.

P0 #4: Now validates recovery_end_ema.

P1 #8: Use engine's REQUIRED_BRANCH_MATRIX_KEYS for matrix validation.

Also validates array shapes and metadata consistency.



Raises RuntimeError if required keys are missing or have wrong shapes.

"""

# Validate required scalars

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())



# P1 #8: Use engine's matrix keys for validation

missing_matrices = set(engine.REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())



# P1 #8: Validate basic shapes for key arrays
```

```
shape_errors = []

if "B_final" in branch_result:

expected_shape = (FROZEN_N, FROZEN_N)

actual_shape = branch_result["B_final"].shape

if actual_shape != expected_shape:

shape_errors.append(f"B_final shape {actual_shape} != {expected_shape}")



# Validate trajectory lengths

expected_samples = (FROZEN_FUTURE_STEPS + 50 - 1) // 50 # diagnostic_stride=50

for key in ("sample_times", "density_trajectory", "similarity_trajectory"):

if key in branch_result:

actual_len = len(branch_result[key])

if actual_len != expected_samples:

shape_errors.append(f"{key} length {actual_len} != {expected_samples}")



# Validate horizon metadata consistency

# P0 #3: Validate rho and T_window with expected values
```

```
# P0 #4: Validate recovery_end_ema

metadata_errors = []

if "rho" in branch_result:

# P0 #3: Validate rho matches expected value

expected_rho = rho # Passed as parameter to this function

observed_rho = branch_result["rho"]

if float(observed_rho) != float(expected_rho):

metadata_errors.append(f"rho={observed_rho} != {expected_rho}")

else:

metadata_errors.append("missing rho")



if "T_window" in branch_result:

# P0 #3: Validate T_window matches expected value

observed_T_window = branch_result["T_window"]

if float(observed_T_window) != float(expected_T_window):

metadata_errors.append(f"T_window={observed_T_window} != {expected_T_window}")

else:
```

```
metadata_errors.append("missing T_window")



if "window_steps" in branch_result:

expected_window_steps = int(round(branch_result["T_window"] / FROZEN_DT))

if branch_result["window_steps"] != expected_window_steps:

metadata_errors.append(f"window_steps {branch_result['window_steps']} !=
{expected_window_steps}")



if "fixed_immediate_end" in branch_result:

if branch_result["fixed_immediate_end"] != 1750:

metadata_errors.append(f"fixed_immediate_end {branch_result['fixed_immediate_end']} != 1750")



if "fixed_recovery_end" in branch_result:

if branch_result["fixed_recovery_end"] != 8750:

metadata_errors.append(f"fixed_recovery_end {branch_result['fixed_recovery_end']} != 8750")



# P0 #4: Validate recovery_end_ema

if "recovery_end_ema" in branch_result:
```

```
expected_recovery_end = min(

5 * int(round(branch_result["T_window"] / FROZEN_DT)),

FROZEN_FUTURE_STEPS,

)

if branch_result["recovery_end_ema"] != expected_recovery_end:

metadata_errors.append(f"recovery_end_ema={branch_result['recovery_end_ema']} !=
{expected_recovery_end}")

else:

metadata_errors.append("missing recovery_end_ema")



# Combine all errors

all_errors = []

if missing_scalars:

all_errors.extend([f"missing scalar: {k}" for k in sorted(missing_scalars)[:10]])

if missing_matrices:

all_errors.extend([f"missing matrix: {k}" for k in sorted(missing_matrices)[:10]])

if shape_errors:
```

all_errors.extend(shape_errors[:10])

if metadata_errors:

all_errors.extend(metadata_errors[:10])

if all_errors:

raise RuntimeError(

f"Branch '{branch}' has {len(all_errors)} validation errors: "

f"{all_errors[:5]}... ({len(all_errors)} total)"

)

==========
Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file.

Local helper for file hashing - does not depend on engine utilities.

"""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: handle.read(1024 * 1024), b""):

```
digest.update(block)

return digest.hexdigest()

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)
```

```
handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary), str(path))

finally:

if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""
```

```
path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)
```

```
def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")



path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp",

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

[dataframe.to](http://dataframe.to)_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)
```

```
================================================================================
Configuration and Validation
================================================================================
```

```
def validate_engine_contract() -> None:
```

text

"""P1 #12: Validate that driver constants agree with engine.

Ensures driver and engine have consistent frozen production values.

"""

checks = {

"N": (FROZEN_N, engine.FROZEN_N),

"dt": (FROZEN_DT, engine.FROZEN_DT),

"tau": (FROZEN_TAU, engine.FROZEN_TAU),

"T_star": (FROZEN_T_STAR, engine.FROZEN_T_STAR),

"B_c": (FROZEN_B_C, engine.FROZEN_B_C),

"B_init_scale": (FROZEN_B_INIT_SCALE, engine.FROZEN_B_INIT_SCALE),

}

for name, (driver_value, engine_value) in checks.items():

if driver_value != engine_value:

raise RuntimeError(

```
f"Driver/engine {name} mismatch: "

f"driver={driver_value}, engine={engine_value}"

)



if not np.array_equal(FROZEN_OMEGA, engine.FROZEN_OMEGA):

raise RuntimeError(

f"Driver/engine omega mismatch: "

f"driver={FROZEN_OMEGA.tolist()}, engine={engine.FROZEN_OMEGA.tolist()}"

)



# Also check other important constants

if FROZEN_T_SIM != engine.FROZEN_T_SIM:

raise RuntimeError(f"Driver/engine T_SIM mismatch: {FROZEN_T_SIM} !=
{engine.FROZEN_T_SIM}")

if FROZEN_DELTA != engine.FROZEN_DELTA:

raise RuntimeError(f"Driver/engine DELTA mismatch: {FROZEN_DELTA} !=
{engine.FROZEN_DELTA}")

if FROZEN_ETA != engine.FROZEN_ETA:
```

```
raise RuntimeError(f"Driver/engine ETA mismatch: {FROZEN_ETA} != {engine.FROZEN_ETA}")

if FROZEN_GAMMA != engine.FROZEN_GAMMA:

raise RuntimeError(f"Driver/engine GAMMA mismatch: {FROZEN_GAMMA} !=
{engine.FROZEN_GAMMA}")

if FROZEN_MU != engine.FROZEN_MU:

raise RuntimeError(f"Driver/engine MU mismatch: {FROZEN_MU} != {engine.FROZEN_MU}")

if FROZEN_A != engine.FROZEN_A:

raise RuntimeError(f"Driver/engine A mismatch: {FROZEN_A} != {engine.FROZEN_A}")

if FROZEN_Q != engine.FROZEN_Q:

raise RuntimeError(f"Driver/engine Q mismatch: {FROZEN_Q} != {engine.FROZEN_Q}")

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,
```

```
omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800
```

```
split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

# ============================================================================
# Simulation Functions
# ============================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text
```

"""Simulate a single (seed, rho) combination.

CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates
study_branches subset. This ensures future_shuffle gets the 7th RNG sequence
(not the 4th) as in production.

Returns complete results for all branches at this (seed, rho).
"""
Create branch RNGs using FULL frozen branch order for correct namespace
branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)

Create a protocol with just the study branches
study_protocol = engine.ProtocolConfig(
T_sim=protocol.T_sim,
burn_end_time=protocol.burn_end_time,
split_time=protocol.split_time,
future_duration=protocol.future_duration,

```
diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# P0 #5: Validate prefix outputs

validate_prefix_outputs(prefix)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."
```

```
)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,

)


# P0 #3-4: Validate branch outputs with expected metadata

validate_branch_outputs(branch, branch_results[branch], rho, config.T_window)


# P1 #9: Validate branch outputs immediately after simulation

# Check both primary and secondary outcomes for finiteness

for key in PRIMARY_OUTCOMES + SECONDARY_OUTCOMES:

if key not in branch_results[branch]:
```

```
raise RuntimeError(
f"schema: {branch} missing {key}"
)
if not is_finite_number(branch_results[branch][key]):
raise RuntimeError(
f"schema: {branch} has non-finite {key}={branch_results[branch][key]}"
)

# Add metadata
branch_results[branch]["T_window"] = config.T_window
branch_results[branch]["rho"] = rho


return {
"seed": int(seed),
"rho": rho,
"config": {
"N": config.N,
"dt": config.dt,
```

```
"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

def run_baseline_equivalence(

text
```

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,

frozen_csv_path: str,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

Load frozen manifest to compare seeds

try:

with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

except json.JSONDecodeError as e:

return {

"status": "failed",

"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",

```
"seeds_match": False,
}

# Validate manifest has required structure
if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:
return {
"status": "failed",
"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",
"seeds_match": False,
}

frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]

# Verify seed list matches frozen manifest EXACTLY
# This is critical: baseline equivalence tests a specific cohort
if len(seeds) != len(frozen_seeds):
return {
```

```
"status": "failed",

"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}
```

```
# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Validate frozen CSV path exists

if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,

}



# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches
```

```
# to certify the engine as reusable infrastructure
baseline_protocol = engine.ProtocolConfig(
T_sim=FROZEN_T_SIM,
burn_end_time=160.0,
split_time=480.0,
future_duration=320.0,
diagnostic_stride=50,
branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification
)


# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches
results = [engine.run](http://engine.run)_baseline_equivalence(
seeds=seeds,
config=config,
protocol=baseline_protocol,
frozen_manifest_path=frozen_manifest_path,
frozen_npz_path=frozen_npz_path,
```

```
frozen_csv_path=frozen_csv_path,

branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace

)



# Add seed comparison info

results["seed_comparison"] = {

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}



return results

================================================================================
Manifest and Output
================================================================================

def build_manifest(

text

master_seed: int,
```

```
seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""

# Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,
```

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}

protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

}

study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

```
"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes

dependencies = {}



# Engine source hash

try:

engine_path = Path(engine.__file__).resolve()

engine_hash = file_sha256(engine_path)

dependencies["aief_experiment_engine_v1"] = {

"path": str(engine_path),

"sha256": engine_hash,

}

except Exception as e:

dependencies["aief_experiment_engine_v1"] = {
```

```
"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",

"error": str(e),

}



# Driver source hash (this file)

try:

driver_path = Path(__file__).resolve()

driver_hash = file_sha256(driver_path)

dependencies["AIEF_1_4_Twindow_sensitivity"] = {

"path": str(driver_path),

"sha256": driver_hash,

}

except Exception as e:

dependencies["AIEF_1_4_Twindow_sensitivity"] = {

"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",
```

```
"error": str(e),
}

manifest = {
"study_name": STUDY_NAME,
"workflow_version": WORKFLOW_VERSION,
"source_sha256": source_sha256(),
"dependencies": dependencies,
"model": model_params,
"protocol": protocol_params,
"study": study_params,
"randomization": {
"master_seed": int(master_seed),
"paired_seeds": [int(seed) for seed in seeds],
"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",
"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,
```

```
# P1 #14: Record the full RNG namespace explicitly

"branch_rng_namespace": list(FROZEN_BRANCH_ORDER),

"executed_branches": list(STUDY_BRANCHES),

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",
```

```
"T_*": FROZEN_T_STAR,
"fixed_physical_horizons": {
"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",
"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",
"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",
},
"ema_normalized_horizons": {
"immediate": "0-1 T_window",
"recovery": "1-5 T_window",
"late": "5 T_window to end",
},
"full_regeneration": "Each (seed, rho) pair is independently initialized",
},
}

return manifest
def flatten_result(
```

```
text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results
```

```
for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text
```

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

validate_branch_outputs(branch, branch_result, rho, branch_result.get("T_window", float('nan')))

row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}

for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

```
row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

# ============================================================================
# Main Execution
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",
```

```
type=int,
default=DEFAULT_MASTER_SEED,
help="fixed integer master seed (default: 20260714). For production, must match frozen data.",
)
parser.add_argument(
"--n-seeds",
type=int,
default=DEFAULT_N_SEEDS,
help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
)
parser.add_argument(
"--run-root",
default=DEFAULT_RUN_ROOT,
help=f"output root directory (default: {DEFAULT_RUN_ROOT})",
)
parser.add_argument(
"--frozen-manifest",
default=None,
```

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

```
parser.add_argument(
"--overwrite",
action="store_true",
help="overwrite existing run directory. Use with caution - will delete existing data.",
)
# NOTE: --skip-baseline removed for production safety
# Baseline equivalence is MANDATORY and cannot be skipped
# For debugging, use a separate nonproduction namespace
parser.add_argument(
"--dry-run",
action="store_true",
help="validate configuration and exit without running",
)
parser.add_argument(
"--baseline-only",
action="store_true",
help=(
"run and save the mandatory frozen-baseline "
```

```
"certificate, then exit without launching the grid"

),

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,
```

```
)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# P1 #12: Validate driver/engine contract

try:

validate_engine_contract()

except RuntimeError as e:

print(f"\nFATAL: Driver/engine contract violation: {e}")

sys.exit(1)



# Check dry-run mode FIRST to avoid contaminating production directory

# P0 #1: Dry-run should not require frozen paths
```

```
if args.dry_run:

# Validate driver/engine contract (already validated above, but keep for clarity)

try:

validate_engine_contract()

except RuntimeError as e:

print(f"\nDry run: Driver/engine contract violation: {e}")

sys.exit(1)


# P1 #11: Dry-run should actually validate configuration

# Validate all levels

protocol_config = create_protocol_config()

try:

# Validate base config (rho=1.0)

base_config = create_model_config(1.0)

base_config.validate()

protocol_config.validate(base_config)


# Validate all rho configs
```

```
for rho in RHO_VALUES:

config = create_model_config(rho)

config.validate()

protocol_config.validate(config)


# Validate study design

if not set(STUDY_BRANCHES) &lt;= set(FROZEN_BRANCH_ORDER):

raise ValueError(f"STUDY_BRANCHES {STUDY_BRANCHES} not subset of
FROZEN_BRANCH_ORDER {FROZEN_BRANCH_ORDER}")

if len(STUDY_BRANCHES) != len(set(STUDY_BRANCHES)):

raise ValueError(f"STUDY_BRANCHES has duplicates: {STUDY_BRANCHES}")

if len(RHO_VALUES) != len(set(RHO_VALUES)):

raise ValueError(f"RHO_VALUES has duplicates: {RHO_VALUES}")

if 1.0 not in RHO_VALUES:

raise ValueError("RHO_VALUES must include 1.0 for baseline equivalence")


print("Dry run: configuration validated successfully.")

print(f" Validated {len(RHO_VALUES)} rho values: {RHO_VALUES}")
```

```
print(f" Validated {len(STUDY_BRANCHES)} study branches: {STUDY_BRANCHES}")

print(f" Validated {args.n_seeds} seeds from master seed {args.master_seed}")

print(f"Configuration hash: {config_hash}")

print("To execute, run without --dry-run")

print(f"Expected run directory: {Path([args.run](http://args.run)_root) / config_hash}")

except Exception as e:

print(f"Dry run: configuration validation FAILED: {e}")

sys.exit(1)

return



# P1 #10: Validate mandatory input paths before creating/deleting run directory

# This prevents contaminating a production root after a simple path error

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:
```

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Create run directory

run_dir = Path(args.run_root) / config_hash

```
# Check if run directory already exists

if run_dir.exists():

if args.overwrite:

# Overwrite mode: remove existing directory

print(f"Overwriting existing run directory: {run_dir}")

try:

shutil.rmtree(run_dir)

except Exception as e:

print(f"Cannot overwrite: error removing directory: {e}")

sys.exit(1)

run_dir.mkdir(parents=True, exist_ok=True)


else:

# Default: fail if directory exists

print(f"Run directory {run_dir} already exists")

print("Use --overwrite to replace")

sys.exit(1)
```

```
else:

# New run directory

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")

print(f" output: {run_dir}")

print()
```

```
# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")
```

```
print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



# Run baseline equivalence - MANDATORY for all seeds

try:

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,
```

```
frozen_npz_path=args.frozen_npz,

frozen_csv_path=args.frozen_scalar,

)

except Exception as exc:

# Handle baseline equivalence exception - save failure manifest

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_exception",

"exception_type": type(exc).__name__,

"exception_message": str(exc),

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")
```

```
print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from
AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")
```

```
print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)



# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"
```

```
atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Save per-seed baseline results separately for easier audit

if "seed_results" in baseline_results:

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_seed_path, baseline_results["seed_results"])
```

```
print(f" Per-seed results saved to {baseline_seed_path}")


# P0 #3: Update manifest with baseline equivalence certificate

# Record all frozen input hashes using CORRECT engine key names

# Engine returns: frozen_manifest_file_sha256, frozen_npz_file_sha256,
frozen_scalar_csv_file_sha256

# Engine also returns: frozen_manifest, frozen_npz, frozen_manifest_hash


frozen_input_hashes = {}


# Get frozen manifest file hash (engine key: frozen_manifest_file_sha256)

if "frozen_manifest_file_sha256" in baseline_results:

frozen_input_hashes["manifest_file_sha256"] = baseline_results["frozen_manifest_file_sha256"]

if "frozen_manifest" in baseline_results:

frozen_input_hashes["manifest_path"] = baseline_results["frozen_manifest"]


# Get frozen NPZ file hash (engine key: frozen_npz_file_sha256)

if "frozen_npz_file_sha256" in baseline_results:

frozen_input_hashes["npz_file_sha256"] = baseline_results["frozen_npz_file_sha256"]

if "frozen_npz" in baseline_results:
```

```
frozen_input_hashes["npz_path"] = baseline_results["frozen_npz"]


# Get frozen scalar CSV file hash (engine key: frozen_scalar_csv_file_sha256)
if "frozen_scalar_csv_file_sha256" in baseline_results:
frozen_input_hashes["scalar_csv_file_sha256"] = baseline_results["frozen_scalar_csv_file_sha256"]


# Get canonical JSON hash of frozen manifest (engine key: frozen_manifest_hash)
if "frozen_manifest_hash" in baseline_results:
frozen_input_hashes["manifest_canonical_json_sha256"] = baseline_results["frozen_manifest_hash"]


# Read frozen production source hash from frozen manifest itself
# This is the proper way - don't search for production source files
try:
with open(args.frozen_manifest, 'r', encoding='utf-8') as f:
frozen_manifest_data = json.load(f)
if "source_sha256" in frozen_manifest_data:
frozen_input_hashes["frozen_production_source"] = frozen_manifest_data["source_sha256"]
except Exception:
```

```
pass


# Also include scalar CSV path for completeness
if args.frozen_scalar:
frozen_input_hashes["scalar_csv_path"] = str(Path(args.frozen_scalar).resolve())


manifest["baseline_equivalence"] = {
"status": "passed",
"n_seeds_tested": baseline_results["n_seeds"],
"n_seeds_passed": baseline_results["n_seeds"],
"engine_source_sha256": baseline_results.get("engine_source_sha256"),
"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),
"frozen_npz_hash": baseline_results.get("frozen_npz_file_sha256"),
"frozen_csv_hash": baseline_results.get("frozen_scalar_csv_file_sha256", "N/A"),
"frozen_production_source_hash": frozen_input_hashes.get("frozen_production_source", "N/A"),
"frozen_input_hashes": frozen_input_hashes,
"certificate_path": str(baseline_cert_path),
"seed_results_path": str(baseline_seed_path) if "seed_results" in baseline_results else "N/A",
```

```
"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches
}

# Re-save manifest with baseline certificate
atomic_write_json(manifest_path, manifest)

# P0 #2: Handle --baseline-only mode
if args.baseline_only:
manifest["execution_mode"] = "baseline_only"
manifest["baseline_certified"] = True
manifest["freezeable"] = False
manifest["completion"] = {
"status": "baseline_only_completed",
"study_grid_started": False,
"baseline_passed": True,
}
manifest["output_hashes"] = {
"baseline_certificate": file_sha256(baseline_cert_path),
```

"baseline_seed_results": file_sha256(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

}

atomic_write_json(manifest_path, manifest)

print("\nBaseline-only run completed successfully.")

print("The T_window grid was not started.")

return

Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")

all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {} # Full payload in memory; checkpoints also written per-condition

```
# Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0



started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)



# Fatal error flag for breaking outer loop on schema/config mismatch

fatal_error = False



for seed_index, seed in enumerate(seeds):

# Check for fatal error from previous seed

if fatal_error:

print(f" Skipping seed {seed} due to fatal schema/config error")

break



for rho in RHO_VALUES:
```

```
condition_start = time.time()

# Create config for this rho
config = create_model_config(rho)

# Checkpoint path for this condition (saved for diagnostic recovery, not resume)
checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"

try:
# Validate config before simulation
config.validate()
protocol.validate(config)

# Run simulation
result = simulate_seed_rho(
seed=seed,
rho=rho,
config=config,
protocol=protocol,
```

```
study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)
```

```
# Save checkpoint atomically
save_npz_atomic(checkpoint_path, matrix_payload)

condition_status.append({
"seed": seed,
"rho": rho,
"status": "success",
"elapsed_seconds": time.time() - condition_start,
})
successful_conditions += 1

except Exception as e:
error_str = str(e)
condition_status.append({
"seed": seed,
"rho": rho,
"status": "failed",
```

```
"error": error_str,

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Stop immediately on schema/config mismatch - use fatal flag to break outer loop

if "schema" in error_str.lower() or "config" in error_str.lower() or "missing" in error_str.lower():

print(f"\n FATAL: Schema/config mismatch detected - stopping immediately")

print(f" This error will affect all remaining conditions")

# Set fatal error flag to break outer loop after inner loop

fatal_error = True

break


# Status

elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"
```

```
if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

# P1 #16: pandas is mandatory (script exits at import without it), so remove dead fallback

if condition_status:

# pandas is guaranteed to be available due to import-time check

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")



# Record condition status in manifest

manifest["condition_status"] = {

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,

"successful_rows": successful_conditions,
```

"failed_rows": failed_conditions,

}

Also record pandas version in environment

manifest["environment"]["pandas"] = pd.__version__

print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")

If any conditions failed, mark as non-freezeable

if failed_conditions > 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")

Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

```
actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")


# Validate primary outcomes are present and finite

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



# P1 #5: Validate secondary outcomes are present and finite

missing_secondary = []

for row in all_scalar_rows:

for key in SECONDARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):
```

```
missing_secondary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



if missing_secondary:

print(f"\nERROR: {len(missing_secondary)} secondary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_secondary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"
```

```
npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))



# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())



temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())
```

```
os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

# P1 #5: Require both primary and secondary outcomes to be complete

complete = (

failed_conditions == 0

and successful_conditions == total_combinations

and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

and len(missing_secondary) == 0
```

```
)



# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"

manifest["freezeable"] = complete and baseline_passed



# Final manifest update with completion status

manifest["completion"] = {

"status": "completed" if complete else "incomplete",

"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"n_missing_secondary_outcomes": len(missing_secondary),

"elapsed_seconds": time.time() - started,
```

```
"freezeable": manifest["freezeable"],

}



# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}



# Main outputs

if csv_path.exists():

output_hashes["scalar_csv"] = file_sha256(csv_path)

if npz_path.exists():

output_hashes["branch_npz"] = file_sha256(npz_path)



# Condition status

condition_status_path = run_dir / "twindow_condition_status.csv"

if condition_status_path.exists():

output_hashes["condition_status_csv"] = file_sha256(condition_status_path)
```

```
# Baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

if baseline_cert_path.exists():

output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)



# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

if baseline_seed_path.exists():

output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)



if output_hashes:

manifest["output_hashes"] = output_hashes



# P1 #15: Condition checkpoints are diagnostic, non-authoritative artifacts

# The consolidated NPZ is the authority

manifest["checkpoints"] = {

"role": "diagnostic, non-authoritative",
```

"count": successful_conditions,

"authoritative_consolidation": str(npz_path),

}

Re-save final manifest

atomic_write_json(manifest_path, manifest)

Print completion status

if complete and baseline_passed:

print(f"\nRun COMPLETED successfully and is FREEZEABLE")

elif complete and not baseline_passed:

print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")

P0 #4: Exit nonzero for non-freezeable runs

sys.exit(1)

else:

print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions > 0:

```
print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")

if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")

# P0 #4: Exit nonzero for non-freezeable runs

sys.exit(1)



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

# P1 #17: Print conditionally - only if files exist

if csv_path.exists():

print(f"Saved {csv_path}")

if npz_path.exists():

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")
```

```
# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Use the missing_primary count already computed earlier

if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:
```

```
for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]


if not available_keys:

print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")

continue


stats = {}

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

# Use sample SD (ddof=1) for descriptive statistics
```

stats[key] = f"{np.mean(values):.4f}±{np.std(values, ddof=1):.4f}"

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

Ignoring copy/paste corruption, the driver has one deterministic full-grid failure caused by validation order, plus a direct undefined-variable bug in that validator.
Verdict

Baseline-only path: likely works with the corrected engine.
Full grid as currently written: NO-GO — every first condition will fail before output extraction.
Fix the two P0 issues below before running even the baseline-only test, because changing the driver afterward changes its source hash and configuration identity.

P0: deterministic condition failure
1. Branch metadata is validated before it is added

In simulate_seed_rho() the order is:

Python

branch_results[branch] = engine.simulate_future(...)

validate_branch_outputs(
branch,

branch_results[branch],
rho,
config.T_window,
)

Add metadata
branch_results[branch]["T_window"] = config.T_window
branch_results[branch]["rho"] = rho

But validate_branch_outputs() requires both:

text

rho
T_window

and also uses:

Python

branch_result["T_window"]

when validating window_steps.

Therefore, before metadata has been added:

it records missing rho;
it records missing T_window;
it then attempts:

Python

branch_result["T_window"]

and raises:

text

KeyError: 'T_window'

The first (seed,ρ,continue)(seed,ρ,continue) grid condition will fail.
Required order

Python

```
branch_result = engine.simulate_future(
split_state.clone(),
prefix,
branch,
config,
study_protocol,
rng,
)

# Add condition metadata before validation.
branch_result["T_window"] = config.T_window
branch_result["rho"] = rho

# Now validate independently against expected values.
validate_branch_outputs(
branch=branch,
branch_result=branch_result,
expected_rho=rho,
expected_T_window=config.T_window,
)

# Validate primary and secondary estimands.
for key in PRIMARY_OUTCOMES + SECONDARY_OUTCOMES:
if key not in branch_result:
raise RuntimeError(
f"schema: {branch} missing {key}"
)
if not is_finite_number(branch_result[key]):
raise RuntimeError(
f"schema: {branch} has non-finite "
```

f"{key}={branch_result[key]}"
)

branch_results[branch] = branch_result

Do not run the grid before changing this order.
2. rho is undefined inside validate_branch_outputs()

The function receives:

Python

expected_rho: float

but then does:

Python

expected_rho = rho

There is no local or global rho in that function. Once metadata is added before validation, this line will raise:

text

NameError: name 'rho' is not defined

Remove it completely:

Python

if "rho" in branch_result:
observed_rho = branch_result["rho"]
if float(observed_rho) != float(expected_rho):
metadata_errors.append(
f"rho={observed_rho} != {expected_rho}"

)
else:
metadata_errors.append("missing rho")

These two bugs together make the current full-grid path unusable.
Recommended corrected metadata block

Use the expected function arguments—not the returned metadata—to calculate expected values:

Python

```
metadata_errors = []

if "rho" not in branch_result:
metadata_errors.append("missing rho")
elif float(branch_result["rho"]) != float(expected_rho):
metadata_errors.append(
f"rho={branch_result['rho']} != {expected_rho}"
)

if "T_window" not in branch_result:
metadata_errors.append("missing T_window")
elif float(branch_result["T_window"]) != float(
expected_T_window
):
metadata_errors.append(
f"T_window={branch_result['T_window']} "
f"!= {expected_T_window}"
)

expected_window_steps = int(
round(expected_T_window / FROZEN_DT)
)
expected_recovery_end = min(
5 * expected_window_steps,
FROZEN_FUTURE_STEPS,
)
expected_fixed_immediate = int(
round(PHYSICAL_IMMEDIATE_END / FROZEN_DT)
```

```
)
expected_fixed_recovery = int(
round(PHYSICAL_RECOVERY_END / FROZEN_DT)
)

expected_metadata = {
"window_steps": expected_window_steps,
"recovery_end_ema": expected_recovery_end,
"fixed_immediate_end": expected_fixed_immediate,
"fixed_recovery_end": expected_fixed_recovery,
}

for key, expected in expected_metadata.items():
if key not in branch_result:
metadata_errors.append(f"missing {key}")
elif int(branch_result[key]) != int(expected):
metadata_errors.append(
f"{key}={branch_result[key]} != {expected}"
)
```

This provides an independent check. Calculating expected values from branch_result["T_window"] weakens the test because a mutually consistent but incorrect returned T_window and window_steps could pass.
P1: fix before the full grid
3. extract_scalar_row() repeats validation using a self-referential expectation

It currently calls:

Python

```
validate_branch_outputs(
branch,
branch_result,
rho,
branch_result.get("T_window", float("nan")),
)
```

The expected TwindowTwindow is taken from the value being validated. That cannot detect an incorrect returned value.

Since simulate_seed_rho() already performs complete validation against config.T_window, the cleanest choice is to remove the second call from extract_scalar_row().

Alternatively use an independently calculated value:

Python

```
validate_branch_outputs(
branch=branch,
branch_result=branch_result,
expected_rho=rho,
expected_T_window=rho * FROZEN_T_STAR,
)
```

I prefer validating once, immediately after simulation, and letting extract_scalar_row() only extract.
4. Require all horizon metadata keys

Current validation requires recovery_end_ema but checks the other metadata only if present:

Python

```
if "window_steps" in branch_result:
...
```

Missing window_steps, fixed_immediate_end, or fixed_recovery_end would not itself be an error.

The unified expected-metadata loop above fixes this.
5. Add NPZ key-collision detection

Before:

Python

```
all_matrix_payload.update(matrix_payload)
```

add:

Python

overlap = set(all_matrix_payload) & set(matrix_payload)

if overlap:
raise RuntimeError(
f"schema: NPZ key collision: "
f"{sorted(overlap)[:10]}"
)

all_matrix_payload.update(matrix_payload)

The current fixed grid should not collide, but this protects the consolidated authority file.
6. Validate the study protocol inside simulate_seed_rho()

After constructing:

Python

study_protocol = engine.ProtocolConfig(...)

run:

Python

config.validate()
study_protocol.validate(config)

The outer loop already validates a nearly identical protocol, but the function should validate the actual object it uses.

Also require:

Python

if tuple(study_branches) != tuple(STUDY_BRANCHES):
raise RuntimeError(
f"Unexpected study branch set/order: "
f"{study_branches}"
)

for this versioned production driver.
7. Strengthen matrix shape validation

The driver now correctly requires the complete engine matrix schema, and the manifest records it. But only B_final gets a 12×1212×12 shape check.

Before the full grid, validate every matrix field that should be 12×1212×12, including the fixed and normalized matrices. Require finiteness as well.

At minimum:

Python

MATRIX_12X12_KEYS = tuple(
key
for key in REQUIRED_BRANCH_MATRIX_KEYS
if key not in {
"sample_times",
"density_trajectory",
"similarity_trajectory",
}
)

for key in MATRIX_12X12_KEYS:
if key not in branch_result:
continue

value = np.asarray(branch_result[key])

if value.shape != (FROZEN_N, FROZEN_N):
shape_errors.append(

f"{key} shape {value.shape} "
f"!= {(FROZEN_N, FROZEN_N)}"
)

if not np.all(np.isfinite(value)):
shape_errors.append(
f"{key} contains non-finite values"
)

similarity_trajectory may legitimately contain NaNs after zero-BB resets, so do not require it to be entirely finite.
8. Validate the exact sample-time schedule

Length 640 is good but insufficient. Add:

Python

expected_times = (
np.arange(
0,
FROZEN_FUTURE_STEPS,
50,
dtype=float,
)
+ 1.0
) * FROZEN_DT

observed_times = np.asarray(
branch_result["sample_times"],
dtype=float,
)

if not np.array_equal(observed_times, expected_times):
shape_errors.append(
"sample_times does not match the expected "
"0.01-to-319.51 diagnostic schedule"
)

Also require all three trajectory arrays to have equal lengths.

Manifest/provenance improvements
9. Add missing explicit model fields

The manifest should include:

Python

"scale_epsilon": 1e-12,

under model.

Add pandas to the environment during build_manifest() rather than only after the full grid:

Python

"environment": {
"python": platform.python_version(),
"numpy": np.__version__,
"pandas": pd.__version__,
"platform": platform.platform(),
},

That gives baseline-only runs complete environment provenance.
10. Strengthen the driver/engine contract

Add:

Python

extra_checks = {
"scale_epsilon": (
1e-12,
engine.FROZEN_ZERO_TOL,
),
"delay_steps": (
FROZEN_DELAY_STEPS,
engine.FROZEN_DELAY_STEPS,
),

"window_steps": (
FROZEN_WINDOW_STEPS,
engine.FROZEN_WINDOW_STEPS,
),
"recovery_end": (
FROZEN_RECOVERY_END,
engine.FROZEN_RECOVERY_END,
),
}

for name, (driver_value, engine_value) in extra_checks.items():
if driver_value != engine_value:
raise RuntimeError(...)

And:

Python

if tuple(
engine.DEFAULT_PROTOCOL_CONFIG.branches
) != FROZEN_BRANCH_ORDER:
raise RuntimeError(
"Driver/engine frozen branch RNG namespace mismatch"
)

11. Configuration hash remains machine/path dependent

The configuration hash still includes:

absolute source paths;
Python version;
NumPy version;
platform string.

That is not wrong for a local run identifier, but it is not a portable scientific-configuration identity.

For the imminent single-system test, this is not blocking. Record explicitly that the 12-character identifier is a run/environment/source identity, not only a model-parameter hash.

Do not change this after a successful baseline without rerunning the baseline.
Smaller cleanup

Remove the duplicated frozen-path validation block before baseline execution.
validate_engine_contract() is called twice during dry-run.
LEGACY_OUTCOMES appears unused.
load_npz() appears unused.
is_finite_number() accepts booleans because bool subclasses int.
Incomplete-run reporting still omits the missing-secondary count.
Add exception type to failed condition rows.
all_errors counts only truncated missing-key lists, so its reported total may understate the actual number of validation errors.
The docstring should use the exact:

text

T_* = 17.502917152858814...

rather than equality to 17.5.

What is now correct

The important architecture is sound:

baseline-only mode exists and exits before the grid;
dry-run no longer requires frozen paths;
full prefix regeneration occurs for every (s,ρ)(s,ρ);
no split state is reused across different TwindowTwindow;
the full seven-branch RNG namespace is retained;
only the four prespecified study branches are executed;
primary fixed-physical and secondary normalized outcomes are checked;
prefix schema and shape checks are present;
the complete engine matrix schema is required and recorded;
study metadata is separated from historical scalar fields;
frozen input hashes use the engine’s actual key names;
condition checkpoints are atomic and non-authoritative;
consolidated outputs and condition status are hashed;
incomplete runs exit nonzero;

baseline-only and full-study manifests are distinguished.

Final decision

The current driver’s baseline-only path may work, but the full grid will fail because metadata validation occurs before metadata insertion and because rho is undefined inside the validator.

Fix those two issues before any saved test. Since the driver source hash is part of the run identity, the baseline-only certificate should be generated only after this correction. After that, the driver is ready for staged testing with the corrected engine.

Okay,now the last file:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd

==========
====

Configuration
==========
====

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Recovery DID expected count: 40 seeds 4 non-baseline rho 3 non-continue branches 2 systems 2 metrics * 13 outcomes = 24960

EXPECTED_N_RECOVERY_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1) len(NON_CONTINUE_BRANCHES) 2 2 * 13
Recovery groups: 5 rho 4 branches 2 systems * 2 metrics = 80 groups

EXPECTED_N_RECOVERY_GROUPS = len(EXPECTED_RHO_VALUES)
len(EXPECTED_BRANCHES) 2 * 2
Recovery DID groups: 4 rho 3 branches 2 systems 2 metrics 13 outcomes = 624 groups

EXPECTED_N_RECOVERY_DID_GROUPS = (len(EXPECTED_RHO_VALUES) - 1)
len(NON_CONTINUE_BRANCHES) 2 2 13
Censoring-aware fields that must have 40 finite values per group

RESTRICTED_RECOVERY_FIELDS = (

text

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_observed",

"sustained_observed",

"held_observed",

)
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
Utility Functions
==========

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)

Use pandas with explicit dtypes

df = pd.read_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)

Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = pd.to_numeric(df[col], errors="coerce")

except Exception:

pass

```
return df

def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:
```

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

```
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:
```

```
f.write(text)

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
# Convert NaN and infinity to None for JSON serialization

# Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):
```

return None

return val

elif isinstance(obj, np.ndarray):

Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

fsync the directory

```
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl

digest = hl.sha256()

path = Path(path)

if not path.exists():

return "FILE_NOT_FOUND"

with open(path, "rb") as f:
```

```
for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

============================================================================
====
Metric Functions
============================================================================
====

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.



Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



# Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))
```

crossing_indices = np.flatnonzero(above_threshold)

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

```
times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



if len(times) &lt; 2:

return float('nan'), False



# Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step &lt;= 0:

return float('nan'), False



# Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))



# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp; (values &gt;= threshold))
```

```
# Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False



for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break



return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.
```

Handles NaN values by treating them as zero for the deficit calculation.

Uses np.trapezoid if available, falls back to np.trapz for older NumPy versions.

NaN cosine values (from zero-vector weights) are imputed as zero for recovery

and AUC calculations only - this is a prespecified recovery-only convention.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0 (prespecified recovery-only convention for undefined zero-vector cosine)

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration - use np.trapezoid if available, else np.trapz

if hasattr(np, "trapezoid"):

return float(np.trapezoid(deficit, x=times))

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

```
return float('nan')



return float([np.dot](http://np.dot)(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] &gt; threshold

y = B2[edges] &gt; threshold



union = np.count_nonzero(x | y)

if union == 0:

return float('nan')



return float(np.count_nonzero(x &amp;amp; y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text
```

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Get unique seeds, rho values, and branches from the data

seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())

Build complete list of required keys for all conditions

We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

```
prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])



# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "
```

f"This indicates schema mismatch or incomplete simulation."

)

recovery_rows = []

for seed in seeds:

for rho in rho_values:

Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) & (df["rho"] == rho)]["T_window"].iloc[0])

Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) & (df["rho"] == rho) & (df["branch"] == "continue")].iloc[0]

Get reference targets for both horizon systems

Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])

Similarity targets for both horizon systems

```
similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])


for branch in branches:

# INCLUDE continue branch for DID calculation

# For continue: compare trajectory with itself, AUC deficit = 0

# This makes recovery DID well-defined: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]


prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)
```

```
cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# Validate that sample times match between branch and continue

# Use exact equality since all trajectories use the same diagnostic schedule

if not np.array_equal(sample_times, cont_sample_times):

raise RuntimeError(

f"Sample times mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"

)


# Also validate that trajectory lengths match

if len(density_trajectory) != len(cont_density_trajectory):

raise RuntimeError(

f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
```

```
f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"

)

if len(similarity_trajectory) != len(cont_similarity_trajectory):

raise RuntimeError(

f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"

)


# Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

# Process density recovery

density_values = density_trajectory

density_target = density_target_continue


# Calculate first crossing for density
```

```
density_first_crossing, density_first_observed = first_crossing_time(
sample_times, density_values, density_target
)

# Calculate sustained recovery for density (stays above target for rest of trajectory)
density_sustained, density_sustained_observed = first_time_sustained_for_rest(
sample_times, density_values, density_target
)

# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)
```

```
# Process similarity recovery

similarity_values = similarity_trajectory

similarity_target = similarity_target


# Calculate first crossing for similarity

sim_first_crossing, sim_first_observed = first_crossing_time(

sample_times, similarity_values, similarity_target

)


# Calculate sustained recovery for similarity

sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(

sample_times, similarity_values, similarity_target

)


# Calculate held recovery for similarity

sim_held, sim_held_observed = first_time_held_for_duration(

sample_times, similarity_values, similarity_target, t_window

)
```

```
# Calculate AUC deficit for similarity
sim_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_similarity_trajectory, similarity_values
)

# Calculate restricted times (horizon for censored, event_time for observed)
# Use the last sample time as the horizon (not the elapsed difference)
horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')

# For density metrics
density_restricted_first = density_first_crossing if density_first_observed else horizon_time
density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time
density_restricted_held = density_held if density_held_observed else horizon_time

# For similarity metrics
sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time
sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time
sim_restricted_held = sim_held if sim_held_observed else horizon_time
```

Calculate normalized times (t_rec / T_window)

Raw observed times (conditional on recovery)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window > 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window > 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window > 0 else float('nan')

sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window > 0 else float('nan')

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Censoring-aware restricted normalized times (cohort-level estimates)

These use restricted times (horizon for censored, event_time for observed) divided by T_window

density_restricted_first_over_Twindow = density_restricted_first / t_window if np.isfinite(density_restricted_first) and t_window > 0 else float('nan')

density_restricted_sustained_over_Twindow = density_restricted_sustained / t_window if np.isfinite(density_restricted_sustained) and t_window > 0 else float('nan')

density_restricted_held_over_Twindow = density_restricted_held / t_window if np.isfinite(density_restricted_held) and t_window > 0 else float('nan')

sim_restricted_first_over_Twindow = sim_restricted_first / t_window if np.isfinite(sim_restricted_first) and t_window > 0 else float('nan')

sim_restricted_sustained_over_Twindow = sim_restricted_sustained / t_window if np.isfinite(sim_restricted_sustained) and t_window > 0 else float('nan')

sim_restricted_held_over_Twindow = sim_restricted_held / t_window if np.isfinite(sim_restricted_held) and t_window > 0 else float('nan')

Add rows for density metrics

Note: For continue branch, AUC deficit = 0 (comparing trajectory with itself)

Held duration uses T_window for both horizon systems (Option B: documented below)

Density targets are horizon-independent (same 0.9 * D_B_final_continue for both systems)

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,

"held_time": density_held,

"held_observed": density_held_observed,

"restricted_first_crossing_time": density_restricted_first,

"restricted_sustained_time": density_restricted_sustained,

"restricted_held_time": density_restricted_held,

"auc_deficit_vs_continue": density_auc_deficit,

"first_crossing_over_Twindow": density_first_over_Twindow,

"sustained_time_over_Twindow": density_sustained_over_Twindow,

```
"held_time_over_Twindow": density_held_over_Twindow,

"restricted_first_over_Twindow": density_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": density_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": density_restricted_held_over_Twindow,

})


# Add rows for similarity metrics

# Similarity targets are horizon-dependent (different for fixed_physical vs normalized_EMA)

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "similarity",

"target_value": similarity_target,

"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",

"target_horizon_dependence": "horizon_dependent",
```

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

"restricted_first_over_Twindow": sim_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": sim_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": sim_restricted_held_over_Twindow,

```
})


if not recovery_rows:
raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")


# Validate expected count: 40 seeds × 5 rho × 4 branches × 2 metrics × 2 horizon systems = 3200
# Now includes continue branch for DID calculation
expected_count = len(seeds) *len(rho_values)* len(branches) *2* 2
if len(recovery_rows) != expected_count:
raise RuntimeError(
f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "
f"This may indicate a logic error in recovery calculation."
)


recovery_df = pd.DataFrame(recovery_rows)
```

```
print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "

f"Systems: {recovery_df['horizon_system'].nunique()}")



return recovery_df

================================================================================
Stage 1: Validation
================================================================================

def validate_run(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.
```

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

```
try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results



# Validate condition status - MANDATORY for production

condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results



try:

condition_df = [pd.read](http://pd.read)_csv(condition_status_path)
```

```
# Check for duplicate condition-status rows
if condition_df.duplicated(subset=["seed", "rho"]).any():
results["status"] = "failed"
results["errors"].append("Condition status file has duplicate (seed, rho) rows")
return results

# Validate status values
unique_statuses = set(condition_df["status"].unique())
valid_statuses = {"success", "failed"}
invalid_statuses = unique_statuses - valid_statuses
if invalid_statuses:
results["status"] = "failed"
results["errors"].append(
f"Condition status file has invalid status values: {invalid_statuses}"
)
return results
```

```
successful_conditions = (condition_df["status"] == "success").sum()

failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)


expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:

results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "

f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"
```

```
)

return results


# Validate exact condition coverage matches manifest

condition_df["seed"] = condition_df["seed"].astype(str)

condition_df["rho"] = [pd.to](http://pd.to)_numeric(condition_df["rho"], errors="coerce")


observed_conditions = set(

zip(condition_df["seed"], condition_df["rho"])

)


# Get expected conditions from manifest

try:

manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]

expected_conditions_from_manifest = {

(str(seed), float(rho))

for seed in manifest_seeds

for rho in manifest_rho_values
```

```
}

if observed_conditions != expected_conditions_from_manifest:
missing = expected_conditions_from_manifest - observed_conditions
extra = observed_conditions - expected_conditions_from_manifest
if missing:
results["status"] = "failed"
results["errors"].append(
f"Condition status missing {len(missing)} conditions: {list(missing)[:5]}"
)
return results
if extra:
results["status"] = "failed"
results["errors"].append(
f"Condition status has {len(extra)} extra conditions: {list(extra)[:5]}"
)
return results
```

```
except Exception as e:

results["warnings"].append(f"Could not validate condition coverage against manifest: {e}")


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")

return results


# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"
```

```
)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status

completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"
```

```
results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:
```

```
df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)



# Validate unique seeds

unique_seeds = df["seed"].unique()
```

```
if len(unique_seeds) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"

)



# Validate rho values

unique_rho = sorted([float(r) for r in df["rho"].unique()])

expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])

if unique_rho != expected_rho:

results["status"] = "failed"

results["errors"].append(

f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"

)



# Validate branches

unique_branches = sorted(df["branch"].unique())
```

```
expected_branches = sorted(EXPECTED_BRANCHES)

if unique_branches != expected_branches:

results["status"] = "failed"

results["errors"].append(

f"Branches mismatch: expected {expected_branches}, got {unique_branches}"

)



# Validate completeness: each seed/rho/branch combination should exist

missing_combinations = []

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")
```

```
if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate exact cohort coverage in CSV matches manifest

try:

manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
```

```
manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]

manifest_branches = manifest["protocol"]["branches"]


expected_csv_conditions = {

(str(seed), float(rho), branch)

for seed in manifest_seeds

for rho in manifest_rho_values

for branch in manifest_branches

}


observed_csv_conditions = set(

zip(df["seed"], df["rho"], df["branch"])

)


if observed_csv_conditions != expected_csv_conditions:

missing = expected_csv_conditions - observed_csv_conditions

extra = observed_csv_conditions - expected_csv_conditions

if missing:

results["status"] = "failed"
```

```
results["errors"].append(
f"CSV missing {len(missing)} conditions: {list(missing)[:5]}"
)
if extra:
results["status"] = "failed"
results["errors"].append(
f"CSV has {len(extra)} extra conditions: {list(extra)[:5]}"
)
if missing or extra:
return results
except Exception as e:
results["warnings"].append(f"Could not validate CSV cohort against manifest: {e}")


# Validate primary and secondary outcomes are present and finite
all_outcomes = PRIMARY_OUTCOMES + SECONDARY_NORMALIZED_OUTCOMES
for outcome in all_outcomes:
if outcome not in df.columns:
```

```
results["status"] = "failed"

results["errors"].append(f"Outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Outcome '{outcome}' has {non_numeric.sum()} non-numeric values"

)


# Check that values are finite (not NaN, inf, or -inf)

non_finite = ~np.isfinite(df[outcome])

if non_finite.any():

results["status"] = "failed"

results["errors"].append(

f"Outcome '{outcome}' has {non_finite.sum()} non-finite values"
```

)

P0 #2: Validate T_window exists and is consistent

T_window is study metadata, critical for recovery normalization

if "T_window" not in df.columns:

results["status"] = "failed"

results["errors"].append("T_window missing from scalar CSV")

return results

else:

tw = pd.to_numeric(df["T_window"], errors="coerce")

if (~np.isfinite(tw) | (tw <= 0)).any():

results["status"] = "failed"

results["errors"].append("T_window contains non-finite or non-positive values")

return results

Validate T_window is consistent across branches for each (seed, rho)

for rho, expected_twindow in zip(

[float(r) for r in manifest["study"]["rho_values"]],

```
[float(x) for x in manifest["study"]["T_window_values"]]

):

rho_mask = df["rho"] == rho

values = df.loc[rho_mask, "T_window"].unique()

if len(values) != 1:

results["status"] = "failed"

results["errors"].append(

f"rho={rho} has {len(values)} distinct T_window values"

)

return results

elif float(values[0]) != expected_twindow:

results["status"] = "failed"

results["errors"].append(

f"rho={rho}: T_window={values[0]} != expected {expected_twindow}"

)

return results
```

```
# P0 #3: Validate D_B_final exists and is valid

# D_B_final is used for density targets in recovery analysis

if "D_B_final" not in df.columns:

results["status"] = "failed"

results["errors"].append("D_B_final missing from scalar CSV")

return results

else:

density = [pd.to](http://pd.to)_numeric(df["D_B_final"], errors="coerce")

invalid = (~np.isfinite(density) | (density &lt; 0.0) | (density &gt; 1.0))

if invalid.any():

results["status"] = "failed"

results["errors"].append(

f"D_B_final has {int(invalid.sum())} invalid values (must be in [0,1])"

)

return results



# Get baseline equivalence certificate hash - MANDATORY for production
```

```
baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"

if not baseline_cert_path.exists():

results["status"] = "failed"

results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")

return results

baseline_cert_hash = file_sha256(baseline_cert_path)



# Validate baseline certificate contents

try:

with open(baseline_cert_path, "r") as f:

certificate = json.load(f)

cert_status = certificate.get("status", "unknown")

cert_n_seeds = certificate.get("n_seeds", 0)

cert_seed_results = certificate.get("seed_results", {})



if cert_status != "passed":

results["status"] = "failed"
```

```
results["errors"].append(
f"Baseline certificate status is '{cert_status}', expected 'passed'"
)
return results

if cert_n_seeds != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate has {cert_n_seeds} seeds, expected {EXPECTED_N_SEEDS}"
)
return results

if len(cert_seed_results) != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate has {len(cert_seed_results)} seed results, expected {EXPECTED_N_SEEDS}"
)
return results
```

```
for seed_id, seed_result in cert_seed_results.items():
if seed_result.get("status") != "passed":
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate seed {seed_id} status is '{seed_result.get('status')}', expected 'passed'"
)
return results
except Exception as e:
results["status"] = "failed"
results["errors"].append(f"Could not validate baseline certificate contents: {e}")
return results


# P0 #4: Verify simulation output hashes against manifest
# Validate that actual input files match the recorded hashes in the simulation manifest
recorded_hashes = manifest.get("output_hashes", {})
```

```
files_to_verify = {

"scalar_csv": csv_path,

"branch_npz": npz_path,

"condition_status_csv": condition_status_path,

"baseline_certificate": baseline_cert_path,

}



for label, path in files_to_verify.items():

expected_hash = recorded_hashes.get(label)

observed_hash = file_sha256(path)


if expected_hash is None:

results["status"] = "failed"

results["errors"].append(

f"Simulation manifest missing output hash for {label}"

)

return results

elif observed_hash != expected_hash:
```

```
results["status"] = "failed"

results["errors"].append(

f"Hash mismatch for {label}: expected={expected_hash}, observed={observed_hash}"

)

return results



# Record validation info

results["validation"] = {

"n_rows": actual_rows,

"n_seeds": len(unique_seeds),

"n_rho": len(unique_rho),

"n_branches": len(unique_branches),

"n_duplicates": int(duplicate_rows.sum()),

"n_missing_combinations": len(missing_combinations),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", False),

"baseline_certificate_hash": baseline_cert_hash,
```

```
}


# Also expose at top level for manifest generation

results["baseline_certificate_hash"] = baseline_cert_hash



if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")



return results
```

```
================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}



Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[
```

```
df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]

.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values
```

```
merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)



# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]
```

```
# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")


rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)
```

```
if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)



print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")



return contrasts_df

# ============================================================================
# Stage 3: Branch Difference-in-Differences
# ============================================================================

def compute_branch_did(

text
```

```
df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} -
Y_{s,1,continue}]



For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.



Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]



for rho in sorted([float(r) for r in df["rho"].unique()]):
```

```
# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]
```

```
if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c
```

```
did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count
```

```
expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches:
{did_df['branch'].nunique()}")



return did_df

================================================================================
Stage 4: Cross-T Topology and Recovery
================================================================================

def build_continue_b_final_key(seed: str, rho: float) -> str:

text
```

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))

Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)

topology_rows = []

missing_keys = []

```
for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline


# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)


if rho_key not in npz_data:

missing_keys.append(rho_key)

continue

if baseline_key not in npz_data:

missing_keys.append(baseline_key)

continue


B_rho = npz_data[rho_key]

B_baseline = npz_data[baseline_key]
```

```
# Compute metrics
cos_sim = cosine_similarity(B_rho, B_baseline)
jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)
norm_dist = normalized_distance(B_rho, B_baseline)

topology_rows.append({
"seed": seed,
"rho": rho,
"cross_T_cosine": cos_sim,
"cross_T_Jaccard": jaccard,
"cross_T_normalized_distance": norm_dist,
})

# Validate we have all expected comparisons
expected_count = EXPECTED_N_TOPOLOGY
if len(topology_rows) != expected_count:
raise RuntimeError(
f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "
```

```
f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")



return topology_df

def compute_recovery_did(

text

recovery: pd.DataFrame,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]
```

For each seed, non-baseline rho, and NON-CONTINUE branch only, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

Skips continue branch as it generates trivial zero-DID rows.

Expected count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

"""

if recovery.empty:

return pd.DataFrame()

did_rows = []

seeds = sorted(recovery["seed"].unique())

```
rho_values = sorted([float(r) for r in recovery["rho"].unique()])

horizon_systems = sorted(recovery["horizon_system"].unique())

metrics = sorted(recovery["metric"].unique())



# Recovery outcomes to compute DID for

# Prioritize censoring-aware restricted outcomes as primary

recovery_outcomes = [

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",
```

```
]



# Also compute DID for recovery indicators

indicator_outcomes = [

"first_crossing_observed",

"sustained_observed",

"held_observed",

]



for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]



for rho in rho_values:

# Skip baseline rho=1.0

if rho == 1.0:

continue



rho_data = seed_data[seed_data["rho"] == rho]
```

```
# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in NON_CONTINUE_BRANCHES:

# EXCLUDE continue branch - it generates trivial zero-DID rows

# DID is only defined for non-continue branches: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

for horizon_system in horizon_systems:

for metric in metrics:

# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]
```

```
# R_{s,ρ,c} (continue branch at same rho)

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,1,b} (baseline rho=1.0, same branch)

r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# R_{s,1,c} (baseline rho=1.0, continue)

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;
```

```
(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# Validate exactly one match for each cell - missing cells indicate schema error

matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,

"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "

f"horizon={horizon_system}, metric={metric}, "

f"outcome={outcome}, found={len(match)}"
```

```
)

r_rho_b = float(r_rho_b_match.iloc[0])
r_rho_c = float(r_rho_c_match.iloc[0])
r_base_b = float(r_base_b_match.iloc[0])
r_base_c = float(r_base_c_match.iloc[0])

# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]
branch_effect_rho = r_rho_b - r_rho_c
branch_effect_baseline = r_base_b - r_base_c
did = branch_effect_rho - branch_effect_baseline

did_rows.append({
"seed": seed,
"rho": rho,
"branch": branch,
"horizon_system": horizon_system,
"metric": metric,
"outcome": outcome,
```

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

Compute DID for recovery indicators (binary outcomes)

for outcome in indicator_outcomes:

Get values

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &

(rho_data["horizon_system"] == horizon_system) &

(rho_data["metric"] == metric)

][outcome]

```
r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]
```

```
# Validate exactly one match for each cell - missing cells indicate schema error

matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,

"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "

f"horizon={horizon_system}, metric={metric}, "

f"outcome={outcome}, found={len(match)}"

)


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])
```

```
r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# For binary outcomes, use difference in proportions

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,
```

```
"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})



if not did_rows:

return pd.DataFrame()



# Validate exact count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

if len(did_rows) != EXPECTED_N_RECOVERY_DID:

raise RuntimeError(

f"Expected exactly {EXPECTED_N_RECOVERY_DID} recovery DID rows, got {len(did_rows)}. "

f"This indicates a logic error - continue branch should be excluded."

)



did_df = pd.DataFrame(did_rows)
```

```
print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "

f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")



return did_df

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.
```

For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

```
if not regime_contrasts.empty:

# Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)
```

```
ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),
```

```
std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary
```

```
print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),

cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)
```

```
# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)

for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))
```

```
bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper


# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper
```

```
summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries

# Include both raw event times AND censoring-aware restricted times

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(

# Raw event times (conditional on observed recovery)

mean_first_crossing=("first_crossing_time", "mean"),

std_first_crossing=("first_crossing_time", "std"),

n_first_crossing=("first_crossing_time", "count"),

mean_sustained=("sustained_time", "mean"),
```

```
std_sustained=("sustained_time", "std"),

n_sustained=("sustained_time", "count"),

mean_held=("held_time", "mean"),

std_held=("held_time", "std"),

n_held=("held_time", "count"),

mean_auc_deficit=("auc_deficit_vs_continue", "mean"),

std_auc_deficit=("auc_deficit_vs_continue", "std"),

n_auc_deficit=("auc_deficit_vs_continue", "count"),


# Censoring-aware restricted times (cohort-level estimates)

mean_restricted_first=("restricted_first_crossing_time", "mean"),

std_restricted_first=("restricted_first_crossing_time", "std"),

n_restricted_first=("restricted_first_crossing_time", "count"),

mean_restricted_sustained=("restricted_sustained_time", "mean"),

std_restricted_sustained=("restricted_sustained_time", "std"),

n_restricted_sustained=("restricted_sustained_time", "count"),

mean_restricted_held=("restricted_held_time", "mean"),

std_restricted_held=("restricted_held_time", "std"),
```

```
n_restricted_held=("restricted_held_time", "count"),


# Normalized times (t/T_window) - raw observed (conditional on recovery)
mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),
std_first_over_Twindow=("first_crossing_over_Twindow", "std"),
n_first_over_Twindow=("first_crossing_over_Twindow", "count"),
mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),
std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),
n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),
mean_held_over_Twindow=("held_time_over_Twindow", "mean"),
std_held_over_Twindow=("held_time_over_Twindow", "std"),
n_held_over_Twindow=("held_time_over_Twindow", "count"),


# Censoring-aware restricted normalized times (cohort-level estimates)
mean_restricted_first_over_Twindow=("restricted_first_over_Twindow", "mean"),
std_restricted_first_over_Twindow=("restricted_first_over_Twindow", "std"),
n_restricted_first_over_Twindow=("restricted_first_over_Twindow", "count"),
mean_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "mean"),
```

```
std_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "std"),

n_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "count"),

mean_restricted_held_over_Twindow=("restricted_held_over_Twindow", "mean"),

std_restricted_held_over_Twindow=("restricted_held_over_Twindow", "std"),

n_restricted_held_over_Twindow=("restricted_held_over_Twindow", "count"),


# Recovery probabilities

observed_rate_first=("first_crossing_observed", "mean"),

observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)


# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))
```

```
bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper


# Bootstrap restricted first crossing time (cohort-level, includes censored)

restricted_first_values = group["restricted_first_crossing_time"].dropna().values

if len(restricted_first_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))

bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)
```

```
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper


# Bootstrap restricted sustained time

restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper


# Bootstrap restricted held time

restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &amp;gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper



# Bootstrap normalized first crossing time over T_window

first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values

if len(first_over_Twindow_values) &amp;gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values

if len(sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[
```

```
(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper


# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))

bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;
```

```
(recovery_summary["branch"] == branch) &amp;amp;
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
(recovery_summary["metric"] == metric)
].index
recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower
recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper

# Bootstrap AUC deficit
auc_values = group["auc_deficit_vs_continue"].dropna().values
if len(auc_values) &gt; 0:
indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))
bootstrap_means = np.nanmean(auc_values[indices], axis=1)
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[
(recovery_summary["rho"] == rho) &amp;amp;
(recovery_summary["branch"] == branch) &amp;amp;
```

```
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


# Bootstrap restricted normalized first crossing time over T_window

restricted_first_over_Twindow_values = group["restricted_first_over_Twindow"].dropna().values

if len(restricted_first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_over_Twindow_values), size=(n_bootstrap,
len(restricted_first_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
```

```
(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first_over_Twindow"] = ci_upper



# Bootstrap restricted normalized sustained time over T_window

restricted_sustained_over_Twindow_values =
group["restricted_sustained_over_Twindow"].dropna().values

if len(restricted_sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_over_Twindow_values), size=(n_bootstrap,
len(restricted_sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))



idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained_over_Twindow"] = ci_upper


# Bootstrap restricted normalized held time over T_window

restricted_held_over_Twindow_values = group["restricted_held_over_Twindow"].dropna().values

if len(restricted_held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_over_Twindow_values), size=(n_bootstrap,
len(restricted_held_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held_over_Twindow"] = ci_upper


# Bootstrap recovery probabilities (observed rates)

# First crossing observed rate

first_observed_values = group["first_crossing_observed"].dropna().values

if len(first_observed_values) &gt; 0:

indices = rng.integers(0, len(first_observed_values), size=(n_bootstrap, len(first_observed_values)))

bootstrap_means = np.nanmean(first_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_observed_rate_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_first"] = ci_upper


# Sustained observed rate

sustained_observed_values = group["sustained_observed"].dropna().values

if len(sustained_observed_values) &gt; 0:

indices = rng.integers(0, len(sustained_observed_values), size=(n_bootstrap,
len(sustained_observed_values)))

bootstrap_means = np.nanmean(sustained_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index
```

```
recovery_summary.loc[idx, "ci_lower_observed_rate_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_sustained"] = ci_upper


# Held observed rate

held_observed_values = group["held_observed"].dropna().values

if len(held_observed_values) &gt; 0:

indices = rng.integers(0, len(held_observed_values), size=(n_bootstrap, len(held_observed_values)))

bootstrap_means = np.nanmean(held_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_held"] = ci_lower
```

```
recovery_summary.loc[idx, "ci_upper_observed_rate_held"] = ci_upper


summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

# ============================================================================
# Report Generation
# ============================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,
```

```
recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,

recovery_did_summary: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})
```

```
lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:
```

```
for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)
```

```
lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)

if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:
```

```
lines.append(" No secondary regime contrast data")

lines.append("")



# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:
```

```
mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):
```

```
rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")
```

```
# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")
```

```
else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")

lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")
```

```
# Cohort-level estimates (includes censored) - restricted times

lines.append(f" Cohort-level estimates (all seeds, censoring-aware):")

lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")

lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")

if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")



# Cohort-level normalized times (restricted t/T_window)

lines.append(f" Cohort-level normalized times (restricted, all seeds):")

lines.append(f" Mean restricted first/t_window: {float(row.get('mean_restricted_first_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_first_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_first_over_Twindow', 0))})")

if "ci_lower_restricted_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first_over_Twindow']):.4f},
{float(row['ci_upper_restricted_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted sustained/t_window:
{float(row.get('mean_restricted_sustained_over_Twindow', float('nan'))):.4f} ±
{float(row.get('std_restricted_sustained_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_sustained_over_Twindow', 0))})")

if "ci_lower_restricted_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained_over_Twindow']):.4f},
{float(row['ci_upper_restricted_sustained_over_Twindow']):.4f}]")

lines.append(f" Mean restricted held/t_window: {float(row.get('mean_restricted_held_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_held_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_held_over_Twindow', 0))})")

if "ci_lower_restricted_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_held_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held_over_Twindow']):.4f},
{float(row['ci_upper_restricted_held_over_Twindow']):.4f}]")



# Recovery probabilities with CI

lines.append(f" Recovery probabilities (with 95% bootstrap CI):")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

if "ci_lower_observed_rate_first" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_first", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_first']):.4f},
{float(row['ci_upper_observed_rate_first']):.4f}]")

lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

if "ci_lower_observed_rate_sustained" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_sustained']):.4f},
{float(row['ci_upper_observed_rate_sustained']):.4f}]")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")

if "ci_lower_observed_rate_held" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_held']):.4f},
{float(row['ci_upper_observed_rate_held']):.4f}]")



# Conditional-on-recovery estimates (observed seeds only)

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")
```

```
lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")



# Conditional-on-recovery normalized times (observed seeds only)

lines.append(f" Conditional-on-recovery normalized times (observed seeds only):")

lines.append(f" First crossing/t_window: {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Sustained/t_window: {float(row['mean_sustained_over_Twindow']):.4f} ±
{float(row['std_sustained_over_Twindow']):.4f} (n={int(row['n_sustained_over_Twindow'])})")

if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")

lines.append(f" Held/t_window: {float(row['mean_held_over_Twindow']):.4f} ±
{float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")

if "ci_lower_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held_over_Twindow']):.4f},
{float(row['ci_upper_held_over_Twindow']):.4f}]")

else:

lines.append(" No recovery scaling data")

lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)
```

```
lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:

for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])

n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")
```

```
else:

lines.append(" No recovery DID data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,
```

```
bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {
```

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,

"source": {

"path": str(source_path),

"sha256": source_hash,

},

"command_line": command_line,

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

```
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--n-bootstrap",
type=int,
default=1000,
help="number of bootstrap iterations (default: 1000)",
)
parser.add_argument(
```

```
"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")
```

```
if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")

return True

def main() -> None:

text

args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)



# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:
```

```
print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



# Require exactly one input CSV and NPZ

if len(csv_files) != 1:

print(f"Expected exactly one scalar CSV, found {len(csv_files)}: {[[p.name](http://p.name) for p in
csv_files]}")

sys.exit(1)

if len(npz_files) != 1:

print(f"Expected exactly one branch NPZ, found {len(npz_files)}: {[[p.name](http://p.name) for p in
npz_files]}")

sys.exit(1)

if len(manifest_files) != 1:
```

```
print(f"Expected exactly one manifest, found {len(manifest_files)}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)



# Check for stale files in output directory

expected_outputs = [

"twindow_paired_regime_contrasts.csv",

"twindow_paired_regime_contrasts_fixed_physical.csv",
```

"twindow_paired_regime_contrasts_normalized_EMA.csv",

"twindow_branch_interactions.csv",

"twindow_branch_interactions_fixed_physical.csv",

"twindow_branch_interactions_normalized_EMA.csv",

"twindow_cross_condition_topology.csv",

"twindow_recovery_by_seed.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_regime_secondary_summary.csv",

"twindow_branch_DID_secondary_summary.csv",

"twindow_recovery_interactions.csv",

"twindow_recovery_interactions_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]

```
stale_files = []

if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)


if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)


# If --overwrite is passed, remove ALL expected files before starting

if stale_files and args.overwrite:

print(f" Removing {len(stale_files)} stale files from output directory...")

for expected_file in expected_outputs:
```

```
file_path = output_dir / expected_file

if file_path.exists():

try:

file_path.unlink()

except Exception as e:

print(f" WARNING: Could not remove stale file {expected_file}: {e}")



output_dir.mkdir(parents=True, exist_ok=True)



# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")
```

```
print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")
```

```
df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)

print("\nStage 2: Computing regime contrasts...")
```

```
regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)



# Stage 3: Branch DID (Primary outcomes - fixed_physical)
```

```
print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary branch DID into long format

branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)



# Stage 4: Cross-T Topology
```

```
print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)



# Stage 4b: Recovery Scaling (REQUIRED for production)

print("\nStage 4b: Computing recovery scaling metrics...")

recovery = compute_recovery_scaling(df, npz_path, b_c)



# Validate recovery groups have exactly 40 seeds each

print("\n Validating recovery group sizes...")

recovery_group_sizes = recovery.groupby(

["rho", "branch", "horizon_system", "metric"]

).size()



# P0 #5: Validate expected number of recovery groups

if len(recovery_group_sizes) != EXPECTED_N_RECOVERY_GROUPS:

raise RuntimeError(

f"Expected {EXPECTED_N_RECOVERY_GROUPS} recovery groups, "
```

```
f"got {len(recovery_group_sizes)}"

)



bad_groups = recovery_group_sizes[recovery_group_sizes != EXPECTED_N_SEEDS]

if not bad_groups.empty:

raise RuntimeError(

f"Recovery groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)



# Validate restricted recovery fields have finite values

print(" Validating restricted recovery fields...")

for field in RESTRICTED_RECOVERY_FIELDS:

non_finite = ~np.isfinite([pd.to](http://pd.to)_numeric(recovery[field], errors="coerce"))

if non_finite.any():

raise RuntimeError(

f"Recovery field '{field}' has {int(non_finite.sum())} non-finite values"
```

```
)


print(" ✅ Recovery validation passed")


# Stage 4c: Recovery Interaction DID
print("\nStage 4c: Computing recovery interaction DID...")
recovery_did = compute_recovery_did(recovery)


# Stage 5: Seed-Level Summaries
print("\nStage 5: Computing seed-level summaries...")
summaries = compute_seed_level_summaries(
regime_contrasts, branch_did, topology, recovery,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed,
)
```

```
# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")

secondary_summaries = compute_seed_level_summaries(

regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Validate recovery DID groups have exactly 40 seeds each

print("\n Validating recovery DID group sizes...")

recovery_did_group_sizes = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

).size()



# P0 #5: Validate expected number of recovery DID groups

if len(recovery_did_group_sizes) != EXPECTED_N_RECOVERY_DID_GROUPS:

raise RuntimeError(
```

```
f"Expected {EXPECTED_N_RECOVERY_DID_GROUPS} recovery DID groups, "

f"got {len(recovery_did_group_sizes)}"

)



if not (recovery_did_group_sizes == EXPECTED_N_SEEDS).all():

bad_groups = recovery_did_group_sizes[recovery_did_group_sizes != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Recovery DID groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)

print(" ✅ Recovery DID validation passed")



# Stage 5c: Recovery DID summaries

print("\nStage 5c: Computing recovery DID summaries...")

recovery_did_summaries = {}

if not recovery_did.empty:

recovery_did_summary = recovery_did.groupby(
```

```
["rho", "branch", "horizon_system", "metric", "outcome"],

as_index=False,

).agg(

mean=("recovery_difference_in_differences", "mean"),

std=("recovery_difference_in_differences", "std"),

n=("recovery_difference_in_differences", "count"),

)


# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):

values = group["recovery_difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;amp;

(recovery_did_summary["branch"] == branch) &amp;amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_did_summary["metric"] == metric) &amp;amp;

(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary

print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())
```

```
recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)

output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")



# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"
```

```
save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)

regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"

save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")



# Branch DID (combined primary + secondary)

branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_combined_path, branch_did_combined)

output_files["branch_did_combined"] = branch_did_combined_path

print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")



# Branch DID (primary only - fixed_physical)
```

```
branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Branch DID (secondary only - normalized_EMA)

branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"

save_csv_atomic(branch_did_secondary_path, branch_did_secondary)

output_files["branch_did_secondary"] = branch_did_secondary_path

print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")
```

```
# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)

output_files["recovery_by_seed"] = recovery_raw_path

print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"
```

```
save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:

recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")
```

```
# Recovery DID (raw)
if not recovery_did.empty:
recovery_did_path = output_dir / "twindow_recovery_interactions.csv"
save_csv_atomic(recovery_did_path, recovery_did)
output_files["recovery_did"] = recovery_did_path
print(f" Saved: {recovery_did_[path.name](http://path.name)}")


# Recovery DID summaries
if not recovery_did_summary.empty:
recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"
save_csv_atomic(recovery_did_summary_path, recovery_did_summary)
output_files["recovery_did_summary"] = recovery_did_summary_path
print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")


# Secondary summaries
if not regime_summaries_secondary.empty:
regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"
```

```
save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)

output_files["regime_summary_secondary"] = regime_summary_secondary_path

print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")



if not did_summaries_secondary.empty:

did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"

save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)

output_files["did_summary_secondary"] = did_summary_secondary_path

print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")



# Text report

report = generate_text_report(

validation, regime_contrasts, branch_did, topology,

regime_summaries, did_summaries, topology_summaries,

recovery_summaries, regime_summaries_secondary, did_summaries_secondary,

recovery_did_summary

)
```

```
report_path = output_dir / "twindow_sensitivity_report.txt"
save_text_atomic(report_path, report)
output_files["report"] = report_path
print(f" Saved: {report_[path.name](http://path.name)}")


# Analysis manifest
analysis_manifest = generate_analysis_manifest(
csv_path, npz_path, manifest_path, validation, output_files,
source_path=Path(__file__),
bootstrap_seed=args.bootstrap_seed,
n_bootstrap=args.n_bootstrap,
command_line=command_line,
)
manifest_path_out = output_dir / "analysis_manifest.json"
save_json_atomic(manifest_path_out, analysis_manifest)
output_files["analysis_manifest"] = manifest_path_out
print(f" Saved: {manifest_path_[out.name](http://out.name)}")
```

```
# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()
```

Ignoring copy/paste corruption, this version fixes the previous import-time NON_CONTINUE_BRANCHES error and closes the most important recovery-DID and provenance gaps.
Verdict

Import/runtime structure: no obvious deterministic startup failure
Primary and secondary contrasts: correct
Recovery scaling and recovery DID: structurally correct

Input provenance: substantially improved
Readiness: near-GO, with a few validation gaps worth fixing before the analysis source is frozen

Unlike the prior version, this script should import and reach Stage 1 on a valid production run.
P0: production-validation gaps
1. Manifest cohort-validation exceptions are still downgraded to warnings

In two places, inability to validate against the manifest becomes only a warning:

Python

```
except Exception as e:
results["warnings"].append(
f"Could not validate condition coverage against manifest: {e}"
)
```

and:

Python

```
except Exception as e:
results["warnings"].append(
f"Could not validate CSV cohort against manifest: {e}"
)
```

For production analysis, the manifest is mandatory authority. If its cohort, rho grid, or branch structure cannot be read, validation must fail.
Fix

Replace both warning paths with:

Python

```
except Exception as e:
results["status"] = "failed"
results["errors"].append(
f"Could not validate exact cohort against manifest: {e}"
)
```

return results

Do not continue when exact seed/condition identity is unknown.
2. Topology rows can contain NaN while still passing the 160-row check

compute_cross_t_topology() validates only the number of rows. These functions can return NaN:

Python

cosine_similarity(...)
jaccard_similarity(...)
normalized_distance(...)

A zero denominator, malformed matrix, or nonfinite matrix could therefore yield 160 rows but fewer than 40 usable seed values per rho.
Fix matrix validation

Before computing metrics:

Python

def validate_topology_matrix(
matrix: np.ndarray,
key: str,
) -> np.ndarray:
matrix = np.asarray(matrix, dtype=float)

if matrix.shape != (12, 12):
raise RuntimeError(
f"{key} shape {matrix.shape} != (12, 12)"
)

if not np.all(np.isfinite(matrix)):
raise RuntimeError(
f"{key} contains non-finite values"
)

if not np.allclose(

matrix,
matrix.T,
rtol=0.0,
atol=1e-12,
):
raise RuntimeError(f"{key} is not symmetric")

if not np.allclose(
np.diag(matrix),
0.0,
rtol=0.0,
atol=1e-12,
):
raise RuntimeError(
f"{key} diagonal is not zero"
)

if np.any(matrix < -1e-12):
raise RuntimeError(
f"{key} contains negative weights"
)

return matrix

Then:

Python

B_rho = validate_topology_matrix(
npz_data[rho_key],
rho_key,
)
B_baseline = validate_topology_matrix(
npz_data[baseline_key],
baseline_key,
)

After constructing topology_df:

Python

```
topology_columns = (
"cross_T_cosine",
"cross_T_Jaccard",
"cross_T_normalized_distance",
)

for column in topology_columns:
if not np.isfinite(topology_df[column]).all():
raise RuntimeError(
f"Topology metric {column} contains "
f"non-finite values"
)

topology_sizes = topology_df.groupby("rho").size()

if len(topology_sizes) != 4:
raise RuntimeError(
f"Expected 4 topology rho groups, "
f"got {len(topology_sizes)}"
)

if not (topology_sizes == EXPECTED_N_SEEDS).all():
raise RuntimeError(
f"Topology rho groups do not all contain "
f"{EXPECTED_N_SEEDS} seeds"
)
```

3. Recovery trajectory lengths are not checked against sample_times

Current code checks:

text

branch density versus continue density
branch similarity versus continue similarity

but not:

Python

```
len(sample_times)
== len(density_trajectory)
== len(similarity_trajectory)
```

A branch and continue could both have the same malformed density length while their time array has another length.
Fix

For branch and continue separately:

Python

```
if not (
len(sample_times)
== len(density_trajectory)
== len(similarity_trajectory)
):
raise RuntimeError(
f"Branch trajectory length mismatch: "
f"seed={seed}, rho={rho}, branch={branch}, "
f"times={len(sample_times)}, "
f"density={len(density_trajectory)}, "
f"similarity={len(similarity_trajectory)}"
)
```

```
if not (
len(cont_sample_times)
== len(cont_density_trajectory)
== len(cont_similarity_trajectory)
):
raise RuntimeError(
f"Continue trajectory length mismatch: "
f"seed={seed}, rho={rho}"
)
```

Also require the expected 640 saved samples.
4. Validate the exact saved-time schedule

The recovery estimands depend on the physical time axis and censoring horizon of 319.51. Require:

Python

```
expected_times = (
np.arange(0, 32000, 50, dtype=float) + 1.0
) * 0.01

if not np.array_equal(sample_times, expected_times):
raise RuntimeError(
f"Unexpected sample-time schedule: "
f"seed={seed}, rho={rho}, branch={branch}"
)
```

This also establishes:

text

```
first sample = 0.01
last sample = 319.51
sample spacing = 0.5
n samples = 640
```

Use the manifest’s dt, future duration, and diagnostic stride if you want to avoid duplicating constants.
5. Validate trajectory ranges

Density must be finite and in [0,1][0,1]:

Python

```
if not np.all(np.isfinite(density_trajectory)):
raise RuntimeError(...)

if np.any(
```

(density_trajectory < 0.0)
| (density_trajectory > 1.0)
):
raise RuntimeError(...)

For similarity, NaNs are permitted only as the documented zero-vector cosine condition. Finite values should satisfy:

Python

finite_similarity = similarity_trajectory[
np.isfinite(similarity_trajectory)
]

if np.any(
(finite_similarity < -1.0 - 1e-12)
| (finite_similarity > 1.0 + 1e-12)
):
raise RuntimeError(...)

Do the same for the continue trajectories.
P1: complete summary validation
6. Recovery group counts are now correct, but other summaries remain unchecked

You now correctly require:

text

80 recovery groups
624 recovery-DID groups
40 physical seed rows per group

Add analogous checks for the other summaries.
Regime summaries

Each primary or secondary set should have:
4×3=12
4×3=12

groups, each with n=40n=40.

Python

```
if len(regime_summary) != 12:
raise RuntimeError(...)
```

```
if not (regime_summary["n"] == 40).all():
raise RuntimeError(...)
```

Branch DID summaries

Each primary or secondary set should have:
4×3×3=36
4×3×3=36

groups, each with n=40n=40.
Topology summary

Require four rho rows and:

Python

```
cosine_n == 40
jaccard_n == 40
distance_n == 40
```

Recovery summaries

After aggregation require:

Python

```
len(recovery_summary) == 80
```

and all restricted-time counts equal 40.
Recovery-DID summaries

Require:

Python

len(recovery_did_summary) == 624

For restricted outcomes, AUC, and indicators, summary n must equal 40. Conditional raw recovery-time outcomes may legitimately have n < 40.
7. Verify the baseline certificate cohort, not just its size

Current validation checks:

status passed;
40 seed results;
each result passed.

Also require exact equality between:

Python

set(certificate["seed_results"])

and:

Python

{
str(seed)
for seed in manifest["randomization"]["paired_seeds"]
}

For example:

Python

```
expected_cert_seeds = {
str(seed)
for seed in manifest["randomization"]["paired_seeds"]
}
observed_cert_seeds = set(cert_seed_results)

if observed_cert_seeds != expected_cert_seeds:
results["status"] = "failed"
results["errors"].append(
"Baseline certificate seed cohort does not "
"match the simulation manifest"
)
return results
```

Also require the certificate protocol branch list to be the frozen seven-branch order.
8. Validate manifest study values against the prespecified constants

The CSV is compared against the manifest, but the manifest itself should be compared against:

Python

```
EXPECTED_RHO_VALUES
EXPECTED_BRANCHES
EXPECTED_N_SEEDS
```

Require:

Python

```
tuple(float(x) for x in manifest["study"]["rho_values"])
== EXPECTED_RHO_VALUES
```

and:

Python

tuple(manifest["protocol"]["branches"])
== EXPECTED_BRANCHES

Also require exactly 40 manifest seeds.

This prevents a self-consistent but wrong manifest/CSV pair from passing.
Recovery metadata and interpretation
9. Record the cosine zero-imputation counts in the raw table

The AUC function now clearly documents zero imputation, which is good. The raw recovery output should record when it was used.

For similarity rows add:

Python

```
"target_nan_samples_imputed_zero": int(
np.sum(~np.isfinite(similarity_trajectory))
),
"reference_nan_samples_imputed_zero": int(
np.sum(~np.isfinite(
cont_similarity_trajectory
))
),
"similarity_nan_convention": (
"undefined zero-vector cosine imputed as "
"zero for recovery and AUC only"
),
```

For density rows, either omit these fields or set both counts to zero.

This separates the saved finite trajectory fact—undefined cosine—from the analysis convention used for recovery.
10. Document the discrete held-duration definition precisely

Current rows say:

Python

"held_duration_definition": "one_current_T_window"

The implementation is actually:

Python

ceil(T_window / saved_sample_spacing)

consecutive saved samples.

Use:

Python

"held_duration_definition": (
"ceil(T_window / diagnostic_sample_spacing) "
"consecutive saved samples"
),
"held_required_samples": required_samples,
"diagnostic_sample_spacing": sample_step,

This preserves the existing historical convention without overstating it as continuous-time persistence.
11. Density rows are intentionally duplicated

Density recovery uses the same target and trajectory in both horizon systems. Those rows are duplicated intentionally so the recovery table has a balanced schema.

Add to the report and manifest:

text

Density recovery rows are duplicated across horizon_system by design.
They are horizon-independent and must not be pooled across the two labels.

Similarity targets are horizon-dependent; similarity AUC itself remains target-independent and is consequently repeated across horizon labels.
Operational integrity
12. Validate inputs before deleting stale outputs

Currently --overwrite deletes existing analysis outputs before validate_run() is called.

Safer order:

locate exactly one CSV/NPZ/manifest;
run Stage 1 validation;
only if validation passes, inspect and clean the output directory;
run analysis.

Otherwise a malformed new simulation run could cause deletion of a previously valid analysis before failing validation.
13. load_npz_selective() should raise on corrupt NPZ

It currently prints a warning and returns an empty dictionary. The caller eventually reports missing keys, but the original corruption error is obscured.

Prefer:

Python

```
if not path.is_file():
raise FileNotFoundError(path)


try:
...
except Exception as exc:
raise RuntimeError(
f"Could not load NPZ {path}: {exc}"
) from exc
```

14. CSV output should fsync the file before replacement

Use:

Python

```
with open(
temp_path,
"w",
encoding="utf-8",
newline="",
) as handle:
df.to_csv(handle, index=False)
handle.flush()
os.fsync(handle.fileno())

os.replace(temp_path, path)
```

Directory fsync can remain afterward.

This is operational hardening, not a mathematical blocker.
Reporting cleanup
15. The summary docstring still says standard error

The implementation computes sample standard deviation:

Python

```
std=("...", "std")
```

Change:

text

Std error

to:

text

Sample standard deviation

The report’s mean ± std notation is then accurate.
16. Conditional sustained/held CIs remain absent

The report checks for:

text

ci_lower_sustained
ci_upper_sustained
ci_lower_held
ci_upper_held

but they are not computed.

This is optional. Either add those conditional bootstrap intervals or remove the unused report checks.
17. Expand analysis-manifest expected counts

Add:

Python

"expected_n_recovery_rows": 3200,
"expected_n_recovery_groups": 80,
"expected_n_recovery_did_rows": 24960,
"expected_n_recovery_did_groups": 624,

Also record:

text

similarity NaN recovery convention
density horizon duplication
discrete held-duration convention

What is now correct

The important previous defects are closed:

NON_CONTINUE_BRANCHES is defined before use;
primary fixed-physical outcomes are validated;
secondary normalized outcomes are validated;
exact CSV cohort coverage is checked;
T_window is finite, positive, and condition-consistent;
D_B_final is finite and in [0,1][0,1];
baseline certificate content is inspected;
baseline certificate hash is exposed;
CSV, NPZ, condition status, and baseline certificate hashes are checked against the simulation manifest;
recovery target metadata correctly distinguishes horizon-independent density from horizon-dependent similarity;
np.trapezoid is preferred;
recovery DID excludes continue;
missing recovery-DID cells are fatal;
exact recovery-DID row count is 24,960;
recovery has exactly 80 groups;
recovery DID has exactly 624 groups;
each physical recovery and recovery-DID group has 40 seed rows;
stale-file protection covers all generated outputs;
the seed remains the inferential unit;
fixed-primary and normalized-secondary estimands remain separate.

Final decision

There is no longer an obvious deterministic startup or estimand-construction bug. Before freezing this analysis driver, add:

fatal manifest-cohort validation;
topology matrix/metric finiteness checks;
exact trajectory schedule/range checks;
exact summary group-count checks.

After those, this analysis script is ready for the full post-grid analysis. It has no role in authorizing the baseline-only run; that remains the responsibility of the corrected engine and study driver.

Lets do another round of Evaluation of our three working files, here is the first file:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
==
====
Frozen Production Constants (DO NOT CHANGE)
==
====

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = float(np.exp(-FROZEN_DT / FROZEN_T_WINDOW))

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END / FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END / FROZEN_DT)) # 8750
Baseline horizon metadata for validation

BASELINE_HORIZON_METADATA = {

text

"window_steps": FROZEN_WINDOW_STEPS,

"recovery_end_ema": FROZEN_RECOVERY_END,

"fixed_immediate_end": FIXED_PHYSICAL_IMMEDIATE_STEPS,

"fixed_recovery_end": FIXED_PHYSICAL_RECOVERY_STEPS,

}
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

def T_star(self) -> float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega

@property

def alpha(self) -> float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return float(np.exp(-self.dt / self.T_window))

@property

def capacity(self) -> float:

"""Projection capacity: (a - delta) / q"""

return (self.a - self.delta) / self.q

@property

def delay_steps(self) -> int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))

```
def validate(self) -&gt; None:

"""Validate model configuration with strict consistency checks.


Uses rtol=0, atol=0 for exact compatibility with frozen production.

All scalar parameters must be finite.

"""

# P0 #5: Validate N is a positive integer (not a float)

if (

isinstance(self.N, (bool, np.bool_))

or not isinstance(self.N, (int, np.integer))

or self.N &lt;= 0

):

raise ValueError("N must be a positive integer")


# Validate all other scalar parameters are finite

if not np.isfinite(self.dt) or self.dt &lt;= 0:

raise ValueError("dt must be finite and positive")
```

if not np.isfinite(self.tau) or self.tau <= 0:

raise ValueError("tau must be finite and positive")

if not np.isfinite(self.a) or self.a <= 0:

raise ValueError("a must be finite and positive")

if not np.isfinite(self.q) or self.q <= 0:

raise ValueError("q must be finite and positive")

if not np.isfinite(self.delta) or self.delta < 0:

raise ValueError("delta must be finite and nonnegative")

if not np.isfinite(self.eta) or self.eta < 0:

raise ValueError("eta must be finite and nonnegative")

if not np.isfinite(self.mu) or self.mu < 0:

raise ValueError("mu must be finite and nonnegative")

if not np.isfinite(self.gamma) or self.gamma < 0:

raise ValueError("gamma must be finite and nonnegative")

if not np.isfinite(self.T_window) or self.T_window <= 0:

raise ValueError("T_window must be finite and positive")

if not np.isfinite(self.B_c) or self.B_c < 0:

raise ValueError("B_c must be finite and nonnegative")

if not np.isfinite(self.B_init_scale) or self.B_init_scale < 0:

raise ValueError("B_init_scale must be finite and nonnegative")

if not np.isfinite(self.scale_epsilon) or self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be finite and positive")

Validate N is positive integer

if self.N <= 0:

raise ValueError("N must be positive")

Validate omega

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if not np.all(np.isfinite(self.omega)):

raise ValueError("omega must contain only finite values")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

Strict consistency: delay_steps * dt must equal tau (exact)

if not np.isclose(self.delay_steps * self.dt, self.tau, rtol=0.0, atol=0.0):

raise ValueError(

f"delay_steps *dt ({self.delay_steps}* {self.dt} = {self.delay_steps * self.dt}) "

f"must equal tau ({self.tau}) exactly"

)

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model with strict consistency checks.

Uses rtol=0, atol=0 for exact compatibility with frozen production.

"""

P0 #6: Validate protocol values are finite

if not np.all(np.isfinite([self.T_sim, self.burn_end_time, self.split_time, self.future_duration])):

raise ValueError("All protocol time parameters must be finite")

P0 #6: Validate branches are non-empty and have no duplicates

if not self.branches:

raise ValueError("branches must be nonempty")

```
if len(self.branches) != len(set(self.branches)):

raise ValueError("protocol branches contain duplicates")


# P1 #10-11: Validation hardening - integer validation

# Validate diagnostic_stride is a positive integer (not bool)

if (

isinstance(self.diagnostic_stride, (bool, np.bool_))

or not isinstance(self.diagnostic_stride, (int, np.integer))

or self.diagnostic_stride &lt;= 0

):

raise ValueError("diagnostic_stride must be a positive integer")


if self.T_sim &lt;= 0:

raise ValueError("T_sim must be positive")

if not (0 &lt; self.burn_end_time &lt; self.split_time &lt; self.T_sim):

raise ValueError("Invalid time parameters: must have 0 &lt; burn_end &lt; split &lt; T_sim")

if self.future_duration &lt;= 0:

raise ValueError("future_duration must be positive")
```

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Strict consistency: split_time + future_duration must equal T_sim

if not np.isclose(self.split_time + self.future_duration, self.T_sim, rtol=0.0, atol=0.0):

raise ValueError(

f"split_time ({self.split_time}) + future_duration ({self.future_duration}) "

f"= {self.split_time + self.future_duration} must equal T_sim ({self.T_sim}) exactly"

)

Validate that all time parameters are compatible with dt (exact divisibility)

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt}) exactly")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=0.0, atol=0.0):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt}) exactly")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Cached Utility Values
==========
Cache for compute_edges to avoid recomputing on every step

EDGESCACHE: dict[int, tuple[np.ndarray, np.ndarray]] = {}
Cache for omega array conversion

OMEGAARRAY_CACHE: dict[tuple[float, ...], np.ndarray] = {}
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges.

Cached to avoid recomputing on every step.

"""

if N not in *EDGES*CACHE:

*EDGES*CACHE[N] = np.triu_indices(N, k=1)

return *EDGES*CACHE[N]

def get_omega_array(omega_tuple: tuple[float, ...]) -> np.ndarray:

text

"""Get omega as numpy array with caching.

Cached to avoid recomputing on every step.

"""

if omega_tuple not in *OMEGA*ARRAY_CACHE:

*OMEGA*ARRAY_CACHE[omega_tuple] = np.asarray(omega_tuple, dtype=float)

return *OMEGA*ARRAY_CACHE[omega_tuple]

def validate_B_matrix(B: np.ndarray, N: int, eps: float = FROZEN_ZERO_TOL) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

Recommended #5: Use strict rtol=0.0, atol=FROZEN_ZERO_TOL

if not np.allclose(B, B.T, rtol=0.0, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, rtol=0.0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B < -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

```
"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()
```

```
def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""
```

```
text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
```

RNG Utilities (matching frozen production exactly)
==========

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].

This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.

Frozen: *, branch*root = SeedSequence(seed).spawn(2)

```
branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

================================================================================
Projection Accumulator (matching production)
================================================================================

@dataclass

class ProjectionTotals:
```

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -> None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -> dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations > 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()

Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]

def clone(self) -> "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)

@property

def N(self) -> int:

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency.

Checks:

- State dimension matches config.N

- History length is exactly delay_steps + 1

- All arrays have correct shapes

- B matrix is symmetric, zero-diagonal, non-negative

- All history vectors have shape (N,)

- All arrays are finite

```
"""
# Check state dimension matches config
if self.N != config.N:
raise RuntimeError(
f"State N ({self.N}) does not match config N ({config.N})"
)

# Check history length
expected_history_len = config.delay_steps + 1
if len(self.history) != expected_history_len:
raise RuntimeError(
f"History length {len(self.history)} != expected {expected_history_len} "
f"(delay_steps={config.delay_steps}) at step {step}"
)

# Check all history vectors have correct shape
for i, h in enumerate(self.history):
if h.shape != (self.N,):
```

```
raise RuntimeError(
f"History[{i}] shape {h.shape} != ({self.N},)"
)

# Check theta shape
if self.theta.shape != (self.N,):
raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape
if self.B.shape != (self.N, self.N):
raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape
if self.coherence_acc.shape != (self.N, self.N):
raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check all arrays are finite
if not np.all(np.isfinite(self.theta)):
raise RuntimeError("theta contains non-finite values")
```

```
if not np.all(np.isfinite(self.B)):

raise RuntimeError("B contains non-finite values")

if not np.all(np.isfinite(self.coherence_acc)):

raise RuntimeError("coherence_acc contains non-finite values")

for i, h in enumerate(self.history):

if not np.all(np.isfinite(h)):

raise RuntimeError(f"History[{i}] contains non-finite values")


# Check B symmetry

# Recommended #5: Use strict rtol=0.0, atol=FROZEN_ZERO_TOL

if not np.allclose(self.B, self.B.T, rtol=0.0, atol=FROZEN_ZERO_TOL):

raise RuntimeError("B matrix is not symmetric")


# Check B diagonal

if not np.allclose(np.diag(self.B), 0, rtol=0.0, atol=FROZEN_ZERO_TOL):

raise RuntimeError("B matrix diagonal is not zero")


# Check B non-negativity
```

if np.any(self.B < -FROZEN_ZERO_TOL):

raise RuntimeError("B matrix has negative entries")

==========
Simulation Core (exact reproduction of frozen production)
==========

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.

Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

```
rng = make_prefix_rng(seed)



# Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi



# Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)



# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]
```

```
return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N
```

```
delayed = state.history[0]



# Convert omega to numpy array for indexing (cached)

omega_arr = get_omega_array([config.omega](http://config.omega))



dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)
```

```
# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.



Exact reproduction of production advance_state().
```

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values

Update B

B_unprojected = state.B + config.dt * (

(config.eta + config.mu) * R_driver

- config.mu

- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)

```
# Projection diagnostics (matching production exactly)

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))



# Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))



# Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))
```

projection_active = bool(correction_norm > FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)

Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

Recommended #4: Use named constant FROZEN_GAP_TOL

if g_min < config.delta - FROZEN_GAP_TOL:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")

Update theta

delayed = state.history[0]

```
omega_arr = get_omega_array([config.omega](http://config.omega))

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {

"pre_violation": pre_violation,

"projection_active": projection_active,
```

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)

Get RNG for prefix simulation loop

```
# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0



burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)

# P1 #10-11: State validation at transition boundaries

if step == burn_end or step == hist_end - 1:

state.validate(config, step)
```

if burn_end <= step < hist_end:

R_past_sum += R_physical

R_past_count += 1

prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count > 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.

Exact reproduction of production apply_intervention().

"""

Validate branch is known

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}")

N = state.N

if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

Zero coherence accumulator

```
state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:

# Zero B

state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text
```

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

```
window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)



# Fixed physical horizons (NOT T_window dependent)

# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {
```

```
"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# B matrices (mean B values)

B_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),
```

```
"late": np.zeros((N, N), dtype=float),

}

B_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)
```

```
# Projection accumulator (matching production)
projection = ProjectionTotals()

# Trajectory accumulators
sample_times = []
density_trajectory = []
similarity_trajectory = []

# For retention/emergence calculations
r_sum = 0.0
r_count = 0
r_late_ema_sum = 0.0
r_late_ema_count = 0
r_late_fixed_sum = 0.0
r_late_fixed_count = 0

B_reference = prefix["B_at_split"]
```

```
shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):

R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)


# Update projection accumulator

projection.update(info)


# P1 #10-11: State validation at transition boundaries

if step == 0 or step == window_steps or step == recovery_end_ema or step == fixed_immediate_end or step == fixed_recovery_end:

state.validate(config, step)


# Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver
```

```
# Determine which horizons this step belongs to for BOTH systems

selected = (state.B &gt; config.B_c).astype(float)


# === EMA-Normalized Horizons ===

# Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1


# immediate: 0 to window_steps (exclusive)

if step &lt; window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1


# recovery: window_steps to recovery_end_ema (exclusive)

if window_steps &lt;= step &lt; recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B
```

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1

recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end <= step < fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1

late: fixed_recovery_end to end

if fixed_recovery_end <= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1

Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)

```
# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value
```

```
r_count += 1



# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:

r_late_ema_sum += r_value

r_late_ema_count += 1



if fixed_recovery_end &lt;= step:

r_late_fixed_sum += r_value

r_late_fixed_count += 1




# Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] &gt; 0 else
np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] &gt; 0 else
np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}
```

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps > 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps > 0 else np.zeros((N, N))

Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)

Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)

```
# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_ema = (

float(np.mean(p_late_edges_ema[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Fixed physical

p_late_edges_fixed = P_fixed["late"][edges]

retention_fixed = (
```

```
float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_fixed = (

float(np.mean(p_late_edges_fixed[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)
```

```
# Build result with BOTH horizon systems

result = {

# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,


# === Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],
```

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),

Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

**projection.metrics(),

Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

}

return result

==
Metric Helpers
==

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

```
y = current[edges] &gt; threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x &amp; y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &lt; 3 or np.std(x) &lt; 1e-12 or np.std(y) &lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:
```

```
text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &lt;= 1e-12 or sum_sq &lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)
```

```
def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total &lt;= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities &gt; 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

def time_to_at_least(

text

sample_times: np.ndarray,

values: np.ndarray,

target: float,

) -> float:
```

```
text
```

"""Find the first time at which values reach at least the target.

Matches frozen production exactly.

Returns NaN if target is never reached.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values >= target))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def time_to_below(

```
text
```

sample_times: np.ndarray,

values: np.ndarray,

threshold: float,

) -> float:

text

"""Find the first time at which values drop below the threshold.

Matches frozen production exactly.

Returns NaN if threshold is never dropped below.

"""

sample_times = np.asarray(sample_times, dtype=float)

values = np.asarray(values, dtype=float)

valid = np.isfinite(values)

indices = np.flatnonzero(valid & (values < threshold))

return float(sample_times[indices[0]]) if len(indices) > 0 else float('nan')

def finalize_historical_branch_metrics(

text

branch_results: dict[str, dict[str, Any]],

) -> None:

text

"""Compute and add the 8 historical scalar fields that compare branches to continue.

These fields are computed in frozen production by run_seed() after all branches

are simulated. They must be computed here to match the frozen schema.

The 8 fields are:

- Time_to_50pct_continue_final_density

- Time_to_90pct_continue_final_density

- Time_to_50pct_continue_late_similarity

- Time_to_90pct_continue_late_similarity

- Time_to_similarity_below_0_9

- Time_to_similarity_below_0_5

- Late_persistence_ratio_to_continue

- Late_similarity_ratio_to_continue

These are computed for each branch relative to the continue branch reference values.

"""

Get continue branch results for reference

```
if "continue" not in branch_results:

raise RuntimeError("continue branch is required for historical metric computation")



continue_result = branch_results["continue"]



# Get reference values from continue branch

reference_density = float(continue_result["D_B_final"])

reference_late_similarity = float(continue_result["Weighted_similarity_late"])

reference_late_persistence = float(continue_result["P_mean_late"])



for branch, result in branch_results.items():

# Use direct indexing to fail if keys are missing

times = result["sample_times"]

density = result["density_trajectory"]

similarity = result["similarity_trajectory"]



# Compute time-based metrics relative to continue branch reference
```

```
# Compute time-based metrics relative to continue branch reference
result["Time_to_50pct_continue_final_density"] = time_to_at_least(
times, density, 0.5 * reference_density
)
result["Time_to_90pct_continue_final_density"] = time_to_at_least(
times, density, 0.9 * reference_density
)
result["Time_to_50pct_continue_late_similarity"] = time_to_at_least(
times, similarity, 0.5 * reference_late_similarity
)
result["Time_to_90pct_continue_late_similarity"] = time_to_at_least(
times, similarity, 0.9 * reference_late_similarity
)
result["Time_to_similarity_below_0_9"] = time_to_below(
times, similarity, 0.9
)
result["Time_to_similarity_below_0_5"] = time_to_below(
times, similarity, 0.5
```

```
)

# Compute ratio metrics - use direct indexing to fail if missing
branch_late_persistence = float(result["P_mean_late"])
branch_late_similarity = float(result["Weighted_similarity_late"])

# P0 #8: Restore literal frozen persistence-ratio logic
# Frozen production uses: if abs(reference_late_persistence) &gt; ZERO_TOL
# NOT: if np.isfinite(reference_late_persistence) and abs(...) &gt; ZERO_TOL
# For persistence ratio, use the exact frozen condition
if abs(reference_late_persistence) &gt; FROZEN_ZERO_TOL:
result["Late_persistence_ratio_to_continue"] = (
branch_late_persistence / reference_late_persistence
)
else:
result["Late_persistence_ratio_to_continue"] = float('nan')

# For similarity ratio, keep the np.isfinite() check as frozen source uses it there
```

if np.isfinite(reference_late_similarity) and abs(reference_late_similarity) > FROZEN_ZERO_TOL:

result["Late_similarity_ratio_to_continue"] = (

branch_late_similarity / reference_late_similarity

)

else:

result["Late_similarity_ratio_to_continue"] = float('nan')

==========
Required Output Schema for Baseline Equivalence
==========
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
Extended matrix equivalence map for internal consistency validation at rho=1
Maps extended matrix keys to their frozen equivalents

EXTENDED_MATRIX_EQUIVALENCE_MAP = {

text

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS +
EXTENDED_BRANCH_MATRIX_KEYS
P0 #7: Matrix schema-map integrity should mirror scalar integrity
Validate no duplicates in extended matrix keys

if len(EXTENDED_BRANCH_MATRIX_KEYS) !=
len(set(EXTENDED_BRANCH_MATRIX_KEYS)):

text

duplicates = [k for k in EXTENDED_BRANCH_MATRIX_KEYS if
list(EXTENDED_BRANCH_MATRIX_KEYS).count(k) > 1]

raise RuntimeError(f"Duplicate extended matrix schema keys: {sorted(set(duplicates))}")

Validate matrix map consistency

if set(EXTENDED_BRANCH_MATRIX_KEYS) !=
set(EXTENDED_MATRIX_EQUIVALENCE_MAP.keys()):

text

missing_map = set(EXTENDED_BRANCH_MATRIX_KEYS) -
set(EXTENDED_MATRIX_EQUIVALENCE_MAP.keys())

extra_map = set(EXTENDED_MATRIX_EQUIVALENCE_MAP.keys()) -
set(EXTENDED_BRANCH_MATRIX_KEYS)

raise RuntimeError(

f"Extended matrix map mismatch: "

f"missing={sorted(missing_map)}, extra={sorted(extra_map)}"

)

FROZEN BRANCH SCALAR KEYS: All 42 historical fields from frozen production
These MUST exist in the frozen CSV and are validated against it

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Historical cross-branch time-based metrics (from frozen production)

"Time_to_50pct_continue_final_density",

"Time_to_90pct_continue_final_density",

"Time_to_50pct_continue_late_similarity",

"Time_to_90pct_continue_late_similarity",

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
EXTENDED BRANCH SCALAR KEYS: New fields in v1 for dual-horizon support
These do NOT exist in frozen CSV/NPZ, so are validated internally at rho=1

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Additional extended scalars for complete coverage

"P_mean_full_fixed_physical",

"P_mean_immediate_fixed_physical",

"P_mean_recovery_fixed_physical",

"P_mean_full_normalized_EMA",

"P_mean_immediate_normalized_EMA",

"P_mean_recovery_normalized_EMA",

"Retention_selected_late_fixed_physical",

"Emergence_unselected_late_fixed_physical",

"Retention_selected_late_normalized_EMA",

"Emergence_unselected_late_normalized_EMA",

"Corr_Rpast_Pfull_fixed_physical",

"Corr_Rpast_Pfull_normalized_EMA",

"Jaccard_final_fixed_physical",

)
Extended scalar equivalence map for internal consistency validation at rho=1

Maps extended keys to their frozen equivalents

EXTENDED_SCALAR_EQUIVALENCE_MAP = {

text

"P_mean_full_fixed_physical": "P_mean_full",

"P_mean_immediate_fixed_physical": "P_mean_immediate",

"P_mean_recovery_fixed_physical": "P_mean_recovery",

"P_mean_late_fixed_physical": "P_mean_late",

"P_mean_full_normalized_EMA": "P_mean_full",

"P_mean_immediate_normalized_EMA": "P_mean_immediate",

"P_mean_recovery_normalized_EMA": "P_mean_recovery",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"Corr_Rpast_Pfull_fixed_physical": "Corr_Rpast_Pfull",

"Corr_Rpast_Pfull_normalized_EMA": "Corr_Rpast_Pfull",

"r_mean_late_fixed_physical": "r_mean_late",

"r_mean_late_normalized_EMA": "r_mean_late",

"Retention_selected_late_fixed_physical": "Retention_selected_late",

"Retention_selected_late_normalized_EMA": "Retention_selected_late",

"Emergence_unselected_late_fixed_physical": "Emergence_unselected_late",

"Emergence_unselected_late_normalized_EMA": "Emergence_unselected_late",

"Jaccard_final_fixed_physical": "Jaccard_final",

}
Validate no duplicates in extended scalar keys

if len(EXTENDED_BRANCH_SCALAR_KEYS) !=
len(set(EXTENDED_BRANCH_SCALAR_KEYS)):

text

duplicates = [k for k in EXTENDED_BRANCH_SCALAR_KEYS if
list(EXTENDED_BRANCH_SCALAR_KEYS).count(k) > 1]

raise RuntimeError(f"Duplicate extended scalar schema keys: {sorted(set(duplicates))}")

Validate schema-map consistency

if set(EXTENDED_BRANCH_SCALAR_KEYS) !=
set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys()):

text

missing_map = set(EXTENDED_BRANCH_SCALAR_KEYS) - set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys())

extra_map = set(EXTENDED_SCALAR_EQUIVALENCE_MAP.keys()) - set(EXTENDED_BRANCH_SCALAR_KEYS)

raise RuntimeError(

f"Extended scalar map mismatch: "

f"missing mappings={sorted(missing_map)}, extra mappings={sorted(extra_map)}"

)

All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

```
if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],
```

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

from pathlib import Path

import hashlib

Preflight validation: validate config and protocol before any simulation

try:

config.validate()

except Exception as e:

return {

"status": "failed",

"error": f"Config validation failed: {e}",

```
"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,
"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,
"frozen_csv_path": frozen_csv_path,
}


try:
protocol.validate(config)
except Exception as e:
return {
"status": "failed",
"error": f"Protocol validation failed: {e}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,
"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,
"frozen_csv_path": frozen_csv_path,
}


# Helper function to compute source hash
```

def *source*sha256() -> str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

P1 #6: Preflight all authority files - check existence before opening

Check frozen manifest exists

if not os.path.isfile(frozen_manifest_path):

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,

"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,

"frozen_csv_path": frozen_csv_path,

}

```
# Check frozen NPZ exists
if not os.path.isfile(frozen_npz_path):
return {
"status": "failed",
"error": f"Frozen NPZ not found: {frozen_npz_path}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()) if frozen_manifest_path else None,
"frozen_npz": str(Path(frozen_npz_path).resolve()) if frozen_npz_path else None,
"frozen_csv_path": frozen_csv_path,
}


# Early failure: frozen scalar CSV is required for scalar equivalence
if frozen_csv_path is None:
return {
"status": "failed",
"error": "frozen scalar CSV is required for scalar equivalence",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
```

"frozen_csv_path": None,

}

Check frozen CSV exists (if provided)

if frozen_csv_path is not None and not os.path.isfile(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

Record engine source hash

engine_source_hash = *source*sha256()

results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

```
"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

"diagnostic_stride": protocol.diagnostic_stride,

"branches": list(protocol.branches),

},

"n_seeds": len(seeds),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"engine_source_sha256": engine_source_hash,

"seed_results": {},

}
```

```
# P0 #9: Catch malformed manifest - wrap in try/except

try:

# Load frozen manifest

with open(frozen_manifest_path, "r") as f:

frozen_manifest = json.load(f)


# Validate required manifest structure before indexing

if "randomization" not in frozen_manifest:

return {

"status": "failed",

"error": "Frozen manifest missing 'randomization' section",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if "protocol" not in frozen_manifest:

return {
```

```
"status": "failed",

"error": "Frozen manifest missing 'protocol' section",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if "model" not in frozen_manifest:

return {

"status": "failed",

"error": "Frozen manifest missing 'model' section",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if "schema" not in frozen_manifest:

return {

"status": "failed",
```

"error": "Frozen manifest missing 'schema' section",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if "paired_seeds" not in frozen_manifest["randomization"]:

return {

"status": "failed",

"error": "Frozen manifest missing 'randomization.paired_seeds'",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if "branches" not in frozen_manifest["protocol"]:

return {

"status": "failed",

"error": "Frozen manifest missing 'protocol.branches'",

```
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}


# P0 #10: Compare manifest schema to expected frozen schema

# Validate schema structure

manifest_schema = frozen_manifest.get("schema", {})


manifest_prefix = tuple(manifest_schema.get("prefix_keys", []))

manifest_matrices = tuple(manifest_schema.get("branch_matrix_keys", []))

manifest_scalars = tuple(manifest_schema.get("branch_scalar_keys", []))


# Nonblocking #6: Check manifest schema uniqueness

for schema_name, schema_values in (

("prefix_keys", manifest_prefix),

("branch_matrix_keys", manifest_matrices),

("branch_scalar_keys", manifest_scalars),

):
```

```
if len(schema_values) != len(set(schema_values)):

duplicates = [v for v in schema_values if list(schema_values).count(v) &gt; 1]

return {

"status": "failed",

"error": (

f"Frozen manifest schema "

f"{schema_name} contains duplicates: {sorted(set(duplicates))}"

),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}


# Validate prefix keys

if set(manifest_prefix) != set(REQUIRED_PREFIX_KEYS):

missing_prefix = set(REQUIRED_PREFIX_KEYS) - set(manifest_prefix)

extra_prefix = set(manifest_prefix) - set(REQUIRED_PREFIX_KEYS)
```

```
return {
"status": "failed",
"error": (
f"Manifest prefix_keys mismatch: "
f"missing={sorted(missing_prefix)}, extra={sorted(extra_prefix)}"
),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}

# Validate matrix keys - compare unique sets
if set(manifest_matrices) != set(FROZEN_BRANCH_MATRIX_KEYS):
missing_matrices = set(FROZEN_BRANCH_MATRIX_KEYS) - set(manifest_matrices)
extra_matrices = set(manifest_matrices) - set(FROZEN_BRANCH_MATRIX_KEYS)
return {
"status": "failed",
"error": (
```

```
f"Manifest branch_matrix_keys mismatch: "

f"missing={sorted(missing_matrices)}, extra={sorted(extra_matrices)}"

),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}


# Validate scalar keys - compare unique sets

if set(manifest_scalars) != set(FROZEN_BRANCH_SCALAR_KEYS):

missing_scalars = set(FROZEN_BRANCH_SCALAR_KEYS) - set(manifest_scalars)

extra_scalars = set(manifest_scalars) - set(FROZEN_BRANCH_SCALAR_KEYS)

return {

"status": "failed",

"error": (

f"Manifest branch_scalar_keys mismatch: "

f"missing={sorted(missing_scalars)}, extra={sorted(extra_scalars)}"

),
```

```
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

except (json.JSONDecodeError, OSError) as e:

return {

"status": "failed",

"error": f"Malformed frozen manifest {frozen_manifest_path}: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #4: Validate seed/branch authority - seeds and branches must match manifest

manifest_seeds = [int(x) for x in frozen_manifest["randomization"]["paired_seeds"]]

manifest_branches = tuple(frozen_manifest["protocol"]["branches"])

if list(seeds) != manifest_seeds:
```

```
return {
"status": "failed",
"error": f"Seed authority mismatch: seeds={seeds}, manifest={manifest_seeds}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
if tuple(branch_order) != manifest_branches:
return {
"status": "failed",
"error": f"Branch authority mismatch: branch_order={branch_order}, manifest={manifest_branches}",
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}


# P0 #1: Validate protocol.branches matches manifest branches
```

```
# This is critical: branch_order defines RNG namespace, but protocol.branches defines executed branches

# Both must match the frozen manifest to prevent false-pass

if tuple(protocol.branches) != manifest_branches:

return {

"status": "failed",

"error": (

"Executed baseline branch mismatch: "

f"protocol.branches={protocol.branches}, "

f"manifest={manifest_branches}"

),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #3: Validate baseline horizon metadata

# At rho=1 (T_window = T_star), horizon metadata must match frozen values
```

```
if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

expected_metadata = BASELINE_HORIZON_METADATA

actual_window_steps = int(round(config.T_window / config.dt))

actual_recovery_end_ema = min(5 * actual_window_steps, protocol.future_steps(config))

actual_fixed_immediate_end = int(round(17.5 / config.dt))

actual_fixed_recovery_end = int(round(87.5 / config.dt))


if actual_window_steps != expected_metadata["window_steps"]:

return {

"status": "failed",

"error": f"window_steps mismatch: actual={actual_window_steps},
expected={expected_metadata['window_steps']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_recovery_end_ema != expected_metadata["recovery_end_ema"]:

return {

"status": "failed",
```

```
"error": f"recovery_end_ema mismatch: actual={actual_recovery_end_ema},
expected={expected_metadata['recovery_end_ema']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_fixed_immediate_end != expected_metadata["fixed_immediate_end"]:

return {

"status": "failed",

"error": f"fixed_immediate_end mismatch: actual={actual_fixed_immediate_end},
expected={expected_metadata['fixed_immediate_end']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

if actual_fixed_recovery_end != expected_metadata["fixed_recovery_end"]:

return {

"status": "failed",
```

```
"error": f"fixed_recovery_end mismatch: actual={actual_fixed_recovery_end},
expected={expected_metadata['fixed_recovery_end']}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #9: Catch malformed NPZ - wrap in try/except

try:

# Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)

except Exception as e:

return {

"status": "failed",

"error": f"Malformed frozen NPZ {frozen_npz_path}: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),
```

```
"frozen_csv_path": frozen_csv_path,

}



# P1 #7: Preflight NPZ schema - validate expected keys exist

# Check prefix keys for all seeds

for seed in seeds:

seed_str = str(seed)

for key in REQUIRED_PREFIX_KEYS:

frozen_key = f"prefix_seed_{seed_str}_{key}"

if frozen_key not in frozen_files:

return {

"status": "failed",

"error": f"Missing NPZ key: {frozen_key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}
```

```
# Check branch matrix keys for all seeds and branches

for seed in seeds:

seed_str = str(seed)

for branch in branch_order:

for key in FROZEN_BRANCH_MATRIX_KEYS:

frozen_key = f"{branch}_seed_{seed_str}_{key}"

if frozen_key not in frozen_files:

return {

"status": "failed",

"error": f"Missing NPZ key: {frozen_key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Load frozen scalar CSV - REQUIRED for scalar equivalence
```

```
# Fail immediately if CSV is missing or invalid

if frozen_csv_path is None or not os.path.isfile(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



frozen_scalar_csv: dict[str, dict[str, Any]] = {}

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for line_number, row in enumerate(reader, start=2):

seed_id = row.get("seed", "")
```

```
branch = row.get("branch", "")
# Nonblocking #7: Reject malformed rows explicitly
if not seed_id or not branch:
return {
"status": "failed",
"error": (
f"Frozen CSV line {line_number} "
f"missing seed or branch"
),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"frozen_csv_path": frozen_csv_path,
}
key = f"{branch}_seed_{seed_id}"
# P1 #5: CSV duplicate detection - fail if duplicate (seed, branch) rows
if key in frozen_scalar_csv:
return {
"status": "failed",
```

```
"error": f"Duplicate CSV row detected for {key}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

return {

"status": "failed",

"error": f"Could not load frozen CSV {frozen_csv_path}: {e}",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
```

```
"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# P0 #3: Validate exact CSV key coverage

# Expected: all seeds × all branch_order combinations

expected_csv_keys = {

f"{branch}_seed_{seed}"

for seed in seeds

for branch in branch_order

}

actual_csv_keys = set(frozen_scalar_csv.keys())



if actual_csv_keys != expected_csv_keys:

missing = sorted(expected_csv_keys - actual_csv_keys)

extra = sorted(actual_csv_keys - expected_csv_keys)

return {
```

```
"status": "failed",

"error": (

"Frozen CSV condition-key mismatch: "

f"{len(missing)} missing, {len(extra)} extra"

),

"missing_csv_keys": missing,

"extra_csv_keys": extra,

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Validate all expected scalar columns exist in every CSV row

for csv_key, row_values in frozen_scalar_csv.items():

missing = set(FROZEN_BRANCH_SCALAR_KEYS) - set(row_values)

if missing:

return {
```

"status": "failed",

"error": f"Frozen CSV row {csv_key} missing {len(missing)} required fields",

"missing_columns": sorted(missing),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}

Validate frozen CSV has all expected seeds and branches

Expected: 40 seeds × 7 branches = 280 rows

expected_csv_rows = len(seeds) * len(branch_order)

actual_csv_rows = len(frozen_scalar_csv)

if actual_csv_rows != expected_csv_rows:

return {

"status": "failed",

"error": f"Frozen CSV has {actual_csv_rows} rows, expected {expected_csv_rows} (40 seeds × 7 branches)",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

```
"frozen_npz": str(Path(frozen_npz_path).resolve()),

"frozen_csv_path": frozen_csv_path,

}



# Record frozen hashes - use actual file hashes, not canonical JSON hashes

# Record frozen hashes - use actual file hashes, not canonical JSON hashes

results["frozen_manifest_file_sha256"] = file_sha256(frozen_manifest_path)

results["frozen_npz_file_sha256"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_scalar_csv_file_sha256"] = file_sha256(frozen_csv_path)



# Also record engine source hash

results["engine_source_sha256"] = engine_source_hash



# Also keep canonical JSON hash for reference

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()
```

).hexdigest()

Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]

Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results

Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list(config.omega), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list(config.omega)}"

return results

Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

```
for name, frozen_val, config_val in model_checks:
if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):
results["status"] = "failed"
results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"
return results


# Check derived values
# Recommended #3: Use exact comparison for alpha (rtol=0, atol=0)
if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=0.0):
results["status"] = "failed"
results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"
return results


# P0 #4: Explicitly validate the frozen protocol and safeguard
# Validate protocol quantities against frozen constants and manifest
frozen_protocol = frozen_manifest.get("protocol", {})
```

```
# P0 #2: T_sim is stored in model section, not protocol

protocol_checks = [

("T_sim", protocol.T_sim, FROZEN_T_SIM, frozen_model.get("T_sim")),

("n_steps", protocol.n_steps(config), FROZEN_N_STEPS, frozen_protocol.get("n_steps")),

("burn_end", protocol.burn_end(config), FROZEN_BURN_END, frozen_protocol.get("burn_end")),

("hist_end", protocol.hist_end(config), FROZEN_HIST_END, frozen_protocol.get("hist_end")),

("future_steps", protocol.future_steps(config), FROZEN_FUTURE_STEPS,
frozen_protocol.get("future_steps")),

("diagnostic_stride", protocol.diagnostic_stride, 50, frozen_protocol.get("diagnostic_stride")),

("scale_epsilon", config.scale_epsilon, FROZEN_ZERO_TOL, frozen_model.get("scale_epsilon",
FROZEN_ZERO_TOL)),

]



# P0 #2: Also add manifest checks for horizon fields

# At rho=1, window_steps and recovery_end should match frozen protocol values

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

actual_window_steps = int(round(config.T_window / config.dt))

actual_recovery_end = min(5 * actual_window_steps, protocol.future_steps(config))
```

```
protocol_checks.extend([

("window_steps", actual_window_steps, FROZEN_WINDOW_STEPS,
frozen_protocol.get("window_steps")),

("recovery_end", actual_recovery_end, FROZEN_RECOVERY_END,
frozen_protocol.get("recovery_end")),

])



for name, observed, expected_frozen, expected_manifest in protocol_checks:

# Check against frozen constant first

if observed != expected_frozen:

results["status"] = "failed"

results["error"] = (

f"{name} mismatch: observed={observed}, "

f"expected (frozen)={expected_frozen}"

)

return results


# Also check against manifest if available

if expected_manifest is not None and observed != expected_manifest:
```

```
results["status"] = "failed"

results["error"] = (

f"{name} mismatch: observed={observed}, "

f"expected (manifest)={expected_manifest}"

)

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}, "errors": {}}



try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)
```

```
branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# CRITICAL: Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in engine_results:
```

```
finalize_historical_branch_metrics(engine_results)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"prefix_{key}_present_in_frozen"] = True


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True
```

```
else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"prefix_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} != frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"prefix_{key}"] = None

else:

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"prefix_{key}"] = None


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]
```

```
frozen_prefix = f"{branch}_seed_{seed_str}"


# P0 #2: Validate horizon metadata in branch outputs

# The preflight verifies config would produce correct metadata,

# but we must verify each returned engine_branch actually contains those values

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key, expected in BASELINE_HORIZON_METADATA.items():

observed = engine_branch.get(key)

passed = (

isinstance(observed, (int, np.integer))

and int(observed) == int(expected)

)

seed_results["checks"][f"{branch}_{key}_baseline_exact"] = passed

if not passed:

seed_results["status"] = "failed"

seed_results["errors"][f"{branch}_{key}_baseline_mismatch"] = (

f"observed={observed!r}, expected={expected!r}"
```

```
)

all_passed = False


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===

for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:

seed_results["status"] = "failed"
```

```
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

if engine_val.shape != frozen_val.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} !=
frozen shape {frozen_val.shape}"

seed_results["max_diffs"][f"{branch}_{key}"] = None

else:
```

```
e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}"] = None


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===

# These do NOT exist in frozen NPZ. At rho=1, both horizon systems

# should produce identical results (1750 and 8750 steps match).

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:
```

```
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# Use module-level map for consistency - direct indexing

frozen_equiv_key = EXTENDED_MATRIX_EQUIVALENCE_MAP[key]

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:

if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

if engine_val.shape != engine_frozen_equiv.shape:

seed_results["errors"][f"{branch}_{key}_shape_mismatch"] = f"engine shape {engine_val.shape} !=
equiv shape {engine_frozen_equiv.shape}"

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None

else:
```

```
e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))

else:

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = None


# === SCALAR KEYS ===

# Use module-level constants for consistency

# FROZEN_BRANCH_SCALAR_KEYS: All 42 historical fields from frozen production

# EXTENDED_BRANCH_SCALAR_KEYS: New fields in v1 for dual-horizon support


for key in FROZEN_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)


# P1 #8: Use CSV as authority, not NPZ fallback

# Get from CSV only (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"
```

```
if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]


# FROZEN scalars MUST exist in historical data

if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_frozen"] = True


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:
```

```
seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# For scalars, compare directly - DO NOT silently skip

# Normalize to scalar value

try:

engine_scalar = float(engine_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_is_scalar"] = False

seed_results["errors"][f"{branch}_{key}_type_error"] = f"Cannot convert to float:
{type(engine_val).__name__}: {e}"

all_passed = False

continue


try:

frozen_scalar = float(frozen_val)

except (TypeError, ValueError) as e:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_frozen_is_scalar"] = False
```

seed_results["errors"][f"{branch}_{key}_frozen_type_error"] = f"Cannot convert frozen to float: {type(frozen_val).__name__}: {e}"

all_passed = False

continue

Compare with exact equality (rtol=0, atol=0)

engine_is_nan = bool(np.isnan(engine_scalar))

frozen_is_nan = bool(np.isnan(frozen_scalar))

if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_scalar, frozen_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_scalar - frozen_scalar)) if not (engine_is_nan or frozen_is_nan) else None

```
# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = False

all_passed = False

continue

else:

seed_results["checks"][f"{branch}_{key}_present_in_engine"] = True


# Use module-level map for consistency - direct indexing

frozen_equiv = EXTENDED_SCALAR_EQUIVALENCE_MAP[key]

if frozen_equiv:

# Get the engine's own frozen-equivalent value

engine_frozen_equiv = engine_branch.get(frozen_equiv)
```

```
if engine_frozen_equiv is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{frozen_equiv}_present_in_engine"] = False
all_passed = False
continue

# Validate internal consistency: extended should equal frozen equivalent
try:
engine_scalar = float(engine_val)
equiv_scalar = float(engine_frozen_equiv)
except (TypeError, ValueError) as e:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_internal_consistency"] = False
seed_results["errors"][f"{branch}_{key}_internal_type_error"] = str(e)
all_passed = False
continue

engine_is_nan = bool(np.isnan(engine_scalar))
```

```
equiv_is_nan = bool(np.isnan(equiv_scalar))


if engine_is_nan and equiv_is_nan:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

elif np.isclose(engine_scalar, equiv_scalar, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_internal_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_internal_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_internal"] = float(abs(engine_scalar - equiv_scalar)) if
not (engine_is_nan or equiv_is_nan) else None


except Exception as e:

seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False
```

```
results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():
```

print(f" MAX DIFF: {key} = {val}")

return results

==========
Main API
==========
Known branches - allowlist for validation

KNOWN_BRANCHES = frozenset({

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

})

def run_seed(

```
text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.



CRITICAL: branch_order defines the FULL frozen RNG namespace (7 branches).

protocol.branches defines which branches are actually executed.



This ensures future_shuffle gets the 7th RNG sequence (not the 4th)

as in production, even when only 4 study branches are executed.



Returns complete seed results with all branches.

"""
```

```
# Preflight validation: validate config and protocol

config.validate()

protocol.validate(config)



# Nonblocking #8: Require continue branch for historical metrics

if "continue" not in protocol.branches:

raise ValueError(

"run_seed requires the continue branch "

"to compute historical cross-branch metrics"

)



# Validate all protocol branches are known

for branch in protocol.branches:

if branch not in KNOWN_BRANCHES:

raise ValueError(

f"Unknown branch '{branch}'. Known branches: {sorted(KNOWN_BRANCHES)}"

)
```

```
# Validate all protocol branches are in branch_order (for RNG namespace)

for branch in protocol.branches:

if branch not in branch_order:

raise ValueError(

f"Branch '{branch}' in protocol.branches but not in branch_order. "

f"branch_order must include all protocol branches for correct RNG namespace."

)



# Validate branch_order has no duplicates and all are known

if len(branch_order) != len(set(branch_order)):

raise ValueError(f"branch_order has duplicate branches: {branch_order}")

for branch in branch_order:

if branch not in KNOWN_BRANCHES:

raise ValueError(f"Unknown branch '{branch}' in branch_order. Known branches:
{sorted(KNOWN_BRANCHES)}")
```

```
# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm with FULL branch_order

branch_rngs = make_branch_rngs(seed, branch_order)



# Execute only the branches in protocol.branches, but use RNGs from branch_order

branch_results = {}

for branch in protocol.branches:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"This indicates branch_order={branch_order} does not match the RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]



branch_results[branch] = simulate_future(
```

```
split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)



# Compute historical cross-branch metrics (the 8 time-based fields)

# These must be computed AFTER all branches are simulated

if "continue" in branch_results:

finalize_historical_branch_metrics(branch_results)



return {

"seed": int(seed),

"config": {

"N": config.N,
```

```
"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":
```

text

```
# Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")
```

Ignoring copy/paste corruption, this is now the first engine version for which I find no remaining P0 correctness or certificate-integrity blocker.
Verdict

Core numerical fidelity: approximately 9.7/10
RNG fidelity: approximately 9.7/10
Dual-horizon implementation: approximately 9.7/10

Mandatory baseline gate: approximately 9.7/10
Decision: GO for compile, import, one-seed smoke, and mandatory baseline-only testing

This is approval to test, not evidence that frozen reproduction has passed. That remains an empirical finite-replay fact to be established by the certificate.
Previously identified issues now correctly closed

The current version now handles:

exact positive-integer N validation;
finite protocol parameters;
duplicate-free/nonempty branches;
strict BB-matrix validation tolerances;
named frozen gap tolerance;
exact frozen alpha comparison;
literal frozen persistence-ratio logic;
malformed authority-file handling;
exact frozen seed cohort;
exact full seven-branch RNG namespace;
exact seven executed baseline branches;
manifest schema uniqueness and equality;
exact frozen NPZ key coverage;
malformed and duplicate CSV rows;
exact CSV (seed, branch) coverage;
required scalar columns in every CSV row;
CSV-only scalar authority;
all 42 historical scalar comparisons;
all frozen matrix and trajectory comparisons;
complete extended matrix/scalar mappings;
import-time mapping integrity checks;
fixed/normalized $\rho=1\rho=1$ internal identities;
returned branch horizon metadata;
frozen protocol and safeguard identities;
actual manifest/NPZ/CSV file hashes;
continue requirement for historical metrics.

The earlier unreachable protocol.branches validation and incorrect T_sim manifest lookup are fixed.
No further changes recommended before the baseline

There are remaining optimization opportunities:

cache alpha;
cache delay_steps;
replace list history with the frozen deque;
mark cached arrays read-only;
remove unused imports and helpers.

I recommend not changing those now. They are not correctness issues, and any source modification changes the engine hash and requires repeating the entire certification sequence. First obtain the actual timing and exact-replay results.

If the baseline runtime proves unacceptable, optimize in a separately hashed revision and rerun the baseline gate.
Minor nonblocking observations
1. Redundant validation lines

These remain redundant:

Python

if self.N <= 0:
raise ValueError("N must be positive")

after the stronger positive-integer check, and:

Python

if self.diagnostic_stride <= 0:
...

after the stronger integer/positive check.

They are harmless.
2. Documentation rounding

The header still says:

text

T_WINDOW = ... = 17.5

The exact value is:

text

17.502917152858814...

Use “approximately 17.5” when next editing documentation, but do not alter the source solely for this immediately before certification.
3. Malformed nested manifest types

The gate validates required top-level sections and fields. Extremely malformed structures—such as "protocol" being a list rather than a dictionary—could still raise an exception rather than return a structured failure. The study driver catches baseline exceptions and writes a failure manifest, so this is not a correctness risk for the known frozen authority.
4. Frozen authority hash pinning

The gate records the actual file hashes but does not compare them against hardcoded lineage hashes. Before running, independently confirm that the three supplied artifacts are exactly the established frozen authorities under:

text

runs/5a9f093a2fe0/

Recording the hashes is provenance; comparing them to the prior freeze lineage authenticates that provenance.
Required testing order

Use the previously established work order:

Save hashes of all three source files.
Compile all three.
Import all three.

Run the engine main validation.
Run schema assertions.
Run the one-seed continue exact replay smoke test.
Run the driver dry-run.
Run the mandatory 40-seed, seven-branch --baseline-only test.
Audit the saved certificate.
Confirm source hashes are unchanged.
Only then run the full grid.

The baseline certificate must establish:

text

40/40 seeds passed
7/7 branches executed and passed
all 6 prefix arrays exact
all 11 historical matrix/trajectory fields exact
all 42 historical scalar fields exact
all extended rho=1 matrix identities exact
all extended rho=1 scalar identities exact
all branch horizon metadata exact

If any one of those fails, stop before the grid.
Final assessment

Engine static review: PASS for testing.

No broad scientific claim follows yet. A successful exact certificate will establish that this parameterized engine reproduces the saved frozen baseline on the tested finite cohort and environment.

Next file:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

from future import annotations

import argparse

import csv

```
import hashlib

import json

import os

import platform

import shutil

import sys

import time

import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

print("ERROR: pandas is required for production T_window study")

print("Install with: pip install pandas")
```

```
sys.exit(1)

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5
```

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750

Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5 # Also PHYSICAL_LATE_START

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)

P1 #7: Separate study metadata from frozen scalar output

T_window and rho are study metadata, not part of historical frozen branch scalar schema

STUDY_METADATA_KEYS = (

text

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS +
EXTENDED_BRANCH_SCALAR_KEYS
P1 #6: Removed DESIRED_BRANCH_SCALAR_KEYS as they are always NaN
These fields require finalize_historical_branch_metrics() to be called, which is not done
for individual (seed, rho) runs. The new analysis script computes richer condition-specific
recovery estimands from trajectories, so these legacy fields are not needed.
All scalar keys for CSV output (required + study metadata)

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
STUDY_METADATA_KEYS
P0 #2: Separate frozen and study schemas
FROZEN keys must exist in historical frozen data (11 keys)

FROZEN_BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS
REQUIRED keys for study output - matches engine schema (all frozen + extended keys)

REQUIRED_BRANCH_MATRIX_KEYS = tuple(

text

engine.REQUIRED_BRANCH_MATRIX_KEYS

)
P0 #2: Validate at import that frozen keys are subset of engine keys

if not
set(FROZEN_BRANCH_MATRIX_KEYS).issubset(set(REQUIRED_BRANCH_MATRIX_KEYS)):

```
text

missing = set(FROZEN_BRANCH_MATRIX_KEYS) -
set(REQUIRED_BRANCH_MATRIX_KEYS)

raise RuntimeError(

f"Frozen matrix keys not subset of engine keys: {sorted(missing)}"

)

All matrix keys (use engine's full schema)

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def is_finite_number(value: Any) -> bool:

text

"""Check if a value is a finite number.



Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.

"""

return (

isinstance(

value,

(
```

```
int,

float,

np.integer,

np.floating,

),

)

and np.isfinite(value)

)

def validate_prefix_outputs(prefix: dict[str, Any]) -> None:

text

"""P0 #5: Validate prefix outputs.



Validates that all required prefix keys are present with correct shapes.



Raises RuntimeError if required keys are missing or have wrong shapes.

"""

missing = set(REQUIRED_PREFIX_KEYS) - set(prefix)
```

```
if missing:

raise RuntimeError(f"schema: prefix missing {sorted(missing)}")



expected_shapes = {

"B_at_split": (FROZEN_N, FROZEN_N),

"R_physical_past": (FROZEN_N, FROZEN_N),

"theta_at_split": (FROZEN_N,),

"coherence_acc_at_split": (FROZEN_N, FROZEN_N),

"history_at_split": (FROZEN_DELAY_STEPS + 1, FROZEN_N),

"selected_at_split": (FROZEN_N, FROZEN_N),

}



for key, expected in expected_shapes.items():

if key in prefix:

observed = np.asarray(prefix[key])

if observed.shape != expected:

raise RuntimeError(f"schema: prefix {key} shape {observed.shape} != {expected}")
```

```
def validate_branch_outputs(
text
branch: str,
branch_result: dict[str, Any],
expected_rho: float,
expected_T_window: float,
) -> None:
text
"""Validate that a branch result contains all required outputs.


P0 #3: Now validates rho and T_window metadata.
P0 #4: Now validates recovery_end_ema.
P1 #8: Use engine's REQUIRED_BRANCH_MATRIX_KEYS for matrix validation.
Also validates array shapes and metadata consistency.


Raises RuntimeError if required keys are missing or have wrong shapes.
"""
```

```
# Validate required scalars

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())



# P1 #8: Use engine's matrix keys for validation

missing_matrices = set(engine.REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())



# P1 #7: Strengthen matrix shape validation

# Validate ALL matrix fields that should be N x N

shape_errors = []



# P1 #7: Define which matrix keys should be 12x12

MATRIX_12X12_KEYS = tuple(

key

for key in engine.REQUIRED_BRANCH_MATRIX_KEYS

if key not in {

"sample_times",

"density_trajectory",
```

```
"similarity_trajectory",

}

)



for key in MATRIX_12X12_KEYS:

if key in branch_result:

value = np.asarray(branch_result[key])

if value.shape != (FROZEN_N, FROZEN_N):

shape_errors.append(f"{key} shape {value.shape} != {(FROZEN_N, FROZEN_N)}")

if not np.all(np.isfinite(value)):

shape_errors.append(f"{key} contains non-finite values")



# Validate trajectory lengths

expected_samples = (FROZEN_FUTURE_STEPS + 50 - 1) // 50 # diagnostic_stride=50

for key in ("sample_times", "density_trajectory", "similarity_trajectory"):

if key in branch_result:

actual_len = len(branch_result[key])
```

```
if actual_len != expected_samples:

shape_errors.append(f"{key} length {actual_len} != {expected_samples}")



# P1 #8: Validate all trajectory arrays have equal lengths

if all(key in branch_result for key in ("sample_times", "density_trajectory", "similarity_trajectory")):

lengths = [len(branch_result[key]) for key in ("sample_times", "density_trajectory",
"similarity_trajectory")]

if len(set(lengths)) &gt; 1:

shape_errors.append(f"Trajectory length mismatch: {lengths}")



# P1 #8: Validate exact sample-time schedule

if "sample_times" in branch_result:

expected_times = (

np.arange(0, FROZEN_FUTURE_STEPS, 50, dtype=float) + 1.0

) * FROZEN_DT

observed_times = np.asarray(branch_result["sample_times"], dtype=float)

if not np.array_equal(observed_times, expected_times):

shape_errors.append(
```

```
"sample_times does not match the expected "

"0.01-to-319.51 diagnostic schedule"

)



# Validate horizon metadata consistency

# P0 #3: Validate rho and T_window with expected values

# P0 #4: Validate recovery_end_ema

# P0 #2 FIX: Use expected_rho parameter, not undefined rho variable

metadata_errors = []

if "rho" in branch_result:

# P0 #3: Validate rho matches expected value

observed_rho = branch_result["rho"]

if float(observed_rho) != float(expected_rho):

metadata_errors.append(f"rho={observed_rho} != {expected_rho}")

else:

metadata_errors.append("missing rho")
```

```
if "T_window" in branch_result:

# P0 #3: Validate T_window matches expected value

observed_T_window = branch_result["T_window"]

if float(observed_T_window) != float(expected_T_window):

metadata_errors.append(f"T_window={observed_T_window} != {expected_T_window}")

else:

metadata_errors.append("missing T_window")



# P1 #8: Use independent expected values, not self-referential from branch_result

# Validate window_steps using expected_T_window, not branch_result["T_window"]

if "window_steps" in branch_result:

expected_window_steps = int(round(expected_T_window / FROZEN_DT))

if branch_result["window_steps"] != expected_window_steps:

metadata_errors.append(f"window_steps {branch_result['window_steps']} !=
{expected_window_steps}")

else:

metadata_errors.append("missing window_steps")
```

```
if "fixed_immediate_end" in branch_result:

if branch_result["fixed_immediate_end"] != 1750:

metadata_errors.append(f"fixed_immediate_end {branch_result['fixed_immediate_end']} != 1750")

else:

metadata_errors.append("missing fixed_immediate_end")



if "fixed_recovery_end" in branch_result:

if branch_result["fixed_recovery_end"] != 8750:

metadata_errors.append(f"fixed_recovery_end {branch_result['fixed_recovery_end']} != 8750")

else:

metadata_errors.append("missing fixed_recovery_end")



# P0 #4: Validate recovery_end_ema using expected_T_window

if "recovery_end_ema" in branch_result:

expected_recovery_end = min(

5 * int(round(expected_T_window / FROZEN_DT)),
```

```
FROZEN_FUTURE_STEPS,

)

if branch_result["recovery_end_ema"] != expected_recovery_end:

metadata_errors.append(f"recovery_end_ema={branch_result['recovery_end_ema']} !=
{expected_recovery_end}")

else:

metadata_errors.append("missing recovery_end_ema")



# Combine all errors

all_errors = []

if missing_scalars:

all_errors.extend([f"missing scalar: {k}" for k in sorted(missing_scalars)[:10]])

if missing_matrices:

all_errors.extend([f"missing matrix: {k}" for k in sorted(missing_matrices)[:10]])

if shape_errors:

all_errors.extend(shape_errors[:10])

if metadata_errors:

all_errors.extend(metadata_errors[:10])
```

if all_errors:

raise RuntimeError(

f"Branch '{branch}' has {len(all_errors)} validation errors: "

f"{all_errors[:5]}... ({len(all_errors)} total)"

)

==========
Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

```
for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file.



Local helper for file hashing - does not depend on engine utilities.

"""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def source_sha256() -> str | None:
```

```
text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary), str(path))
```

```
finally:

if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory
```

```
fd, temporary = tempfile.mkstemp(
dir=str(path.parent),
prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz",
)
try:
with os.fdopen(fd, "wb") as handle:
np.savez_compressed(handle, **payload)
handle.flush()
os.fsync(handle.fileno())
os.replace(temporary, str(path))
finally:
if os.path.exists(temporary):
os.unlink(temporary)
def load_npz(path: str | Path) -> dict[str, Any]:
text
"""Load NPZ file."""
```

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{path.name}.",

suffix=".tmp",

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

dataframe.to_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

```
================================================================================
Configuration and Validation
================================================================================
```

def validate_engine_contract() -> None:

text

"""P1 #12: Validate that driver constants agree with engine.

Ensures driver and engine have consistent frozen production values.

"""

checks = {

"N": (FROZEN_N, engine.FROZEN_N),

"dt": (FROZEN_DT, engine.FROZEN_DT),

"tau": (FROZEN_TAU, engine.FROZEN_TAU),

"T_star": (FROZEN_T_STAR, engine.FROZEN_T_STAR),

"B_c": (FROZEN_B_C, engine.FROZEN_B_C),

"B_init_scale": (FROZEN_B_INIT_SCALE, engine.FROZEN_B_INIT_SCALE),

}

for name, (driver_value, engine_value) in checks.items():

if driver_value != engine_value:

raise RuntimeError(

f"Driver/engine {name} mismatch: "

f"driver={driver_value}, engine={engine_value}"

)

```
if not np.array_equal(FROZEN_OMEGA, engine.FROZEN_OMEGA):

raise RuntimeError(

f"Driver/engine omega mismatch: "

f"driver={FROZEN_OMEGA.tolist()}, engine={engine.FROZEN_OMEGA.tolist()}"

)



# Also check other important constants

if FROZEN_T_SIM != engine.FROZEN_T_SIM:

raise RuntimeError(f"Driver/engine T_SIM mismatch: {FROZEN_T_SIM} !=
{engine.FROZEN_T_SIM}")

if FROZEN_DELTA != engine.FROZEN_DELTA:

raise RuntimeError(f"Driver/engine DELTA mismatch: {FROZEN_DELTA} !=
{engine.FROZEN_DELTA}")

if FROZEN_ETA != engine.FROZEN_ETA:

raise RuntimeError(f"Driver/engine ETA mismatch: {FROZEN_ETA} != {engine.FROZEN_ETA}")

if FROZEN_GAMMA != engine.FROZEN_GAMMA:
```

```
raise RuntimeError(f"Driver/engine GAMMA mismatch: {FROZEN_GAMMA} !=
{engine.FROZEN_GAMMA}")

if FROZEN_MU != engine.FROZEN_MU:

raise RuntimeError(f"Driver/engine MU mismatch: {FROZEN_MU} != {engine.FROZEN_MU}")

if FROZEN_A != engine.FROZEN_A:

raise RuntimeError(f"Driver/engine A mismatch: {FROZEN_A} != {engine.FROZEN_A}")

if FROZEN_Q != engine.FROZEN_Q:

raise RuntimeError(f"Driver/engine Q mismatch: {FROZEN_Q} != {engine.FROZEN_Q}")



# P1 #10: Strengthen driver/engine contract with additional checks

extra_checks = {

"scale_epsilon": (1e-12, engine.FROZEN_ZERO_TOL),

"delay_steps": (FROZEN_DELAY_STEPS, engine.FROZEN_DELAY_STEPS),

"window_steps": (FROZEN_WINDOW_STEPS, engine.FROZEN_WINDOW_STEPS),

"recovery_end": (FROZEN_RECOVERY_END, engine.FROZEN_RECOVERY_END),

}

for name, (driver_value, engine_value) in extra_checks.items():

if driver_value != engine_value:
```

```
raise RuntimeError(
f"Driver/engine {name} mismatch: driver={driver_value}, engine={engine_value}"
)


# P1 #10: Validate branch RNG namespace matches
if tuple(engine.DEFAULT_PROTOCOL_CONFIG.branches) != FROZEN_BRANCH_ORDER:
raise RuntimeError(
"Driver/engine frozen branch RNG namespace mismatch"
)
def create_model_config(rho: float) -> engine.ModelConfig:
text
"""Create ModelConfig for a given rho value."""
T_window = rho * FROZEN_T_STAR
return engine.ModelConfig(
N=FROZEN_N,
dt=FROZEN_DT,
tau=FROZEN_TAU,
```

```
omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800
```

```
split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

# ============================================================================
# Simulation Functions
# ============================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text
```

"""Simulate a single (seed, rho) combination.

CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates
study_branches subset. This ensures future_shuffle gets the 7th RNG sequence
(not the 4th) as in production.

Returns complete results for all branches at this (seed, rho).
"""
Create branch RNGs using FULL frozen branch order for correct namespace
branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)

Create a protocol with just the study branches
study_protocol = engine.ProtocolConfig(
T_sim=protocol.T_sim,
burn_end_time=protocol.burn_end_time,
split_time=protocol.split_time,
future_duration=protocol.future_duration,

```
diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# P1 #6: Validate study protocol inside simulate_seed_rho()

try:

config.validate()

study_protocol.validate(config)

except Exception as e:

raise RuntimeError(f"Protocol validation failed: {e}")



# P1 #6: Require study_branches to match STUDY_BRANCHES

if tuple(study_branches) != STUDY_BRANCHES:

raise RuntimeError(

f"Unexpected study branch set/order: {study_branches}"

)
```

```
# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# P0 #5: Validate prefix outputs

validate_prefix_outputs(prefix)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_result = engine.simulate_future(

split_state.clone(),
```

```
prefix,

branch,

config,

study_protocol,

rng,

)


# P0 #1: Add metadata BEFORE validation

branch_result["T_window"] = config.T_window

branch_result["rho"] = rho


# P0 #3-4: Validate branch outputs with expected metadata

validate_branch_outputs(branch, branch_result, rho, config.T_window)


# P1 #9: Validate branch outputs immediately after simulation

# Check both primary and secondary outcomes for finiteness

for key in PRIMARY_OUTCOMES + SECONDARY_OUTCOMES:

if key not in branch_result:

raise RuntimeError(
```

```
f"schema: {branch} missing {key}"

)

if not is_finite_number(branch_result[key]):

raise RuntimeError(

f"schema: {branch} has non-finite {key}={branch_result[key]}"

)


branch_results[branch] = branch_result



return {

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,
```

```
"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

def run_baseline_equivalence(

text

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,
```

frozen_csv_path: str,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

```
# Load frozen manifest to compare seeds
try:
with open(frozen_manifest_path, 'r', encoding='utf-8') as f:
frozen_manifest = json.load(f)
except FileNotFoundError:
return {
"status": "failed",
"error": f"Frozen manifest not found: {frozen_manifest_path}",
"seeds_match": False,
}
except json.JSONDecodeError as e:
return {
"status": "failed",
"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",
"seeds_match": False,
}
```

```
# Validate manifest has required structure
if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:
return {
"status": "failed",
"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",
"seeds_match": False,
}

frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]

# Verify seed list matches frozen manifest EXACTLY
# This is critical: baseline equivalence tests a specific cohort
if len(seeds) != len(frozen_seeds):
return {
"status": "failed",
"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",
"frozen_seeds": [int(s) for s in frozen_seeds],
```

```
"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,
```

```
# but the seed generation method should match



# Validate frozen CSV path exists

if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,

}



# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,
```

```
burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,

diagnostic_stride=50,

branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification

)



# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches

results = [engine.run](http://engine.run)_baseline_equivalence(

seeds=seeds,

config=config,

protocol=baseline_protocol,

frozen_manifest_path=frozen_manifest_path,

frozen_npz_path=frozen_npz_path,

frozen_csv_path=frozen_csv_path,

branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace

)
```

```
# Add seed comparison info

results["seed_comparison"] = {

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}



return results

================================================================================
Manifest and Output
================================================================================

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],
```

) -> dict[str, Any]:

text

"""Build study manifest."""

Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

```
# P1 #9: Add scale_epsilon to manifest
"scale_epsilon": 1e-12,
}

protocol_params = {
"T_sim": FROZEN_T_SIM,
"burn_end_time": 160.0,
"split_time": 480.0,
"future_duration": 320.0,
"diagnostic_stride": 50,
"branches": list(STUDY_BRANCHES),
}

study_params = {
"rho_values": list(rho_values),
"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],
"n_seeds": len(seeds),
```

```
"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes

dependencies = {}



# Engine source hash

try:

engine_path = Path(engine.__file__).resolve()

engine_hash = file_sha256(engine_path)

dependencies["aief_experiment_engine_v1"] = {

"path": str(engine_path),

"sha256": engine_hash,

}

except Exception as e:

dependencies["aief_experiment_engine_v1"] = {

"path": "NOT_FOUND",
```

```
"sha256": "NOT_AVAILABLE",

"error": str(e),

}



# Driver source hash (this file)

try:

driver_path = Path(__file__).resolve()

driver_hash = file_sha256(driver_path)

dependencies["AIEF_1_4_Twindow_sensitivity"] = {

"path": str(driver_path),

"sha256": driver_hash,

}

except Exception as e:

dependencies["AIEF_1_4_Twindow_sensitivity"] = {

"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",

"error": str(e),
```

```
}



manifest = {

"study_name": STUDY_NAME,

"workflow_version": WORKFLOW_VERSION,

"source_sha256": source_sha256(),

"dependencies": dependencies,

"model": model_params,

"protocol": protocol_params,

"study": study_params,

"randomization": {

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

# P1 #14: Record the full RNG namespace explicitly
```

"branch_rng_namespace": list(FROZEN_BRANCH_ORDER),

"executed_branches": list(STUDY_BRANCHES),

},

P1 #9: Add pandas to environment

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"pandas": pd.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

```
return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value
```

```
# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],
```

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.

P1 #3: Removed duplicate validation call.

Validation is already performed in simulate_seed_rho() immediately after simulation.

This function only extracts.

"""

P1 #3: Removed duplicate validation - already validated in simulate_seed_rho()

The branch_result already has T_window and rho added before validation

row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

```
}



for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

# ============================================================================
# Main Execution
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,
```

```
)
parser.add_argument(
"--master-seed",
type=int,
default=DEFAULT_MASTER_SEED,
help="fixed integer master seed (default: 20260714). For production, must match frozen data.",
)
parser.add_argument(
"--n-seeds",
type=int,
default=DEFAULT_N_SEEDS,
help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
)
parser.add_argument(
"--run-root",
default=DEFAULT_RUN_ROOT,
help=f"output root directory (default: {DEFAULT_RUN_ROOT})",
```

```
)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "
```

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--overwrite",

action="store_true",

help="overwrite existing run directory. Use with caution - will delete existing data.",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

parser.add_argument(

```
"--baseline-only",

action="store_true",

help=(

"run and save the mandatory frozen-baseline "

"certificate, then exit without launching the grid"

),

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest
```

```
manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# P1 #12: Validate driver/engine contract

try:

validate_engine_contract()

except RuntimeError as e:

print(f"\nFATAL: Driver/engine contract violation: {e}")
```

```
sys.exit(1)



# Check dry-run mode FIRST to avoid contaminating production directory

# P0 #1: Dry-run should not require frozen paths

if args.dry_run:

# Validate driver/engine contract (already validated above, but keep for clarity)

try:

validate_engine_contract()

except RuntimeError as e:

print(f"\nDry run: Driver/engine contract violation: {e}")

sys.exit(1)


# P1 #11: Dry-run should actually validate configuration

# Validate all levels

protocol_config = create_protocol_config()

try:

# Validate base config (rho=1.0)

base_config = create_model_config(1.0)
```

```
base_config.validate()

protocol_config.validate(base_config)


# Validate all rho configs

for rho in RHO_VALUES:

config = create_model_config(rho)

config.validate()

protocol_config.validate(config)


# Validate study design

if not set(STUDY_BRANCHES) &amp;lt;= set(FROZEN_BRANCH_ORDER):

raise ValueError(f"STUDY_BRANCHES {STUDY_BRANCHES} not subset of
FROZEN_BRANCH_ORDER {FROZEN_BRANCH_ORDER}")

if len(STUDY_BRANCHES) != len(set(STUDY_BRANCHES)):

raise ValueError(f"STUDY_BRANCHES has duplicates: {STUDY_BRANCHES}")

if len(RHO_VALUES) != len(set(RHO_VALUES)):

raise ValueError(f"RHO_VALUES has duplicates: {RHO_VALUES}")

if 1.0 not in RHO_VALUES:
```

```
raise ValueError("RHO_VALUES must include 1.0 for baseline equivalence")


print("Dry run: configuration validated successfully.")
print(f" Validated {len(RHO_VALUES)} rho values: {RHO_VALUES}")
print(f" Validated {len(STUDY_BRANCHES)} study branches: {STUDY_BRANCHES}")
print(f" Validated {args.n_seeds} seeds from master seed {args.master_seed}")
print(f"Configuration hash: {config_hash}")
print("To execute, run without --dry-run")
print(f"Expected run directory: {Path([args.run](http://args.run)_root) / config_hash}")
except Exception as e:
print(f"Dry run: configuration validation FAILED: {e}")
sys.exit(1)
return


# P1 #10: Validate mandatory input paths before creating/deleting run directory
# This prevents contaminating a production root after a simple path error
for path_name, path_value in [
("frozen-manifest", args.frozen_manifest),
```

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Create run directory

run_dir = Path(args.run_root) / config_hash

Check if run directory already exists

if run_dir.exists():

if args.overwrite:

Overwrite mode: remove existing directory

print(f"Overwriting existing run directory: {run_dir}")

try:

shutil.rmtree(run_dir)

except Exception as e:

print(f"Cannot overwrite: error removing directory: {e}")

sys.exit(1)

run_dir.mkdir(parents=True, exist_ok=True)

else:

Default: fail if directory exists

```
print(f"Run directory {run_dir} already exists")

print("Use --overwrite to replace")

sys.exit(1)

else:

# New run directory

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")
```

```
print(f" output: {run_dir}")

print()



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),
```

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Run baseline equivalence - MANDATORY for all seeds

```
try:
baseline_results = run_baseline_equivalence(
seeds=seeds,
frozen_manifest_path=args.frozen_manifest,
frozen_npz_path=args.frozen_npz,
frozen_csv_path=args.frozen_scalar,
)
except Exception as exc:
# Handle baseline equivalence exception - save failure manifest
failure_manifest = {
**manifest,
"config_hash": config_hash,
"status": "FAILED",
"failure_reason": "baseline_equivalence_exception",
"exception_type": type(exc).__name__,
"exception_message": str(exc),
"freezeable": False,
}
```

```
failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from
AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")
```

```
print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)
```

```
# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Save per-seed baseline results separately for easier audit
```

```
if "seed_results" in baseline_results:

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_seed_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_seed_path}")


# P0 #3: Update manifest with baseline equivalence certificate

# Record all frozen input hashes using CORRECT engine key names

# Engine returns: frozen_manifest_file_sha256, frozen_npz_file_sha256,
frozen_scalar_csv_file_sha256

# Engine also returns: frozen_manifest, frozen_npz, frozen_manifest_hash


frozen_input_hashes = {}


# Get frozen manifest file hash (engine key: frozen_manifest_file_sha256)

if "frozen_manifest_file_sha256" in baseline_results:

frozen_input_hashes["manifest_file_sha256"] = baseline_results["frozen_manifest_file_sha256"]

if "frozen_manifest" in baseline_results:

frozen_input_hashes["manifest_path"] = baseline_results["frozen_manifest"]
```

```
# Get frozen NPZ file hash (engine key: frozen_npz_file_sha256)
if "frozen_npz_file_sha256" in baseline_results:
frozen_input_hashes["npz_file_sha256"] = baseline_results["frozen_npz_file_sha256"]
if "frozen_npz" in baseline_results:
frozen_input_hashes["npz_path"] = baseline_results["frozen_npz"]

# Get frozen scalar CSV file hash (engine key: frozen_scalar_csv_file_sha256)
if "frozen_scalar_csv_file_sha256" in baseline_results:
frozen_input_hashes["scalar_csv_file_sha256"] = baseline_results["frozen_scalar_csv_file_sha256"]

# Get canonical JSON hash of frozen manifest (engine key: frozen_manifest_hash)
if "frozen_manifest_hash" in baseline_results:
frozen_input_hashes["manifest_canonical_json_sha256"] = baseline_results["frozen_manifest_hash"]

# Read frozen production source hash from frozen manifest itself
# This is the proper way - don't search for production source files
try:
with open(args.frozen_manifest, 'r', encoding='utf-8') as f:
frozen_manifest_data = json.load(f)
```

```
if "source_sha256" in frozen_manifest_data:
frozen_input_hashes["frozen_production_source"] = frozen_manifest_data["source_sha256"]
except Exception:
pass

# Also include scalar CSV path for completeness
if args.frozen_scalar:
frozen_input_hashes["scalar_csv_path"] = str(Path(args.frozen_scalar).resolve())

manifest["baseline_equivalence"] = {
"status": "passed",
"n_seeds_tested": baseline_results["n_seeds"],
"n_seeds_passed": baseline_results["n_seeds"],
"engine_source_sha256": baseline_results.get("engine_source_sha256"),
"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),
"frozen_npz_hash": baseline_results.get("frozen_npz_file_sha256"),
"frozen_csv_hash": baseline_results.get("frozen_scalar_csv_file_sha256", "N/A"),
"frozen_production_source_hash": frozen_input_hashes.get("frozen_production_source", "N/A"),
```

```
"frozen_input_hashes": frozen_input_hashes,

"certificate_path": str(baseline_cert_path),

"seed_results_path": str(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches

}


# Re-save manifest with baseline certificate

atomic_write_json(manifest_path, manifest)


# P0 #2: Handle --baseline-only mode

if args.baseline_only:

manifest["execution_mode"] = "baseline_only"

manifest["baseline_certified"] = True

manifest["freezeable"] = False

manifest["completion"] = {

"status": "baseline_only_completed",

"study_grid_started": False,

"baseline_passed": True,
```

```
}

manifest["output_hashes"] = {

"baseline_certificate": file_sha256(baseline_cert_path),

"baseline_seed_results": file_sha256(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

}

atomic_write_json(manifest_path, manifest)

print("\nBaseline-only run completed successfully.")

print("The T_window grid was not started.")

return



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")
```

all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {} # Full payload in memory; checkpoints also written per-condition

Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0

started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)

Fatal error flag for breaking outer loop on schema/config mismatch

fatal_error = False

for seed_index, seed in enumerate(seeds):

Check for fatal error from previous seed

if fatal_error:

```
print(f" Skipping seed {seed} due to fatal schema/config error")

break


for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition (saved for diagnostic recovery, not resume)

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


try:

# Validate config before simulation

config.validate()

protocol.validate(config)


# Run simulation

result = simulate_seed_rho(
```

```
seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)
```

```
# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)


# P1 #5: NPZ key-collision detection

overlap = set(all_matrix_payload) &amp;amp; set(matrix_payload)

if overlap:

raise RuntimeError(

f"schema: NPZ key collision: {sorted(overlap)[:10]}"

)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",
```

```
"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1


except Exception as e:

error_str = str(e)

condition_status.append({

"seed": seed,

"rho": rho,

"status": "failed",

"error": error_str,

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Stop immediately on schema/config mismatch - use fatal flag to break outer loop

if "schema" in error_str.lower() or "config" in error_str.lower() or "missing" in error_str.lower():
```

```
print(f"\n FATAL: Schema/config mismatch detected - stopping immediately")

print(f" This error will affect all remaining conditions")

# Set fatal error flag to break outer loop after inner loop

fatal_error = True

break



# Status

elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")




# Save condition status table

# P1 #16: pandas is mandatory (script exits at import without it), so remove dead fallback

if condition_status:

# pandas is guaranteed to be available due to import-time check
```

```
status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")


# Record condition status in manifest

manifest["condition_status"] = {

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,

"successful_rows": successful_conditions,

"failed_rows": failed_conditions,

}


# Also record pandas version in environment

manifest["environment"]["pandas"] = pd.__version__


print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")
```

```
# If any conditions failed, mark as non-freezeable

if failed_conditions &gt; 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")



# Validate primary outcomes are present and finite

missing_primary = []
```

```
for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



# P1 #5: Validate secondary outcomes are present and finite

missing_secondary = []

for row in all_scalar_rows:

for key in SECONDARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_secondary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")
```

```
print("Run is INCOMPLETE and NOT freezeable")



if missing_secondary:

print(f"\nERROR: {len(missing_secondary)} secondary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_secondary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))



# Get all fieldnames
```

```
fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:
```

```
save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

# P1 #5: Require both primary and secondary outcomes to be complete

complete = (

failed_conditions == 0

and successful_conditions == total_combinations

and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

and len(missing_secondary) == 0

)



# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"

manifest["freezeable"] = complete and baseline_passed



# Final manifest update with completion status
```

```
manifest["completion"] = {

"status": "completed" if complete else "incomplete",

"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"n_missing_secondary_outcomes": len(missing_secondary),

"elapsed_seconds": time.time() - started,

"freezeable": manifest["freezeable"],

}



# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}



# Main outputs
```

```
if csv_path.exists():

output_hashes["scalar_csv"] = file_sha256(csv_path)

if npz_path.exists():

output_hashes["branch_npz"] = file_sha256(npz_path)



# Condition status

condition_status_path = run_dir / "twindow_condition_status.csv"

if condition_status_path.exists():

output_hashes["condition_status_csv"] = file_sha256(condition_status_path)



# Baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

if baseline_cert_path.exists():

output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)



# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"
```

```
if baseline_seed_path.exists():
output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)


if output_hashes:
manifest["output_hashes"] = output_hashes


# P1 #15: Condition checkpoints are diagnostic, non-authoritative artifacts
# The consolidated NPZ is the authority
manifest["checkpoints"] = {
"role": "diagnostic, non-authoritative",
"count": successful_conditions,
"authoritative_consolidation": str(npz_path),
}


# Re-save final manifest
atomic_write_json(manifest_path, manifest)
```

```
# Print completion status

if complete and baseline_passed:

print(f"\nRun COMPLETED successfully and is FREEZEABLE")

elif complete and not baseline_passed:

print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")

# P0 #4: Exit nonzero for non-freezeable runs

sys.exit(1)

else:

print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions &gt; 0:

print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")

if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")

# P0 #4: Exit nonzero for non-freezeable runs

sys.exit(1)
```

```
# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

# P1 #17: Print conditionally - only if files exist

if csv_path.exists():

print(f"Saved {csv_path}")

if npz_path.exists():

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)
```

```
# Use the missing_primary count already computed earlier

if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]
```

```
if not available_keys:
print(f" rho={rho:.2f}, branch={branch:&amp;lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")
continue

stats = {}
for key in available_keys:
values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]
if values:
# Use sample SD (ddof=1) for descriptive statistics
stats[key] = f"{np.mean(values):.4f}±{np.std(values, ddof=1):.4f}"
else:
stats[key] = "N/A"
print(f" rho={rho:.2f}, branch={branch:&amp;lt;15} | " +
" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))
if name == "main":
text
main()
```

Ignoring copy/paste corruption, the two deterministic full-grid bugs from the prior version are fixed:

metadata is added before validation;
expected_rho is used instead of an undefined rho.

I do not see a remaining P0 correctness blocker.
Verdict

Baseline-only path: ready
Full-regeneration logic: correct
RNG namespace: correct
Condition schema validation: strong
Full-grid path: ready for staged testing
Decision: GO for compile/import/dry-run/one-condition smoke, then baseline-only

This is static approval to test, not evidence that the run succeeds.
Important corrections now present

The driver now correctly implements:

full regeneration for every (seed,ρ)(seed,ρ);
full seven-branch RNG namespace;
four executed study branches;
study-protocol validation;
exact study branch order;
prefix key and shape validation;
metadata insertion before validation;
independent expected rho and T_window checks;
independent window_steps and recovery_end_ema checks;
fixed physical horizon metadata checks;
complete engine matrix schema;
all matrix shapes and finiteness;
exact 640-sample trajectory length;
equal trajectory lengths;
exact 0.01–319.51 sample-time schedule;
primary and secondary finite checks;
NPZ key-collision detection;
engine/driver constant checks;

frozen branch namespace check;
scale safeguard in the manifest;
pandas version in the environment;
baseline-only execution;
nonzero exit for incomplete runs;
atomic checkpoints and consolidated outputs;
complete final output hashes.

The manifest matrix schema now agrees with what the driver validates and writes.
No blocking changes required

I recommend avoiding further architectural or performance edits before certification. The following are worthwhile but nonblocking.
1. Make schema failures trigger the fatal stop immediately

validate_branch_outputs() currently raises:

Python

raise RuntimeError(
f"Branch '{branch}' has ..."
)

The condition loop stops the entire grid only when the error text contains:

text

schema
config
missing

A matrix shape or metadata mismatch may not contain those words, so the driver could continue attempting many conditions after discovering a global schema incompatibility.

Change the validator’s final error to:

Python

raise RuntimeError(
f"schema: Branch '{branch}' has "
f"{len(all_errors)} validation errors: "
f"{all_errors[:5]}... "
f"({len(all_errors)} total)"
)

Alternatively, classify exceptions by type rather than matching message text.

This does not affect successful conditions, but it prevents wasting the grid after a global contract failure.
2. Validate trajectory values, not only shape and time

The analysis script can perform this validation later, but catching corruption at generation time is preferable.

Density should be finite and in [0,1][0,1]:

Python

if "density_trajectory" in branch_result:
density = np.asarray(
branch_result["density_trajectory"],
dtype=float,
)

if not np.all(np.isfinite(density)):
shape_errors.append(
"density_trajectory contains non-finite values"
)

if np.any((density < 0.0) | (density > 1.0)):
shape_errors.append(
"density_trajectory lies outside [0,1]"
)

Similarity may contain NaNs after a zero-BB reset, but finite values should lie in the cosine range:

Python

```
if "similarity_trajectory" in branch_result:
similarity = np.asarray(
branch_result["similarity_trajectory"],
dtype=float,
)
finite = similarity[np.isfinite(similarity)]


if np.any(
(finite < -1.0 - 1e-12)
| (finite > 1.0 + 1e-12)
):
shape_errors.append(
"finite similarity values lie outside [-1,1]"
)
```

This is recommended before the full grid but is not needed to authorize the baseline-only run.
3. Strengthen prefix validation slightly

Prefix shapes are checked. Optional additions:

Python

```
for key in (
"B_at_split",
"R_physical_past",
"theta_at_split",
"coherence_acc_at_split",
"history_at_split",
):
if not np.all(np.isfinite(prefix[key])):
raise RuntimeError(
f"schema: prefix {key} contains non-finite values"
)
```

Also require selected_at_split to contain only 0 and 1.

The engine’s own state checks already make an actual failure unlikely.

Small cleanup

These do not affect the scientific run:

validate_engine_contract() is called twice during dry-run.
Frozen input paths are validated twice during normal execution.
LEGACY_OUTCOMES and load_npz() appear unused.
is_finite_number() accepts booleans.
Incomplete-run reporting does not print the secondary-outcome count.
Condition failure rows could include exception_type.
all_errors reports the length of truncated error lists rather than the true total.
The configuration hash remains machine/path/environment dependent.
The top documentation should state:

text

T_* = 17.502917152858814... ≈ 17.5

None of these changes the estimands or trajectory generation.
One provenance caveat

The configuration hash is built from the complete manifest, including:

absolute engine and driver paths;
Python/NumPy/pandas versions;
platform.

Therefore, treat it as a source/environment/run identity, not a portable model-configuration identity. This is acceptable for the imminent single-system run as long as it is documented and the source files remain unchanged.
Testing recommendation

Because the baseline-only path does not exercise simulate_seed_rho(), add a direct one-condition smoke after dry-run and before the full grid. At minimum test:

text

one frozen seed

rho = 1.0
all four study branches
prefix validation
branch schema validation
exact trajectory schedule
metadata validation
checkpoint serialization

Then proceed:

compile all three files;
import all three;
driver dry-run;
one-seed engine frozen replay;
one-condition study-driver smoke;
mandatory 40-seed seven-branch baseline-only run;
audit the certificate;
only then launch the 200-condition grid.

Final assessment

Driver static review: PASS for staged testing.

Do not call the study successful until the full run reports:

text

200/200 successful conditions
800/800 scalar rows
0 missing primary outcomes
0 missing secondary outcomes
freezeable = true
all final output hashes present

The repeated baseline and grid runs use the same seed cohort and are reproducibility/parameter-study stages, not independent experiments.

Now the final file:

```
#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

from future import annotations
```

```
import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
```

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Recovery DID expected count: 40 seeds 4 non-baseline rho 3 non-continue branches 2 systems 2 metrics * 13 outcomes = 24960

EXPECTED_N_RECOVERY_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1) len(NON_CONTINUE_BRANCHES) 2 2 * 13
Recovery groups: 5 rho 4 branches 2 systems * 2 metrics = 80 groups

EXPECTED_N_RECOVERY_GROUPS = len(EXPECTED_RHO_VALUES)
len(EXPECTED_BRANCHES) 2 * 2
Recovery DID groups: 4 rho 3 branches 2 systems 2 metrics 13 outcomes = 624 groups

EXPECTED_N_RECOVERY_DID_GROUPS = (len(EXPECTED_RHO_VALUES) - 1)
len(NON_CONTINUE_BRANCHES) 2 2 13
Censoring-aware fields that must have 40 finite values per group

RESTRICTED_RECOVERY_FIELDS = (

text

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_observed",

"sustained_observed",

"held_observed",

)
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
Expected sample time schedule for trajectory validation
640 samples from 0.01 to 319.51 with step 0.5

EXPECTED_SAMPLE_TIMES = (np.arange(0, 32000, 50, dtype=float) + 1.0) * 0.01

EXPECTED_N_SAMPLES = len(EXPECTED_SAMPLE_TIMES)
==========
Utility Functions
==========

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)

Use pandas with explicit dtypes

```
df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df

def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory.
```

Raises RuntimeError on corrupt NPZ or load failure.

"""

path = Path(path)

payload: dict[str, Any] = {}

if not path.exists():

raise FileNotFoundError(f"NPZ file not found: {path}")

try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

raise RuntimeError(f"Could not load NPZ {path}: {e}") from e

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

Write CSV with explicit flush and fsync for data integrity

with open(temp_path, "w", encoding="utf-8", newline="") as handle:

df.to_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

```
temp_path.replace(path)

# fsync the directory to ensure the replace is visible

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
```

```
try:
with open(temp_path, "w", encoding="utf-8") as f:
f.write(text)
f.flush()
os.fsync(f.fileno())
temp_path.replace(path)
# fsync the directory
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:
text
```

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

Convert NaN and infinity to None for JSON serialization

Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -> Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

```
elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None

return val

elif isinstance(obj, np.ndarray):

# Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()
```

```
os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl

digest = hl.sha256()

path = Path(path)

if not path.exists():
```

return "FILE_NOT_FOUND"

with open(path, "rb") as f:

for block in iter(lambda: f.read(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

```
================================================================================
Metric Functions
================================================================================
```

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.

Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

```
# Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))



crossing_indices = np.flatnonzero(above_threshold)



if len(crossing_indices) &amp;gt; 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.



Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)
```

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

if len(times) < 2:

return float('nan'), False

Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step <= 0:

return float('nan'), False

Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False

for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break

return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.

Handles NaN values by treating them as zero for the deficit calculation.

Uses np.trapezoid if available, falls back to np.trapz for older NumPy versions.

NaN cosine values (from zero-vector weights) are imputed as zero for recovery

and AUC calculations only - this is a prespecified recovery-only convention.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0 (prespecified recovery-only convention for undefined zero-vector cosine)

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration - use np.trapezoid if available, else np.trapz

if hasattr(np, "trapezoid"):

return float(np.trapezoid(deficit, x=times))

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def validate_topology_matrix(matrix: np.ndarray, key: str) -> np.ndarray:

text

"""Validate a topology (B_final) matrix for production analysis.

Checks:

- Shape is exactly (12, 12)

- All values are finite (no NaN, inf, -inf)

- Matrix is symmetric (rtol=0, atol=1e-12)

- Diagonal is exactly zero (rtol=0, atol=1e-12)

- No negative weights (all values >= -1e-12)

Raises RuntimeError if any check fails.

"""

matrix = np.asarray(matrix, dtype=float)

if matrix.shape != (12, 12):

raise RuntimeError(

f"{key} shape {matrix.shape} != (12, 12)"

)

if not np.all(np.isfinite(matrix)):

raise RuntimeError(

f"{key} contains non-finite values"

)

if not np.allclose(matrix, matrix.T, rtol=0.0, atol=1e-12):

raise RuntimeError(f"{key} is not symmetric")

if not np.allclose(np.diag(matrix), 0.0, rtol=0.0, atol=1e-12):

raise RuntimeError(

f"{key} diagonal is not zero"

)

if np.any(matrix < -1e-12):

raise RuntimeError(

f"{key} contains negative weights"

)

return matrix

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: str | Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Convert to Path for consistency

npz_path = Path(npz_path)

```
# Get unique seeds, rho values, and branches from the data

seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())



# Build complete list of required keys for all conditions

# We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",
```

```
])



# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []
```

```
for seed in seeds:

for rho in rho_values:

# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])


# Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho) &amp;amp; (df["branch"] ==
"continue")].iloc[0]


# Get reference targets for both horizon systems

# Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])


# Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])


for branch in branches:

# INCLUDE continue branch for DID calculation

# For continue: compare trajectory with itself, AUC deficit = 0
```

```
# This makes recovery DID well-defined: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]


prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")
```

```
# P0 #3: Validate trajectory lengths match sample_times

if not (len(sample_times) == len(density_trajectory) == len(similarity_trajectory)):

raise RuntimeError(

f"Trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"times={len(sample_times)}, density={len(density_trajectory)},
similarity={len(similarity_trajectory)}"

)



# P0 #3: Also validate that trajectory lengths match between branch and continue

if len(sample_times) != len(cont_sample_times):

raise RuntimeError(

f"Sample times length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"

)

if len(density_trajectory) != len(cont_density_trajectory):

raise RuntimeError(

f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"
```

```
)

if len(similarity_trajectory) != len(cont_similarity_trajectory):

raise RuntimeError(

f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"

)


# P0 #4: Validate exact saved-time schedule

if not np.array_equal(sample_times, EXPECTED_SAMPLE_TIMES):

raise RuntimeError(

f"Unexpected sample-time schedule for seed={seed}, rho={rho}, branch={branch}. "

f"Expected first={EXPECTED_SAMPLE_TIMES[0]}, last={EXPECTED_SAMPLE_TIMES[-1]},
n={EXPECTED_N_SAMPLES}"

)

if not np.array_equal(cont_sample_times, EXPECTED_SAMPLE_TIMES):

raise RuntimeError(

f"Unexpected continue sample-time schedule for seed={seed}, rho={rho}. "

f"Expected first={EXPECTED_SAMPLE_TIMES[0]}, last={EXPECTED_SAMPLE_TIMES[-1]},
n={EXPECTED_N_SAMPLES}"
```

```
)

# P0 #5: Validate trajectory ranges
# Density must be finite and in [0, 1]
if not np.all(np.isfinite(density_trajectory)):
raise RuntimeError(
f"Density trajectory contains non-finite values for seed={seed}, rho={rho}, branch={branch}"
)
if np.any((density_trajectory &amp;lt; 0.0) | (density_trajectory &amp;gt; 1.0)):
raise RuntimeError(
f"Density trajectory contains values outside [0,1] for seed={seed}, rho={rho}, branch={branch}"
)

# Similarity must be finite, and finite values in [-1, 1]
if not np.all(np.isfinite(similarity_trajectory)):
raise RuntimeError(
f"Similarity trajectory contains non-finite values for seed={seed}, rho={rho}, branch={branch}"
)
```

```
finite_similarity = similarity_trajectory[np.isfinite(similarity_trajectory)]

if len(finite_similarity) &gt; 0 and np.any((finite_similarity &lt; -1.0 - 1e-12) | (finite_similarity &gt; 1.0 + 1e-12)):

raise RuntimeError(

f"Similarity trajectory contains values outside [-1,1] for seed={seed}, rho={rho}, branch={branch}"

)


# Validate continue trajectories as well

if not np.all(np.isfinite(cont_density_trajectory)):

raise RuntimeError(

f"Continue density trajectory contains non-finite values for seed={seed}, rho={rho}"

)

if np.any((cont_density_trajectory &lt; 0.0) | (cont_density_trajectory &gt; 1.0)):

raise RuntimeError(

f"Continue density trajectory contains values outside [0,1] for seed={seed}, rho={rho}"

)

if not np.all(np.isfinite(cont_similarity_trajectory)):

raise RuntimeError(

f"Continue similarity trajectory contains non-finite values for seed={seed}, rho={rho}"
```

)

finite_cont_similarity = cont_similarity_trajectory[np.isfinite(cont_similarity_trajectory)]

if len(finite_cont_similarity) > 0 and np.any((finite_cont_similarity < -1.0 - 1e-12) | (finite_cont_similarity > 1.0 + 1e-12)):

raise RuntimeError(

f"Continue similarity trajectory contains values outside [-1,1] for seed={seed}, rho={rho}"

)

Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

Process density recovery

density_values = density_trajectory

density_target = density_target_continue

Calculate first crossing for density

```
density_first_crossing, density_first_observed = first_crossing_time(
sample_times, density_values, density_target
)

# Calculate sustained recovery for density (stays above target for rest of trajectory)
density_sustained, density_sustained_observed = first_time_sustained_for_rest(
sample_times, density_values, density_target
)

# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)
```

```
# Process similarity recovery

similarity_values = similarity_trajectory

similarity_target = similarity_target


# Calculate first crossing for similarity

sim_first_crossing, sim_first_observed = first_crossing_time(

sample_times, similarity_values, similarity_target

)


# Calculate sustained recovery for similarity

sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(

sample_times, similarity_values, similarity_target

)


# Calculate held recovery for similarity

sim_held, sim_held_observed = first_time_held_for_duration(

sample_times, similarity_values, similarity_target, t_window

)
```

```
# Calculate AUC deficit for similarity

sim_auc_deficit = trapezoidal_auc_deficit(

sample_times, cont_similarity_trajectory, similarity_values

)


# P1 #10: Calculate discrete held-duration parameters

# sample_step is the diagnostic sample spacing (0.5 from EXPECTED_SAMPLE_TIMES)

sample_step = float(sample_times[1] - sample_times[0]) if len(sample_times) &gt; 1 else float('nan')

required_samples = max(1, int(np.ceil(t_window / sample_step))) if np.isfinite(sample_step) and
sample_step &gt; 0 else 0


# Calculate restricted times (horizon for censored, event_time for observed)

# Use the last sample time as the horizon (not the elapsed difference)

horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')


# P1 #9: Count NaN samples for cosine zero-imputation documentation

density_nan_count = int(np.sum(~np.isfinite(density_trajectory)))

similarity_nan_count = int(np.sum(~np.isfinite(similarity_trajectory)))

cont_similarity_nan_count = int(np.sum(~np.isfinite(cont_similarity_trajectory)))
```

```
# For density metrics

density_restricted_first = density_first_crossing if density_first_observed else horizon_time

density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time

density_restricted_held = density_held if density_held_observed else horizon_time


# For similarity metrics

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time


# Calculate normalized times (t_rec / T_window)

# Raw observed times (conditional on recovery)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window &gt; 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window &gt; 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window &gt; 0 else float('nan')


sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window &gt; 0 else float('nan')
```

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Censoring-aware restricted normalized times (cohort-level estimates)

These use restricted times (horizon for censored, event_time for observed) divided by T_window

density_restricted_first_over_Twindow = density_restricted_first / t_window if np.isfinite(density_restricted_first) and t_window > 0 else float('nan')

density_restricted_sustained_over_Twindow = density_restricted_sustained / t_window if np.isfinite(density_restricted_sustained) and t_window > 0 else float('nan')

density_restricted_held_over_Twindow = density_restricted_held / t_window if np.isfinite(density_restricted_held) and t_window > 0 else float('nan')

sim_restricted_first_over_Twindow = sim_restricted_first / t_window if np.isfinite(sim_restricted_first) and t_window > 0 else float('nan')

sim_restricted_sustained_over_Twindow = sim_restricted_sustained / t_window if np.isfinite(sim_restricted_sustained) and t_window > 0 else float('nan')

sim_restricted_held_over_Twindow = sim_restricted_held / t_window if np.isfinite(sim_restricted_held) and t_window > 0 else float('nan')

Add rows for density metrics

Note: For continue branch, AUC deficit = 0 (comparing trajectory with itself)

```
# P1 #11: Density targets are horizon-independent (same 0.9 * D_B_final_continue for both systems)

# P1 #10: Held duration definition is discrete: ceil(T_window / diagnostic_sample_spacing)
consecutive saved samples

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "ceil(T_window / diagnostic_sample_spacing) consecutive saved samples",

"held_required_samples": required_samples,

"diagnostic_sample_spacing": sample_step,

"auc_horizon": "full_future",

"first_crossing_time": density_first_crossing,
```

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,

"held_time": density_held,

"held_observed": density_held_observed,

"restricted_first_crossing_time": density_restricted_first,

"restricted_sustained_time": density_restricted_sustained,

"restricted_held_time": density_restricted_held,

"auc_deficit_vs_continue": density_auc_deficit,

"first_crossing_over_Twindow": density_first_over_Twindow,

"sustained_time_over_Twindow": density_sustained_over_Twindow,

"held_time_over_Twindow": density_held_over_Twindow,

"restricted_first_over_Twindow": density_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": density_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": density_restricted_held_over_Twindow,

})

Add rows for similarity metrics

```
# Similarity targets are horizon-dependent (different for fixed_physical vs normalized_EMA)
# P1 #9: Record NaN imputation counts for cosine zero-vector convention
recovery_rows.append({
"seed": seed,
"rho": rho,
"T_window": t_window,
"branch": branch,
"horizon_system": horizon_system,
"metric": "similarity",
"target_value": similarity_target,
"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",
"target_horizon_dependence": "horizon_dependent",
"held_duration_definition": "ceil(T_window / diagnostic_sample_spacing) consecutive saved samples",
"held_required_samples": required_samples,
"diagnostic_sample_spacing": sample_step,
"auc_horizon": "full_future",
"target_nan_samples_imputed_zero": similarity_nan_count,
"reference_nan_samples_imputed_zero": cont_similarity_nan_count,
```

"similarity_nan_convention": "undefined zero-vector cosine imputed as zero for recovery and AUC only",

"density_horizon_duplication_note": "Density recovery rows are duplicated across horizon_system by design. They are horizon-independent and must not be pooled across the two labels.",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

"restricted_first_over_Twindow": sim_restricted_first_over_Twindow,

```
"restricted_sustained_over_Twindow": sim_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": sim_restricted_held_over_Twindow,

})



if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")



# Validate expected count: 40 seeds × 5 rho × 4 branches × 2 metrics × 2 horizon systems = 3200

# Now includes continue branch for DID calculation

expected_count = len(seeds) *len(rho_values)* len(branches) *2* 2

if len(recovery_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "

f"This may indicate a logic error in recovery calculation."

)



recovery_df = pd.DataFrame(recovery_rows)
```

print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "

f"Systems: {recovery_df['horizon_system'].nunique()}")

return recovery_df

==
Stage 1: Validation
==

def validate_run(

text

csv_path: str | Path,

npz_path: str | Path,

manifest_path: str | Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

```
# Convert to Path objects for consistency

csv_path = Path(csv_path)

npz_path = Path(npz_path)

manifest_path = Path(manifest_path)

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results



# P1 #8: Validate manifest study values against prespecified constants

try:

manifest_rho_values = tuple(float(r) for r in manifest["study"]["rho_values"])

manifest_branches = tuple(manifest["protocol"]["branches"])

manifest_n_seeds = len(manifest["randomization"]["paired_seeds"])
```

```
if manifest_rho_values != EXPECTED_RHO_VALUES:
results["status"] = "failed"
results["errors"].append(
f"Manifest rho_values {manifest_rho_values} != expected {EXPECTED_RHO_VALUES}"
)
return results

if manifest_branches != EXPECTED_BRANCHES:
results["status"] = "failed"
results["errors"].append(
f"Manifest branches {manifest_branches} != expected {EXPECTED_BRANCHES}"
)
return results

if manifest_n_seeds != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
```

```
f"Manifest has {manifest_n_seeds} seeds, expected {EXPECTED_N_SEEDS}"

)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate manifest study values: {e}")

return results



# Validate condition status - MANDATORY for production

condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results



try:

condition_df = [pd.read](http://pd.read)_csv(condition_status_path)
```

```
# Check for duplicate condition-status rows
if condition_df.duplicated(subset=["seed", "rho"]).any():
results["status"] = "failed"
results["errors"].append("Condition status file has duplicate (seed, rho) rows")
return results

# Validate status values
unique_statuses = set(condition_df["status"].unique())
valid_statuses = {"success", "failed"}
invalid_statuses = unique_statuses - valid_statuses
if invalid_statuses:
results["status"] = "failed"
results["errors"].append(
f"Condition status file has invalid status values: {invalid_statuses}"
)
return results

successful_conditions = (condition_df["status"] == "success").sum()
```

```
failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)


expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:

results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "

f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)
```

```
return results


# Validate exact condition coverage matches manifest

condition_df["seed"] = condition_df["seed"].astype(str)

condition_df["rho"] = [pd.to](http://pd.to)_numeric(condition_df["rho"], errors="coerce")


observed_conditions = set(

zip(condition_df["seed"], condition_df["rho"])

)


# Get expected conditions from manifest

try:

manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]

expected_conditions_from_manifest = {

(str(seed), float(rho))

for seed in manifest_seeds

for rho in manifest_rho_values
```

```
}

if observed_conditions != expected_conditions_from_manifest:
missing = expected_conditions_from_manifest - observed_conditions
extra = observed_conditions - expected_conditions_from_manifest
if missing:
results["status"] = "failed"
results["errors"].append(
f"Condition status missing {len(missing)} conditions: {list(missing)[:5]}"
)
return results
if extra:
results["status"] = "failed"
results["errors"].append(
f"Condition status has {len(extra)} extra conditions: {list(extra)[:5]}"
)
return results
except Exception as e:
```

```
results["status"] = "failed"

results["errors"].append(

f"Could not validate exact condition cohort against manifest: {e}"

)

return results


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")

return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":
```

```
results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status

completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS
```

```
# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results
```

```
# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)
```

```
# Validate unique seeds

unique_seeds = df["seed"].unique()

if len(unique_seeds) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"

)



# Validate rho values

unique_rho = sorted([float(r) for r in df["rho"].unique()])

expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])

if unique_rho != expected_rho:

results["status"] = "failed"

results["errors"].append(

f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"

)
```

```
# Validate branches

unique_branches = sorted(df["branch"].unique())

expected_branches = sorted(EXPECTED_BRANCHES)

if unique_branches != expected_branches:

results["status"] = "failed"

results["errors"].append(

f"Branches mismatch: expected {expected_branches}, got {unique_branches}"

)



# Validate completeness: each seed/rho/branch combination should exist

missing_combinations = []

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:
```

```
if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)
```

```
# Validate exact cohort coverage in CSV matches manifest

try:

manifest_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

manifest_rho_values = [float(r) for r in manifest["study"]["rho_values"]]

manifest_branches = manifest["protocol"]["branches"]


expected_csv_conditions = {

(str(seed), float(rho), branch)

for seed in manifest_seeds

for rho in manifest_rho_values

for branch in manifest_branches

}


observed_csv_conditions = set(

zip(df["seed"], df["rho"], df["branch"])

)


if observed_csv_conditions != expected_csv_conditions:

missing = expected_csv_conditions - observed_csv_conditions
```

```
extra = observed_csv_conditions - expected_csv_conditions

if missing:

results["status"] = "failed"

results["errors"].append(

f"CSV missing {len(missing)} conditions: {list(missing)[:5]}"

)

return results

if extra:

results["status"] = "failed"

results["errors"].append(

f"CSV has {len(extra)} extra conditions: {list(extra)[:5]}"

)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(

f"Could not validate exact CSV cohort against manifest: {e}"
```

```
)

return results



# Validate primary and secondary outcomes are present and finite

all_outcomes = PRIMARY_OUTCOMES + SECONDARY_NORMALIZED_OUTCOMES

for outcome in all_outcomes:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Outcome '{outcome}' has {non_numeric.sum()} non-numeric values"

)
```

```
# Check that values are finite (not NaN, inf, or -inf)

non_finite = ~np.isfinite(df[outcome])

if non_finite.any():

results["status"] = "failed"

results["errors"].append(

f"Outcome '{outcome}' has {non_finite.sum()} non-finite values"

)



# P0 #2: Validate T_window exists and is consistent

# T_window is study metadata, critical for recovery normalization

if "T_window" not in df.columns:

results["status"] = "failed"

results["errors"].append("T_window missing from scalar CSV")

return results

else:

tw = [pd.to](http://pd.to)_numeric(df["T_window"], errors="coerce")

if (~np.isfinite(tw) | (tw &lt;= 0)).any():
```

```
results["status"] = "failed"
results["errors"].append("T_window contains non-finite or non-positive values")
return results

# Validate T_window is consistent across branches for each (seed, rho)
for rho, expected_twindow in zip(
[float(r) for r in manifest["study"]["rho_values"]],
[float(x) for x in manifest["study"]["T_window_values"]]
):
rho_mask = df["rho"] == rho
values = df.loc[rho_mask, "T_window"].unique()
if len(values) != 1:
results["status"] = "failed"
results["errors"].append(
f"rho={rho} has {len(values)} distinct T_window values"
)
return results
```

```
elif float(values[0]) != expected_twindow:

results["status"] = "failed"

results["errors"].append(

f"rho={rho}: T_window={values[0]} != expected {expected_twindow}"

)

return results



# P0 #3: Validate D_B_final exists and is valid

# D_B_final is used for density targets in recovery analysis

if "D_B_final" not in df.columns:

results["status"] = "failed"

results["errors"].append("D_B_final missing from scalar CSV")

return results

else:

density = [pd.to](http://pd.to)_numeric(df["D_B_final"], errors="coerce")

invalid = (~np.isfinite(density) | (density &lt; 0.0) | (density &gt; 1.0))

if invalid.any():
```

```
results["status"] = "failed"

results["errors"].append(

f"D_B_final has {int(invalid.sum())} invalid values (must be in [0,1])"

)

return results



# Get baseline equivalence certificate hash - MANDATORY for production

baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"

if not baseline_cert_path.exists():

results["status"] = "failed"

results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")

return results

baseline_cert_hash = file_sha256(baseline_cert_path)



# Validate baseline certificate contents

try:

with open(baseline_cert_path, "r") as f:
```

```
certificate = json.load(f)

cert_status = certificate.get("status", "unknown")

cert_n_seeds = certificate.get("n_seeds", 0)

cert_seed_results = certificate.get("seed_results", {})


if cert_status != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate status is '{cert_status}', expected 'passed'"

)

return results


if cert_n_seeds != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {cert_n_seeds} seeds, expected {EXPECTED_N_SEEDS}"

)

return results
```

```
if len(cert_seed_results) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {len(cert_seed_results)} seed results, expected {EXPECTED_N_SEEDS}"

)

return results


# P1 #7: Verify baseline certificate cohort matches manifest exactly

try:

expected_cert_seeds = {

str(seed)

for seed in manifest["randomization"]["paired_seeds"]

}

observed_cert_seeds = set(cert_seed_results.keys())


if observed_cert_seeds != expected_cert_seeds:

missing_seeds = expected_cert_seeds - observed_cert_seeds
```

```
extra_seeds = observed_cert_seeds - expected_cert_seeds

if missing_seeds:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate missing {len(missing_seeds)} seeds: {list(missing_seeds)[:5]}"

)

return results

if extra_seeds:

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate has {len(extra_seeds)} extra seeds: {list(extra_seeds)[:5]}"

)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(

f"Could not validate baseline certificate seed cohort: {e}"

)
```

```
return results


for seed_id, seed_result in cert_seed_results.items():

if seed_result.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline certificate seed {seed_id} status is '{seed_result.get('status')}', expected 'passed'"

)

return results

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate baseline certificate contents: {e}")

return results



# P0 #4: Verify simulation output hashes against manifest

# Validate that actual input files match the recorded hashes in the simulation manifest

recorded_hashes = manifest.get("output_hashes", {})
```

```
files_to_verify = {
"scalar_csv": csv_path,
"branch_npz": npz_path,
"condition_status_csv": condition_status_path,
"baseline_certificate": baseline_cert_path,
}

for label, path in files_to_verify.items():
expected_hash = recorded_hashes.get(label)
observed_hash = file_sha256(path)

if expected_hash is None:
results["status"] = "failed"
results["errors"].append(
f"Simulation manifest missing output hash for {label}"
)
return results
```

```
elif observed_hash != expected_hash:

results["status"] = "failed"

results["errors"].append(

f"Hash mismatch for {label}: expected={expected_hash}, observed={observed_hash}"

)

return results



# Record validation info

results["validation"] = {

"n_rows": actual_rows,

"n_seeds": len(unique_seeds),

"n_rho": len(unique_rho),

"n_branches": len(unique_branches),

"n_duplicates": int(duplicate_rows.sum()),

"n_missing_combinations": len(missing_combinations),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", False),
```

"baseline_certificate_hash": baseline_cert_hash,

}

Also expose at top level for manifest generation

results["baseline_certificate_hash"] = baseline_cert_hash

if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho, {len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")

return results

==========
Stage 2: Regime Contrasts
==========

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Uses direct merge instead of pivot to avoid MultiIndex column issues.

For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

Filter to continue branch only

```
cont = df.loc[
df["branch"] == "continue",
["seed", "rho", *outcomes],
].copy()


# Get baseline (rho=1.0) values
baseline = (
cont.loc[cont["rho"] == 1.0]
.drop(columns="rho")
.rename(
columns={
outcome: f"{outcome}_baseline"
for outcome in outcomes
}
)
)
```

```
# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)



# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]
```

```
# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")


rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count
```

```
expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)



print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")



return contrasts_df

================================================================================
Stage 3: Branch Difference-in-Differences
================================================================================

def compute_branch_did(
```

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.

Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []

for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]

```
for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}
```

```
y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c
```

```
branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})
```

```
# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches:
{did_df['branch'].nunique()}")



return did_df

================================================================================
Stage 4: Cross-T Topology and Recovery
================================================================================

def build_continue_b_final_key(seed: str, rho: float) -> str:
```

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: str | Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Convert to Path for consistency

npz_path = Path(npz_path)

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))

Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)

```
topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline



# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)



if rho_key not in npz_data:

missing_keys.append(rho_key)

continue

if baseline_key not in npz_data:

missing_keys.append(baseline_key)

continue
```

```
B_rho = validate_topology_matrix(npz_data[rho_key], rho_key)

B_baseline = validate_topology_matrix(npz_data[baseline_key], baseline_key)


# Compute metrics

cos_sim = cosine_similarity(B_rho, B_baseline)

jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)

norm_dist = normalized_distance(B_rho, B_baseline)


# Validate metric outputs are finite

if not np.isfinite(cos_sim):

raise RuntimeError(f"cosine_similarity({rho_key}, {baseline_key}) returned non-finite value")

if not np.isfinite(jaccard):

raise RuntimeError(f"jaccard_similarity({rho_key}, {baseline_key}) returned non-finite value")

if not np.isfinite(norm_dist):

raise RuntimeError(f"normalized_distance({rho_key}, {baseline_key}) returned non-finite value")


topology_rows.append({

"seed": seed,
```

```
"rho": rho,

"cross_T_cosine": cos_sim,

"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



# P0 #2: Validate topology metrics are all finite
```

```
topology_columns = ("cross_T_cosine", "cross_T_Jaccard", "cross_T_normalized_distance")

for column in topology_columns:

if not np.isfinite(topology_df[column]).all():

raise RuntimeError(

f"Topology metric {column} contains non-finite values"

)



# Validate topology group sizes

topology_sizes = topology_df.groupby("rho").size()

if len(topology_sizes) != 4:

raise RuntimeError(

f"Expected 4 topology rho groups, got {len(topology_sizes)}"

)

if not (topology_sizes == EXPECTED_N_SEEDS).all():

raise RuntimeError(

f"Topology rho groups do not all contain {EXPECTED_N_SEEDS} seeds"

)
```

print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")

return topology_df

def compute_recovery_did(

text

recovery: pd.DataFrame,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

For each seed, non-baseline rho, and NON-CONTINUE branch only, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

Skips continue branch as it generates trivial zero-DID rows.

Expected count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

"""

if recovery.empty:

return pd.DataFrame()

did_rows = []

seeds = sorted(recovery["seed"].unique())

rho_values = sorted([float(r) for r in recovery["rho"].unique()])

horizon_systems = sorted(recovery["horizon_system"].unique())

metrics = sorted(recovery["metric"].unique())

```
# Recovery outcomes to compute DID for

# Prioritize censoring-aware restricted outcomes as primary

recovery_outcomes = [

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

]



# Also compute DID for recovery indicators

indicator_outcomes = [
```

```
"first_crossing_observed",

"sustained_observed",

"held_observed",

]



for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]


for rho in rho_values:

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in NON_CONTINUE_BRANCHES:
```

```
# EXCLUDE continue branch - it generates trivial zero-DID rows

# DID is only defined for non-continue branches: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

for horizon_system in horizon_systems:

for metric in metrics:

# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,ρ,c} (continue branch at same rho)

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;
```

```
(rho_data["horizon_system"] == horizon_system) &amp;amp;
(rho_data["metric"] == metric)
][outcome]

# R_{s,1,b} (baseline rho=1.0, same branch)
r_base_b_match = baseline_data[
(baseline_data["branch"] == branch) &amp;amp;
(baseline_data["horizon_system"] == horizon_system) &amp;amp;
(baseline_data["metric"] == metric)
][outcome]

# R_{s,1,c} (baseline rho=1.0, continue)
r_base_c_match = baseline_data[
(baseline_data["branch"] == "continue") &amp;amp;
(baseline_data["horizon_system"] == horizon_system) &amp;amp;
(baseline_data["metric"] == metric)
][outcome]

# Validate exactly one match for each cell - missing cells indicate schema error
```

```
matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,

"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "

f"horizon={horizon_system}, metric={metric}, "

f"outcome={outcome}, found={len(match)}"

)


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])
```

```
r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,
```

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

Compute DID for recovery indicators (binary outcomes)

for outcome in indicator_outcomes:

Get values

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &

(rho_data["horizon_system"] == horizon_system) &

(rho_data["metric"] == metric)

][outcome]

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &

(rho_data["horizon_system"] == horizon_system) &

```
(rho_data["metric"] == metric)

][outcome]


r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# Validate exactly one match for each cell - missing cells indicate schema error

matches = {

"rho_branch": r_rho_b_match,

"rho_continue": r_rho_c_match,
```

```
"baseline_branch": r_base_b_match,

"baseline_continue": r_base_c_match,

}

for label, match in matches.items():

if len(match) != 1:

raise RuntimeError(

f"Recovery DID requires exactly one {label} row; "

f"seed={seed}, rho={rho}, branch={branch}, "

f"horizon={horizon_system}, metric={metric}, "

f"outcome={outcome}, found={len(match)}"

)


r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])


# For binary outcomes, use difference in proportions
```

```
branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,
```

```
"recovery_difference_in_differences": did,

})



if not did_rows:

return pd.DataFrame()



# Validate exact count: 40 seeds *4 non-baseline rho* 3 non-continue branches *2 systems* 2 metrics * 13 outcomes = 24960

if len(did_rows) != EXPECTED_N_RECOVERY_DID:

raise RuntimeError(

f"Expected exactly {EXPECTED_N_RECOVERY_DID} recovery DID rows, got {len(did_rows)}. "

f"This indicates a logic error - continue branch should be excluded."

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "
```

```
f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")



return did_df

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.



For each outcome and rho, compute:

- Mean contrast
```

- Sample standard deviation

- 95% bootstrap CI

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

```
as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))
```

```
idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)
```

```
# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary

print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")


# Topology summaries
```

```
if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),

cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)


# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)

for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity
```

```
cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))
```

```
idx = topology_summary[topology_summary["rho"] == rho].index
topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower
topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper

# Bootstrap normalized distance
distance_values = group["cross_T_normalized_distance"].dropna().values
if len(distance_values) &gt; 0:
indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))
bootstrap_means = np.mean(distance_values[indices], axis=1)
distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))
distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))

idx = topology_summary[topology_summary["rho"] == rho].index
topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower
topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper

summaries["topology"] = topology_summary
print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")
```

```
# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries

# Include both raw event times AND censoring-aware restricted times

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(

# Raw event times (conditional on observed recovery)

mean_first_crossing=("first_crossing_time", "mean"),

std_first_crossing=("first_crossing_time", "std"),

n_first_crossing=("first_crossing_time", "count"),

mean_sustained=("sustained_time", "mean"),

std_sustained=("sustained_time", "std"),

n_sustained=("sustained_time", "count"),

mean_held=("held_time", "mean"),
```

```
std_held=("held_time", "std"),

n_held=("held_time", "count"),

mean_auc_deficit=("auc_deficit_vs_continue", "mean"),

std_auc_deficit=("auc_deficit_vs_continue", "std"),

n_auc_deficit=("auc_deficit_vs_continue", "count"),


# Censoring-aware restricted times (cohort-level estimates)

mean_restricted_first=("restricted_first_crossing_time", "mean"),

std_restricted_first=("restricted_first_crossing_time", "std"),

n_restricted_first=("restricted_first_crossing_time", "count"),

mean_restricted_sustained=("restricted_sustained_time", "mean"),

std_restricted_sustained=("restricted_sustained_time", "std"),

n_restricted_sustained=("restricted_sustained_time", "count"),

mean_restricted_held=("restricted_held_time", "mean"),

std_restricted_held=("restricted_held_time", "std"),

n_restricted_held=("restricted_held_time", "count"),


# Normalized times (t/T_window) - raw observed (conditional on recovery)
```

```
mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),
std_first_over_Twindow=("first_crossing_over_Twindow", "std"),
n_first_over_Twindow=("first_crossing_over_Twindow", "count"),
mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),
std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),
n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),
mean_held_over_Twindow=("held_time_over_Twindow", "mean"),
std_held_over_Twindow=("held_time_over_Twindow", "std"),
n_held_over_Twindow=("held_time_over_Twindow", "count"),

# Censoring-aware restricted normalized times (cohort-level estimates)
mean_restricted_first_over_Twindow=("restricted_first_over_Twindow", "mean"),
std_restricted_first_over_Twindow=("restricted_first_over_Twindow", "std"),
n_restricted_first_over_Twindow=("restricted_first_over_Twindow", "count"),
mean_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "mean"),
std_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "std"),
n_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "count"),
mean_restricted_held_over_Twindow=("restricted_held_over_Twindow", "mean"),
```

```
std_restricted_held_over_Twindow=("restricted_held_over_Twindow", "std"),

n_restricted_held_over_Twindow=("restricted_held_over_Twindow", "count"),


# Recovery probabilities

observed_rate_first=("first_crossing_observed", "mean"),

observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)


# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))

bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[
(recovery_summary["rho"] == rho) &amp;amp;
(recovery_summary["branch"] == branch) &amp;amp;
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
(recovery_summary["metric"] == metric)
].index
recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower
recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper

# Bootstrap restricted first crossing time (cohort-level, includes censored)
restricted_first_values = group["restricted_first_crossing_time"].dropna().values
if len(restricted_first_values) &amp;gt; 0:
indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))
bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[
```

```
(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper


# Bootstrap restricted sustained time

restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &amp;gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[
```

```
(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper


# Bootstrap restricted held time

restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &amp;gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;
```

```
(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper


# Bootstrap normalized first crossing time over T_window

first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values

if len(first_over_Twindow_values) &amp;gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;
```

```
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values

if len(sustained_over_Twindow_values) &amp;gt; 0:

indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
```

```
(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper


# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))

bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper


# Bootstrap AUC deficit

auc_values = group["auc_deficit_vs_continue"].dropna().values

if len(auc_values) &gt; 0:

indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))

bootstrap_means = np.nanmean(auc_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;

(recovery_summary["metric"] == metric)

].index
```

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper

Bootstrap restricted normalized first crossing time over T_window

restricted_first_over_Twindow_values = group["restricted_first_over_Twindow"].dropna().values

if len(restricted_first_over_Twindow_values) > 0:

indices = rng.integers(0, len(restricted_first_over_Twindow_values), size=(n_bootstrap, len(restricted_first_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[

(recovery_summary["rho"] == rho) &

(recovery_summary["branch"] == branch) &

(recovery_summary["horizon_system"] == horizon_system) &

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first_over_Twindow"] = ci_upper

Bootstrap restricted normalized sustained time over T_window

restricted_sustained_over_Twindow_values =
group["restricted_sustained_over_Twindow"].dropna().values

if len(restricted_sustained_over_Twindow_values) > 0:

indices = rng.integers(0, len(restricted_sustained_over_Twindow_values), size=(n_bootstrap,
len(restricted_sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[

(recovery_summary["rho"] == rho) &

(recovery_summary["branch"] == branch) &

(recovery_summary["horizon_system"] == horizon_system) &

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained_over_Twindow"] = ci_upper

```
# Bootstrap restricted normalized held time over T_window

restricted_held_over_Twindow_values = group["restricted_held_over_Twindow"].dropna().values

if len(restricted_held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_over_Twindow_values), size=(n_bootstrap,
len(restricted_held_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held_over_Twindow"] = ci_upper
```

```
# Bootstrap recovery probabilities (observed rates)

# First crossing observed rate

first_observed_values = group["first_crossing_observed"].dropna().values

if len(first_observed_values) &gt; 0:

indices = rng.integers(0, len(first_observed_values), size=(n_bootstrap, len(first_observed_values)))

bootstrap_means = np.nanmean(first_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_first"] = ci_upper
```

```
# Sustained observed rate

sustained_observed_values = group["sustained_observed"].dropna().values

if len(sustained_observed_values) &gt; 0:

indices = rng.integers(0, len(sustained_observed_values), size=(n_bootstrap,
len(sustained_observed_values)))

bootstrap_means = np.nanmean(sustained_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_sustained"] = ci_upper
```

```
# Held observed rate

held_observed_values = group["held_observed"].dropna().values

if len(held_observed_values) &gt; 0:

indices = rng.integers(0, len(held_observed_values), size=(n_bootstrap, len(held_observed_values)))

bootstrap_means = np.nanmean(held_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_held"] = ci_upper


summaries["recovery"] = recovery_summary
```

```
print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

# ============================================================================
# Report Generation
# ============================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,
```

```
recovery_did_summary: pd.DataFrame | None = None,
) -> str:
text
"""Generate human-readable text report."""
lines = []
lines.append("=" * 80)
lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")
lines.append("=" * 80)
lines.append("")


# Validation summary
lines.append("VALIDATION SUMMARY")
lines.append("-" * 40)
val = validation.get("validation", {})
lines.append(f" Status: {validation.get('status', 'unknown')}")
lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")
lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")
```

```
lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:
```

```
out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)

lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)
```

```
if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No secondary regime contrast data")

lines.append("")
```

```
# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])
```

```
ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):

rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]
```

```
for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")
```

```
lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")

else:

lines.append(" No cross-T topology data")
```

```
lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")

lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")



# Cohort-level estimates (includes censored) - restricted times

lines.append(f" Cohort-level estimates (all seeds, censoring-aware):")
```

```
lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")

lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")

if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")



# Cohort-level normalized times (restricted t/T_window)
```

```
lines.append(f" Cohort-level normalized times (restricted, all seeds):")

lines.append(f" Mean restricted first/t_window: {float(row.get('mean_restricted_first_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_first_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_first_over_Twindow', 0))})")

if "ci_lower_restricted_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first_over_Twindow']):.4f},
{float(row['ci_upper_restricted_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted sustained/t_window:
{float(row.get('mean_restricted_sustained_over_Twindow', float('nan'))):.4f} ±
{float(row.get('std_restricted_sustained_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_sustained_over_Twindow', 0))})")

if "ci_lower_restricted_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained_over_Twindow']):.4f},
{float(row['ci_upper_restricted_sustained_over_Twindow']):.4f}]")

lines.append(f" Mean restricted held/t_window: {float(row.get('mean_restricted_held_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_held_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_held_over_Twindow', 0))})")

if "ci_lower_restricted_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held_over_Twindow']):.4f},
{float(row['ci_upper_restricted_held_over_Twindow']):.4f}]")



# Recovery probabilities with CI
```

```
lines.append(f" Recovery probabilities (with 95% bootstrap CI):")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

if "ci_lower_observed_rate_first" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_first", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_first']):.4f},
{float(row['ci_upper_observed_rate_first']):.4f}]")

lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

if "ci_lower_observed_rate_sustained" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_sustained']):.4f},
{float(row['ci_upper_observed_rate_sustained']):.4f}]")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")

if "ci_lower_observed_rate_held" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_held']):.4f},
{float(row['ci_upper_observed_rate_held']):.4f}]")



# Conditional-on-recovery estimates (observed seeds only)

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")

lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):
```

```
lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")



# Conditional-on-recovery normalized times (observed seeds only)

lines.append(f" Conditional-on-recovery normalized times (observed seeds only):")

lines.append(f" First crossing/t_window: {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Sustained/t_window: {float(row['mean_sustained_over_Twindow']):.4f} ±
{float(row['std_sustained_over_Twindow']):.4f} (n={int(row['n_sustained_over_Twindow'])})")
```

```
if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")

lines.append(f" Held/t_window: {float(row['mean_held_over_Twindow']):.4f} ±
{float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")

if "ci_lower_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held_over_Twindow']):.4f},
{float(row['ci_upper_held_over_Twindow']):.4f}]")

else:

lines.append(" No recovery scaling data")

lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)

lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:
```

```
for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])

n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No recovery DID data")
```

```
lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,

bootstrap_seed: int,

n_bootstrap: int,
```

```
command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),
```

"study_name": STUDY_NAME,

"source": {

"path": str(source_path),

"sha256": source_hash,

},

"command_line": command_line,

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

P1 #17: Add recovery expected counts

"expected_n_recovery_rows": 3200,

"expected_n_recovery_groups": EXPECTED_N_RECOVERY_GROUPS,

"expected_n_recovery_did_rows": EXPECTED_N_RECOVERY_DID,

"expected_n_recovery_did_groups": EXPECTED_N_RECOVERY_DID_GROUPS,

P1 #9, #10, #11: Document conventions

"similarity_nan_recovery_convention": "undefined zero-vector cosine imputed as zero for recovery and AUC only",

"density_horizon_duplication": "Density recovery rows are duplicated across horizon_system by design. They are horizon-independent and must not be pooled across the two labels.",

"discrete_held_duration_convention": "ceil(T_window / diagnostic_sample_spacing) consecutive saved samples",

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

```
"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}



return manifest
```

```
================================================================================
Main Execution
================================================================================

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"run_dir",

type=str,

help="run directory containing study outputs",

)

parser.add_argument(

"--output",

type=str,

default=None,
```

help="output directory for analysis (default: <run_dir>/analysis)",

)

parser.add_argument(

"--n-bootstrap",

type=int,

default=1000,

help="number of bootstrap iterations (default: 1000)",

)

parser.add_argument(

"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

```
help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")

if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")

return True

def main() -> None:

text

args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)
```

```
# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:

print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)
```

```
# Require exactly one input CSV and NPZ

if len(csv_files) != 1:

print(f"Expected exactly one scalar CSV, found {len(csv_files)}: {[[p.name](http://p.name) for p in csv_files]}")

sys.exit(1)

if len(npz_files) != 1:

print(f"Expected exactly one branch NPZ, found {len(npz_files)}: {[[p.name](http://p.name) for p in npz_files]}")

sys.exit(1)

if len(manifest_files) != 1:

print(f"Expected exactly one manifest, found {len(manifest_files)}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# P1 #12: Validate inputs BEFORE deleting stale outputs
```

```
# This prevents malformed runs from causing deletion of valid previous analysis

print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print()



# Stage 1: Validation - MUST HAPPEN BEFORE any file operations

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")
```

```
sys.exit(1)



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)



# Save command line for manifest

command_line = " ".join(sys.argv)



# Now that validation passed, check for and remove stale files

expected_outputs = [

"twindow_paired_regime_contrasts.csv",

"twindow_paired_regime_contrasts_fixed_physical.csv",

"twindow_paired_regime_contrasts_normalized_EMA.csv",

"twindow_branch_interactions.csv",
```

"twindow_branch_interactions_fixed_physical.csv",

"twindow_branch_interactions_normalized_EMA.csv",

"twindow_cross_condition_topology.csv",

"twindow_recovery_by_seed.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_regime_secondary_summary.csv",

"twindow_branch_DID_secondary_summary.csv",

"twindow_recovery_interactions.csv",

"twindow_recovery_interactions_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]

stale_files = []

```
if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)


if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)


# If --overwrite is passed, remove ALL expected files before starting

if stale_files and args.overwrite:

print(f" Removing {len(stale_files)} stale files from output directory...")

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():
```

```
try:
file_path.unlink()
except Exception as e:
print(f" WARNING: Could not remove stale file {expected_file}: {e}")


output_dir.mkdir(parents=True, exist_ok=True)


print(f" Output: {output_dir}")
print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")
print()


if validation["status"] == "failed":
print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")
for error in validation["errors"]:
print(f" {error}")
for warning in validation["warnings"]:
print(f" WARNING: {warning}")
```

```
sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")
```

```
print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"
```

```
# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)



# Stage 3: Branch DID (Primary outcomes - fixed_physical)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"
```

```
# Combine primary and secondary branch DID into long format

branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)



# Stage 4b: Recovery Scaling (REQUIRED for production)

print("\nStage 4b: Computing recovery scaling metrics...")

recovery = compute_recovery_scaling(df, npz_path, b_c)



# Validate recovery groups have exactly 40 seeds each

print("\n Validating recovery group sizes...")

recovery_group_sizes = recovery.groupby(

["rho", "branch", "horizon_system", "metric"]

).size()
```

```
# P0 #5: Validate expected number of recovery groups

if len(recovery_group_sizes) != EXPECTED_N_RECOVERY_GROUPS:

raise RuntimeError(

f"Expected {EXPECTED_N_RECOVERY_GROUPS} recovery groups, "

f"got {len(recovery_group_sizes)}"

)



bad_groups = recovery_group_sizes[recovery_group_sizes != EXPECTED_N_SEEDS]

if not bad_groups.empty:

raise RuntimeError(

f"Recovery groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)



# Validate restricted recovery fields have finite values

print(" Validating restricted recovery fields...")
```

```
for field in RESTRICTED_RECOVERY_FIELDS:
non_finite = ~np.isfinite([pd.to](http://pd.to)_numeric(recovery[field], errors="coerce"))
if non_finite.any():
raise RuntimeError(
f"Recovery field '{field}' has {int(non_finite.sum())} non-finite values"
)


print(" ✅ Recovery validation passed")


# Stage 4c: Recovery Interaction DID
print("\nStage 4c: Computing recovery interaction DID...")
recovery_did = compute_recovery_did(recovery)


# Stage 5: Seed-Level Summaries
print("\nStage 5: Computing seed-level summaries...")
summaries = compute_seed_level_summaries(
regime_contrasts, branch_did, topology, recovery,
```

```
n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")

secondary_summaries = compute_seed_level_summaries(

regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Validate recovery DID groups have exactly 40 seeds each

print("\n Validating recovery DID group sizes...")

recovery_did_group_sizes = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

).size()
```

```
# P0 #5: Validate expected number of recovery DID groups

if len(recovery_did_group_sizes) != EXPECTED_N_RECOVERY_DID_GROUPS:

raise RuntimeError(

f"Expected {EXPECTED_N_RECOVERY_DID_GROUPS} recovery DID groups, "

f"got {len(recovery_did_group_sizes)}"

)



if not (recovery_did_group_sizes == EXPECTED_N_SEEDS).all():

bad_groups = recovery_did_group_sizes[recovery_did_group_sizes != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Recovery DID groups do not all contain {EXPECTED_N_SEEDS} seed-level rows. "

f"Bad groups: {bad_groups.head().to_dict()}"

)

print(" ✅ Recovery DID validation passed")



# Stage 5c: Recovery DID summaries
```

```
print("\nStage 5c: Computing recovery DID summaries...")

recovery_did_summaries = {}

if not recovery_did.empty:

recovery_did_summary = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"],

as_index=False,

).agg(

mean=("recovery_difference_in_differences", "mean"),

std=("recovery_difference_in_differences", "std"),

n=("recovery_difference_in_differences", "count"),

)


# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):

values = group["recovery_difference_in_differences"].dropna().values
```

```
if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;amp;

(recovery_did_summary["branch"] == branch) &amp;amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_did_summary["metric"] == metric) &amp;amp;

(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary

print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")
```

```
regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# P1 #6: Validate summary group counts

print("\n Validating summary group counts...")



# Regime summaries: 4 non-baseline rho * 3 outcomes = 12 groups, each with n=40

if not regime_summaries.empty:

if len(regime_summaries) != 12:

raise RuntimeError(

f"Expected 12 regime summary groups, got {len(regime_summaries)}"
```

```
)

if not (regime_summaries["n"] == EXPECTED_N_SEEDS).all():

bad_groups = regime_summaries[regime_summaries["n"] != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Regime summary groups do not all have n={EXPECTED_N_SEEDS}:
{bad_[groups.to](http://groups.to)_dict()}"

)



# Branch DID summaries: 4 non-baseline rho *3 branches* 3 outcomes = 36 groups, each with n=40

if not did_summaries.empty:

if len(did_summaries) != 36:

raise RuntimeError(

f"Expected 36 branch DID summary groups, got {len(did_summaries)}"

)

if not (did_summaries["n"] == EXPECTED_N_SEEDS).all():

bad_groups = did_summaries[did_summaries["n"] != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Branch DID summary groups do not all have n={EXPECTED_N_SEEDS}:
{bad_groups.head().to_dict()}"
```

```
)



# Topology summary: 4 rho rows, each metric n=40

if not topology_summaries.empty:

if len(topology_summaries) != 4:

raise RuntimeError(

f"Expected 4 topology summary rows (one per non-baseline rho), got {len(topology_summaries)}"

)

for col in ["cosine_n", "jaccard_n", "distance_n"]:

if not (topology_summaries[col] == EXPECTED_N_SEEDS).all():

raise RuntimeError(

f"Topology summary {col} does not all equal {EXPECTED_N_SEEDS}"

)



# Recovery summaries: 80 groups (5 rho *4 branches* 2 systems * 2 metrics)

if not recovery_summaries.empty:

if len(recovery_summaries) != EXPECTED_N_RECOVERY_GROUPS:
```

```
raise RuntimeError(

f"Expected {EXPECTED_N_RECOVERY_GROUPS} recovery summary groups, got
{len(recovery_summaries)}"

)

# Check restricted fields have n=40

for field in ["n_restricted_first", "n_restricted_sustained", "n_restricted_held",

"n_restricted_first_over_Twindow", "n_restricted_sustained_over_Twindow",

"n_restricted_held_over_Twindow", "n_auc_deficit"]:

if field in recovery_summaries.columns:

if not (recovery_summaries[field] == EXPECTED_N_SEEDS).all():

bad_groups = recovery_summaries[recovery_summaries[field] != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Recovery summary {field} does not all equal {EXPECTED_N_SEEDS}:
{bad_groups.head().to_dict()}"

)



# Recovery DID summary: 624 groups

if not recovery_did_summary.empty:
```

```
if len(recovery_did_summary) != EXPECTED_N_RECOVERY_DID_GROUPS:

raise RuntimeError(

f"Expected {EXPECTED_N_RECOVERY_DID_GROUPS} recovery DID summary groups, got {len(recovery_did_summary)}"

)

if not (recovery_did_summary["n"] == EXPECTED_N_SEEDS).all():

bad_groups = recovery_did_summary[recovery_did_summary["n"] != EXPECTED_N_SEEDS]

raise RuntimeError(

f"Recovery DID summary groups do not all have n={EXPECTED_N_SEEDS}: {bad_groups.head().to_dict()}"

)



print(" ✅ All summary group counts validated")



# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}
```

```
# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)

output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")



# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)

regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"

save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")
```

```
# Branch DID (combined primary + secondary)
branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"
save_csv_atomic(branch_did_combined_path, branch_did_combined)
output_files["branch_did_combined"] = branch_did_combined_path
print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")

# Branch DID (primary only - fixed_physical)
branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"
save_csv_atomic(branch_did_path, branch_did)
output_files["branch_did"] = branch_did_path
print(f" Saved: {branch_did_[path.name](http://path.name)}")

# Branch DID (secondary only - normalized_EMA)
branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"
save_csv_atomic(branch_did_secondary_path, branch_did_secondary)
output_files["branch_did_secondary"] = branch_did_secondary_path
```

```
print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)

output_files["recovery_by_seed"] = recovery_raw_path

print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries
```

```
if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")
```

```
# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:

recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")



# Recovery DID (raw)

if not recovery_did.empty:

recovery_did_path = output_dir / "twindow_recovery_interactions.csv"

save_csv_atomic(recovery_did_path, recovery_did)

output_files["recovery_did"] = recovery_did_path

print(f" Saved: {recovery_did_[path.name](http://path.name)}")



# Recovery DID summaries

if not recovery_did_summary.empty:
```

```
recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"

save_csv_atomic(recovery_did_summary_path, recovery_did_summary)

output_files["recovery_did_summary"] = recovery_did_summary_path

print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")



# Secondary summaries

if not regime_summaries_secondary.empty:

regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"

save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)

output_files["regime_summary_secondary"] = regime_summary_secondary_path

print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")



if not did_summaries_secondary.empty:

did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"

save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)

output_files["did_summary_secondary"] = did_summary_secondary_path

print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")
```

```
# Text report
report = generate_text_report(
validation, regime_contrasts, branch_did, topology,
regime_summaries, did_summaries, topology_summaries,
recovery_summaries, regime_summaries_secondary, did_summaries_secondary,
recovery_did_summary
)
report_path = output_dir / "twindow_sensitivity_report.txt"
save_text_atomic(report_path, report)
output_files["report"] = report_path
print(f" Saved: {report_[path.name](http://path.name)}")


# Analysis manifest
analysis_manifest = generate_analysis_manifest(
csv_path, npz_path, manifest_path, validation, output_files,
source_path=Path(__file__),
```

```
bootstrap_seed=args.bootstrap_seed,

n_bootstrap=args.n_bootstrap,

command_line=command_line,

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")
```

print("=" * 80)

if name == "main":

text

main()

Ignoring copy/paste corruption, this version closes nearly all prior analysis-validation gaps. I found two important logical problems, one involving the documented NaN recovery convention and one involving recovery-DID summary counts.
Verdict

Primary/secondary contrasts: correct
Cross-TT topology: strongly validated
Recovery raw table: correct except for contradictory NaN validation
Recovery DID construction: correct
Recovery DID summary validation: currently too strict
Decision: NO-GO for final analysis until the two P0 issues below are fixed

These issues do not affect the baseline-only or trajectory-generation stages. They affect post-grid analysis.
P0 issue 1: similarity NaNs are simultaneously allowed and forbidden

The analysis correctly documents the convention:

undefined zero-vector cosine is imputed as zero for recovery and AUC calculations only.

It also records:

Python

similarity_nan_count = int(
np.sum(~np.isfinite(similarity_trajectory))
)

But immediately before that, it rejects every nonfinite similarity sample:

Python

```
if not np.all(np.isfinite(similarity_trajectory)):
raise RuntimeError(
"Similarity trajectory contains non-finite values ..."
)
```

The same contradiction exists for the continue trajectory.

This means:

the declared zero-imputation convention can never actually be used;
similarity_nan_count can only be zero;
a legitimate undefined cosine from a zero-BB state causes fatal analysis failure.

Even if the current four study branches happen to have finite similarities for all saved samples, the code and estimand definition are internally inconsistent.
Required correction

Permit NaN, reject infinity, and validate only finite values against the cosine range:

Python

```
# Similarity NaN is permitted for a zero-vector cosine.
# Infinity is never permitted.
if np.any(np.isinf(similarity_trajectory)):
raise RuntimeError(
f"Similarity trajectory contains infinity for "
f"seed={seed}, rho={rho}, branch={branch}"
)

finite_similarity = similarity_trajectory[
np.isfinite(similarity_trajectory)
]

if np.any(
```

```
(finite_similarity < -1.0 - 1e-12)
| (finite_similarity > 1.0 + 1e-12)
):
raise RuntimeError(
f"Finite similarity values outside [-1,1] for "
f"seed={seed}, rho={rho}, branch={branch}"
)
```

Apply the same logic to cont_similarity_trajectory:

Python

```
if np.any(np.isinf(cont_similarity_trajectory)):
raise RuntimeError(
f"Continue similarity trajectory contains infinity "
f"for seed={seed}, rho={rho}"
)

finite_cont_similarity = cont_similarity_trajectory[
np.isfinite(cont_similarity_trajectory)
]

if np.any(
(finite_cont_similarity < -1.0 - 1e-12)
| (finite_cont_similarity > 1.0 + 1e-12)
):
raise RuntimeError(...)
```

Keep density entirely finite. Density does not have the zero-vector cosine issue.

This makes the saved facts and analysis convention properly distinct:

```
saved cosine is undefined and represented by NaN;
recovery-only processing imputes that NaN as zero;
the raw trajectory remains unchanged.
```

P0 issue 2: all recovery-DID summary groups are incorrectly required to have n=40

The raw recovery-DID table correctly contains 40 physical seed rows per group. You validate this with:

Python

```
recovery_did_group_sizes == EXPECTED_N_SEEDS
```

That is correct.

But the summary then uses pandas count:

Python

```
n=("recovery_difference_in_differences", "count")
```

which excludes NaN values.

For the three conditional raw recovery outcomes:

text

```
first_crossing_over_Twindow
sustained_time_over_Twindow
held_time_over_Twindow
```

a DID is NaN whenever any required branch/reference cell is censored. Thus their summary n may legitimately be less than 40.

The current check:

Python

```
if not (
recovery_did_summary["n"] == EXPECTED_N_SEEDS
).all():
raise RuntimeError(...)
```

will reject valid conditional analyses whenever any seed is censored.
Required correction

Distinguish full-cohort outcomes from conditional outcomes.

Python

```
FULL_COHORT_RECOVERY_DID_OUTCOMES = {
"restricted_first_crossing_time",
"restricted_sustained_time",
"restricted_held_time",
"restricted_first_over_Twindow",
"restricted_sustained_over_Twindow",
"restricted_held_over_Twindow",
"auc_deficit_vs_continue",
"first_crossing_observed",
"sustained_observed",
"held_observed",
}

CONDITIONAL_RECOVERY_DID_OUTCOMES = {
"first_crossing_over_Twindow",
"sustained_time_over_Twindow",
"held_time_over_Twindow",
}
```

Then:

Python

```
full_cohort = recovery_did_summary[
recovery_did_summary["outcome"].isin(
FULL_COHORT_RECOVERY_DID_OUTCOMES
)
]

if not (full_cohort["n"] == EXPECTED_N_SEEDS).all():
bad = full_cohort[
full_cohort["n"] != EXPECTED_N_SEEDS
```

```
]
raise RuntimeError(
"Full-cohort recovery DID summaries do not "
f"all have n={EXPECTED_N_SEEDS}: "
f"{bad.head().to_dict()}"
)


conditional = recovery_did_summary[
recovery_did_summary["outcome"].isin(
CONDITIONAL_RECOVERY_DID_OUTCOMES
)
]


if np.any(
(conditional["n"] < 0)
| (conditional["n"] > EXPECTED_N_SEEDS)
):
raise RuntimeError(
"Conditional recovery DID summary n lies "
"outside [0, 40]"
)
```

The physical group-size check remains exactly 40 for all 624 groups. Only the finite-value count in conditional summaries may be smaller.

The report already displays n, so the conditional support will remain transparent.
P1 corrections
3. Validate secondary regime and DID summary counts

The new summary checks cover only:

text

regime_summaries
did_summaries

not:

text

regime_summaries_secondary
did_summaries_secondary

Add:

Python

```
if len(regime_summaries_secondary) != 12:
raise RuntimeError(
f"Expected 12 secondary regime summary groups, "
f"got {len(regime_summaries_secondary)}"
)

if not (
regime_summaries_secondary["n"]
== EXPECTED_N_SEEDS
).all():
raise RuntimeError(
"Secondary regime groups do not all have n=40"
)

if len(did_summaries_secondary) != 36:
raise RuntimeError(
f"Expected 36 secondary DID summary groups, "
f"got {len(did_summaries_secondary)}"
)

if not (
did_summaries_secondary["n"]
== EXPECTED_N_SEEDS
).all():
raise RuntimeError(
"Secondary DID groups do not all have n=40"
)
```

4. Validate the baseline certificate branch set

The exact certificate seed cohort is now checked. Also require that its protocol contains all seven frozen baseline branches:

Python

```
EXPECTED_FROZEN_BASELINE_BRANCHES = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)
```

Then:

Python

```
cert_branches = tuple(
certificate.get("protocol", {}).get(
"branches",
[],
)
)

if cert_branches != EXPECTED_FROZEN_BASELINE_BRANCHES:
results["status"] = "failed"
results["errors"].append(
f"Baseline certificate branch set/order "
f"{cert_branches} != expected "
f"{EXPECTED_FROZEN_BASELINE_BRANCHES}"
)
return results
```

This prevents a structurally passed subset certificate from authorizing analysis.
Minor metadata cleanup
5. Put the density-duplication note on density rows

Currently:

Python

"density_horizon_duplication_note": ...

is inserted into similarity rows.

Move it to density rows, or include it in all recovery rows. The analysis manifest already records the convention correctly.
6. density_nan_count is unused

Density is required to be finite, so:

Python

density_nan_count

is always zero and never written. Remove it.
7. Remove the no-op assignment

This line has no effect:

Python

similarity_target = similarity_target

Remove it.
Nonblocking reporting improvement

The report still checks for conditional CIs:

text

ci_lower_sustained
ci_upper_sustained

ci_lower_held
ci_upper_held

that are not calculated. This is harmless because the checks simply skip absent columns.

You may either:

add the conditional sustained/held bootstrap intervals; or
remove those report checks.

Restricted-time and recovery-probability CIs remain the primary censoring-aware summaries.
What is now correctly implemented

The current version successfully closes the previous major gaps:

exact prespecified rho grid and branch set;
exact 40-seed cohort;
exact condition-status coverage;
exact CSV condition coverage;
primary and secondary finite scalar outcomes;
positive and condition-consistent TwindowTwindow;
valid density targets;
simulation output hash verification;
exact baseline certificate cohort;
full topology matrix validation;
finite topology metrics;
four topology groups with 40 seeds each;
exact 640-point trajectory schedule;
matching branch/reference trajectory lengths;
density range validation;
recovery-only NaN convention documentation;
discrete held-duration metadata;
horizon-dependent similarity targets;
horizon-independent density targets;
exact 3,200 recovery rows;
exactly 80 recovery groups;
exactly 24,960 recovery-DID rows;
exactly 624 recovery-DID groups;
40 physical seed rows per group;
fixed-primary and normalized-secondary separation;

input validation before stale-output deletion;
atomic CSV/text/JSON output;
complete analysis provenance;
explicit expected output counts and interpretation constraints.

Final decision

Fix:

the contradictory rejection of similarity NaNs; and
the blanket n=40 requirement for conditional recovery-DID summaries.

After those two changes, I find no remaining substantive blocker to running the post-grid analysis.

Lets take another look at this file:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
```

```
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
P1 #4: Expected frozen baseline branches for certificate validation

EXPECTED_FROZEN_BASELINE_BRANCHES = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",
```

)
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Recovery DID expected count: 40 seeds 4 non-baseline rho 3 non-continue branches 2 systems 2 metrics * 13 outcomes = 24960

EXPECTED_N_RECOVERY_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1) len(NON_CONTINUE_BRANCHES) 2 2 * 13
Recovery groups: 5 rho 4 branches 2 systems * 2 metrics = 80 groups

EXPECTED_N_RECOVERY_GROUPS = len(EXPECTED_RHO_VALUES)
len(EXPECTED_BRANCHES) 2 * 2
Recovery DID groups: 4 rho 3 branches 2 systems 2 metrics 13 outcomes = 624 groups

EXPECTED_N_RECOVERY_DID_GROUPS = (len(EXPECTED_RHO_VALUES) - 1)
len(NON_CONTINUE_BRANCHES) 2 2 13
Censoring-aware fields that must have 40 finite values per group

RESTRICTED_RECOVERY_FIELDS = (

text

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_observed",

"sustained_observed",

"held_observed",

)
P0 Issue 2: Recovery DID outcome classification for n validation
Full-cohort outcomes must have n=40 (all seeds present)

FULL_COHORT_RECOVERY_DID_OUTCOMES = {

text

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_observed",

"sustained_observed",

"held_observed",

}
Conditional outcomes may have n<40 due to censoring (NaN values excluded from count)

CONDITIONAL_RECOVERY_DID_OUTCOMES = {

text

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

}
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
Expected sample time schedule for trajectory validation
640 samples from 0.01 to 319.51 with step 0.5

EXPECTED_SAMPLE_TIMES = (np.arange(0, 32000, 50, dtype=float) + 1.0) * 0.01

EXPECTED_N_SAMPLES = len(EXPECTED_SAMPLE_TIMES)
==========
Utility Functions
==========

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

```
path = Path(path)



# Use pandas with explicit dtypes

df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df
```

```
def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory.



Raises RuntimeError on corrupt NPZ or load failure.

"""

path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():

raise FileNotFoundError(f"NPZ file not found: {path}")



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:
```

```
if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

raise RuntimeError(f"Could not load NPZ {path}: {e}") from e



return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

# Write CSV with explicit flush and fsync for data integrity

with open(temp_path, "w", encoding="utf-8", newline="") as handle:
```

df.to_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

```
temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temp_path, "w", encoding="utf-8") as f:
f.write(text)
f.flush()
os.fsync(f.fileno())
temp_path.replace(path)
# fsync the directory
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
```

```
def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



# Convert NaN and infinity to None for JSON serialization

# Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):
```

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None

return val

elif isinstance(obj, np.ndarray):

Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

```
f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl
```

```
digest = hl.sha256()

path = Path(path)

if not path.exists():

return "FILE_NOT_FOUND"

with open(path, "rb") as f:

for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

============================================================================
====
Metric Functions
============================================================================
====

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.



Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)
```

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) & (values >= threshold))

crossing_indices = np.flatnonzero(above_threshold)

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

if len(times) < 2:

return float('nan'), False

Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step <= 0:

return float('nan'), False

```
# Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))



# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))



# Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False



for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break
```

return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.

Handles NaN values by treating them as zero for the deficit calculation.

Uses np.trapezoid if available, falls back to np.trapz for older NumPy versions.

NaN cosine values (from zero-vector weights) are imputed as zero for recovery

and AUC calculations only - this is a prespecified recovery-only convention.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0 (prespecified recovery-only convention for undefined zero-vector cosine)

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration - use np.trapezoid if available, else np.trapz

if hasattr(np, "trapezoid"):

return float(np.trapezoid(deficit, x=times))

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

```
if union == 0:

return float('nan')



return float(np.count_nonzero(x &amp;amp; y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]



diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)



if norm_y &amp;lt;= 1e-12:

return float('nan')
```

return float(norm_diff / norm_y)

def validate_topology_matrix(matrix: np.ndarray, key: str) -> np.ndarray:

text

"""Validate a topology (B_final) matrix for production analysis.